

Resilience in Microservices With **Congestion Control**



Mourjo Sen

September 16, 2025

<https://mourjo.me>



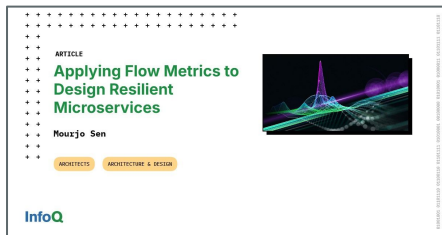
Namaste!

- Born and brought up in India
- 10 years as a software engineer
- Employed at Booking.com



Namaste!

- Born and brought up in India
- 10 years as a software engineer
- Employed at Booking.com
- Writing and speaking for 5 years



Unlocking observability: Structured logging in Spring ...

10 min read · Mar 26, 2024 · [View story](#)

13.6K
Views

Load Shedding in Web Services

10 min read · Sep 4, 2020 · [View story](#)

13.3K
Views

www.youtube.com › watch

Property Based Testing to Preserve Invariants - Fachtalk mit ...

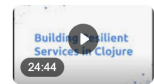


Mourjo Sen spricht in unserem LEGACY & INNOVATION Track beim techCamp über Property-based Testing (PBT). Eine Methode, um die...

YouTube · intersoft GmbH · 1 month ago

www.youtube.com › watch

Building Resilient Services in Clojure



Abstract Building resilient web services in Clojure. By employing strategies like circuit breakers, load shedding, graceful restarts among ...

YouTube · reclojure · 21 déc. 2020

Strategies to build a software observability?



Writing a fair task execution framework — Mourjo Sen

YouTube · IN/Clojure
4 févr. 2019

We Want To

Measure What Matters



We Want To
Measure What Matters

**Customer
Experience**

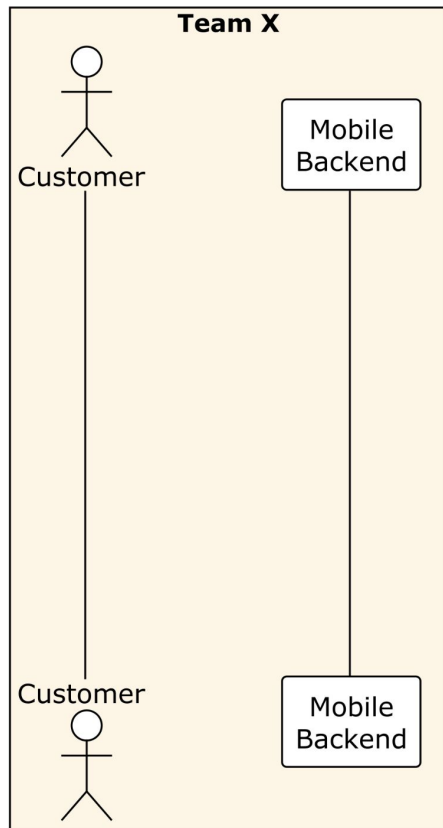




Localized Metrics Miss The Big Picture

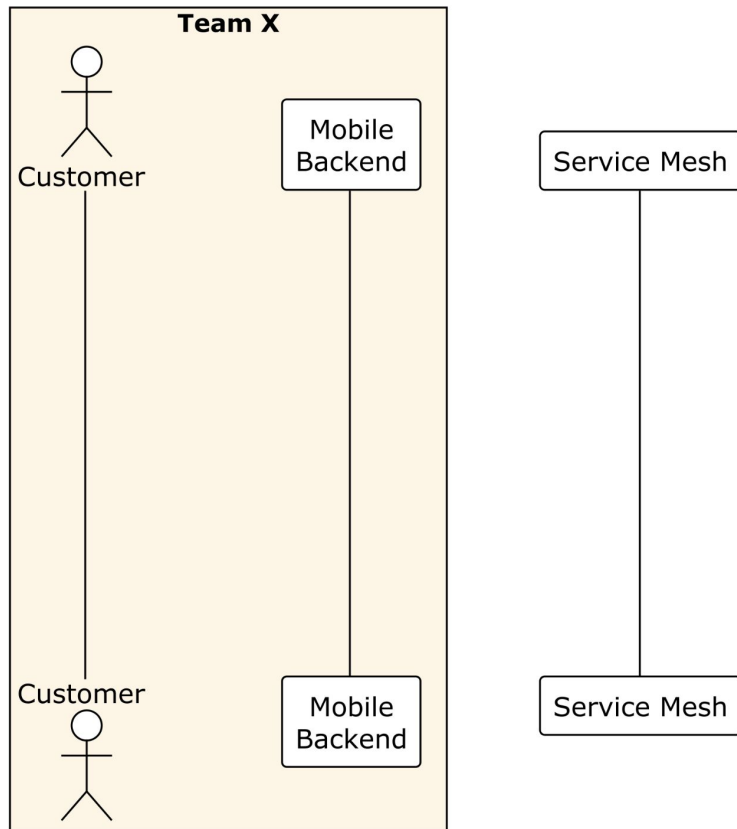


Localized Metrics Miss The Big Picture



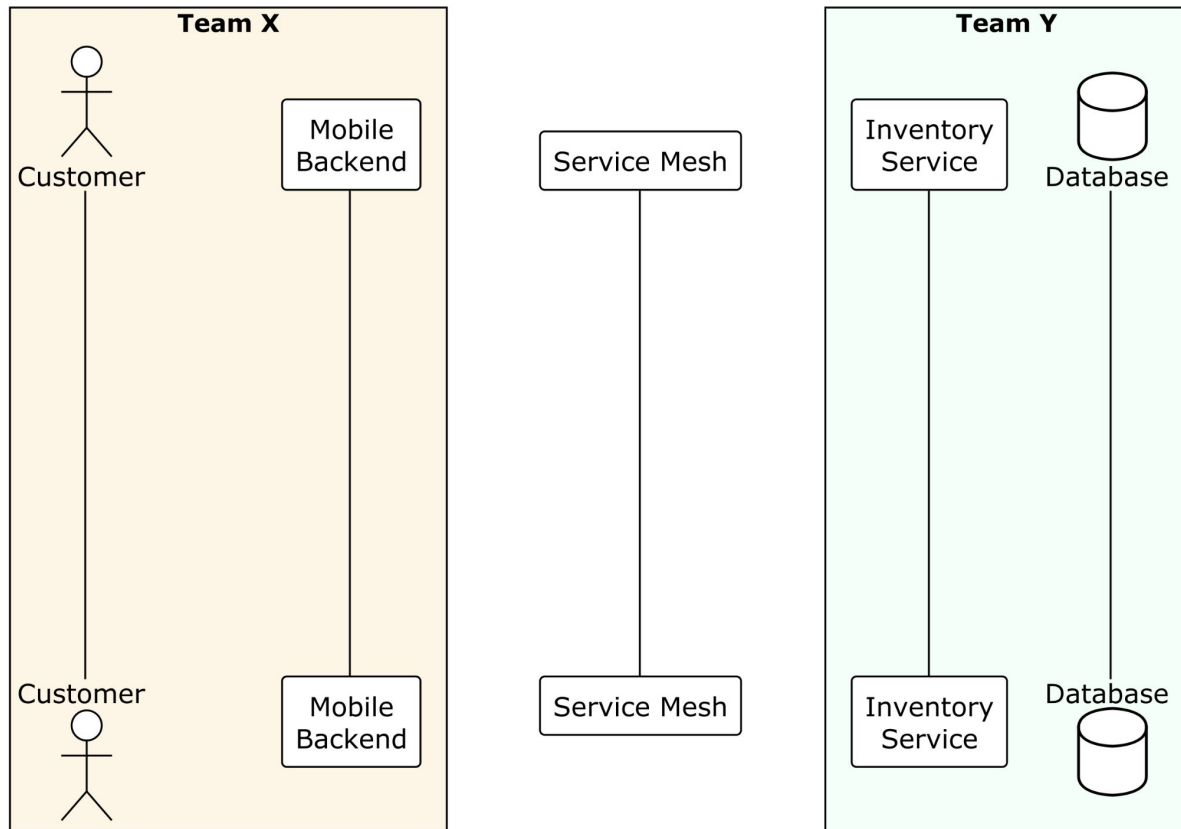


Localized Metrics Miss The Big Picture



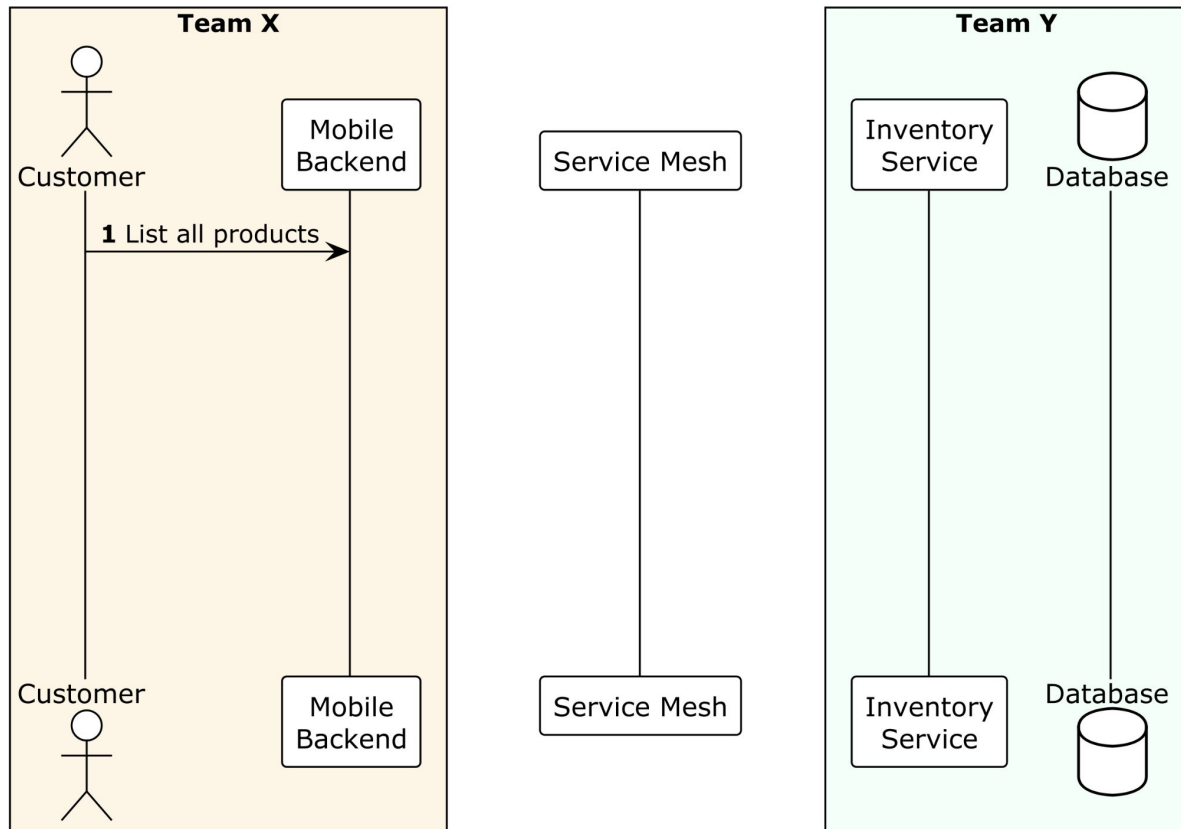


Localized Metrics Miss The Big Picture



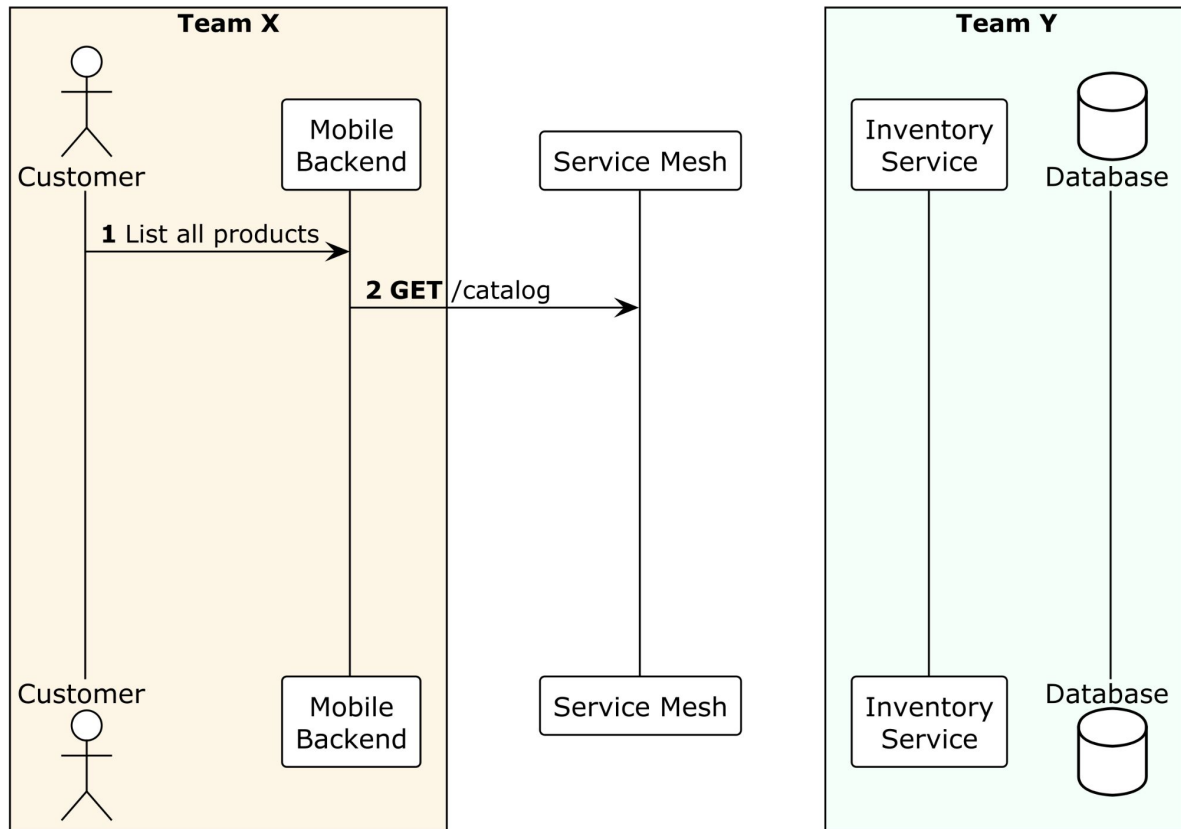


Localized Metrics Miss The Big Picture



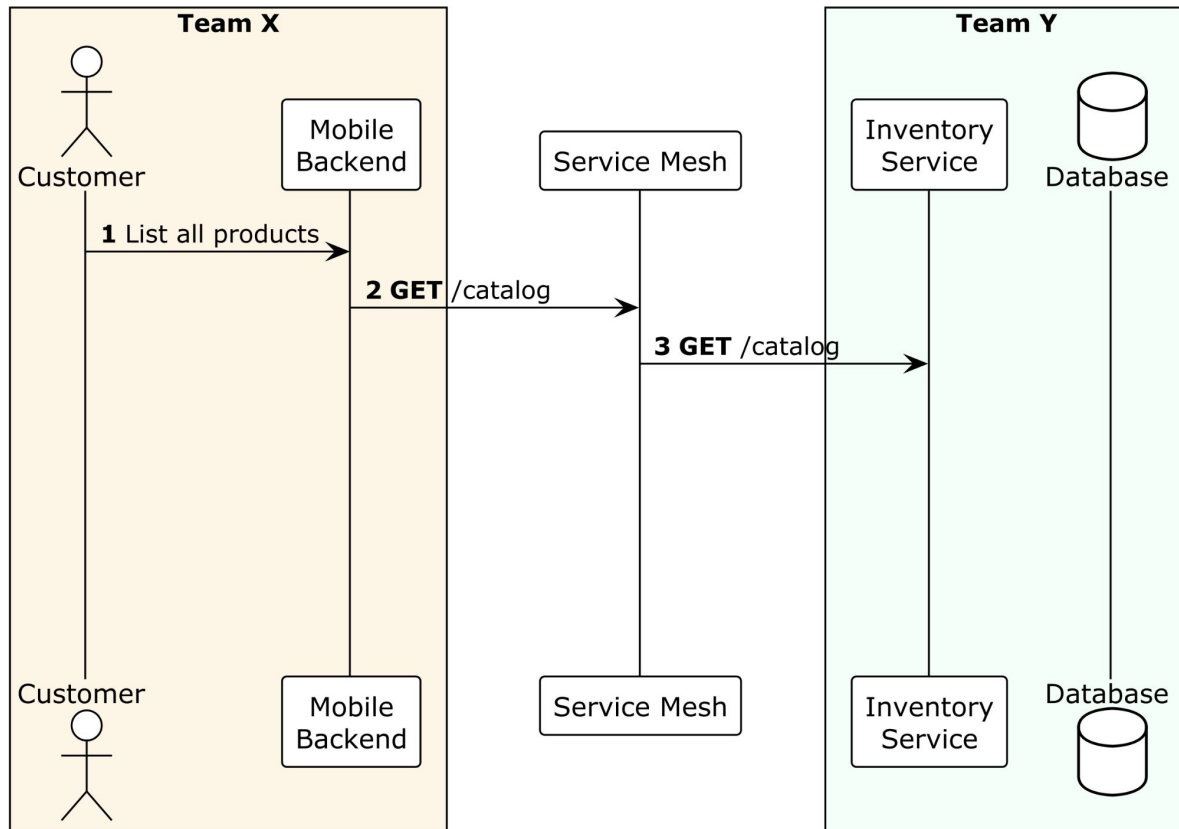


Localized Metrics Miss The Big Picture



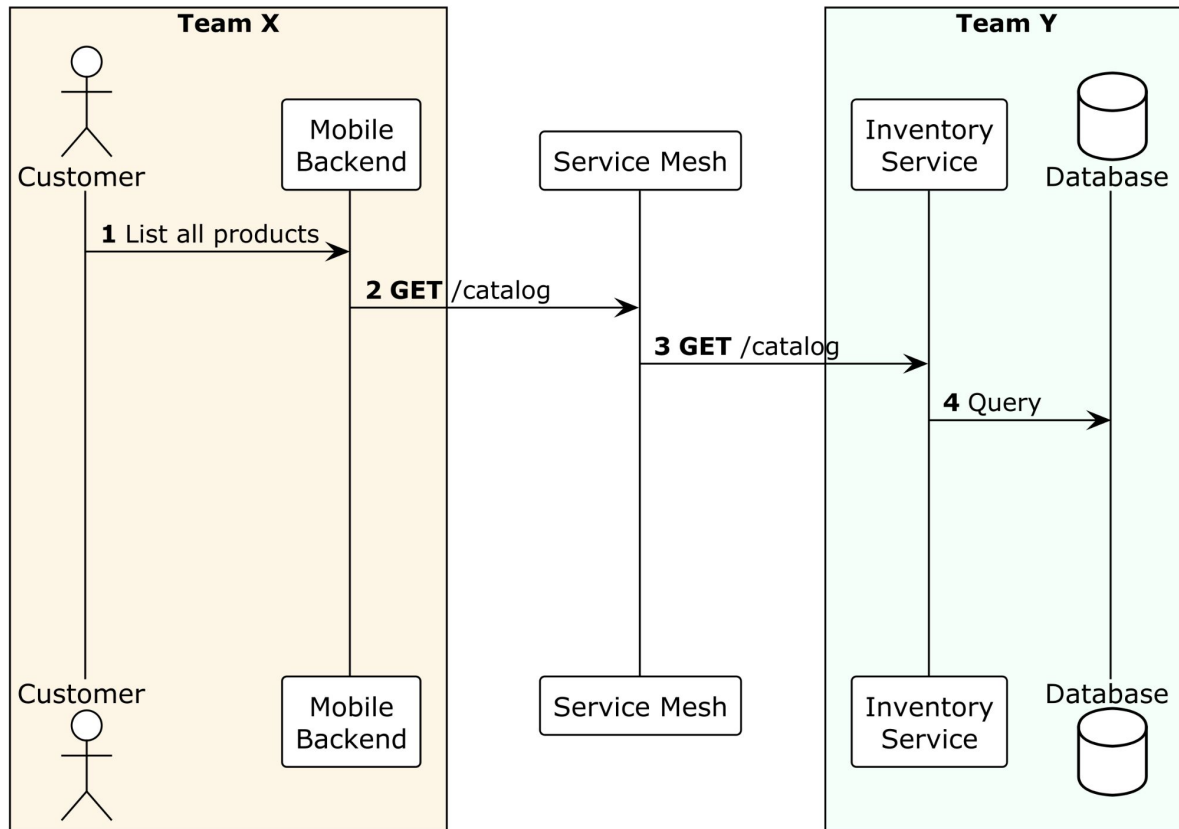


Localized Metrics Miss The Big Picture



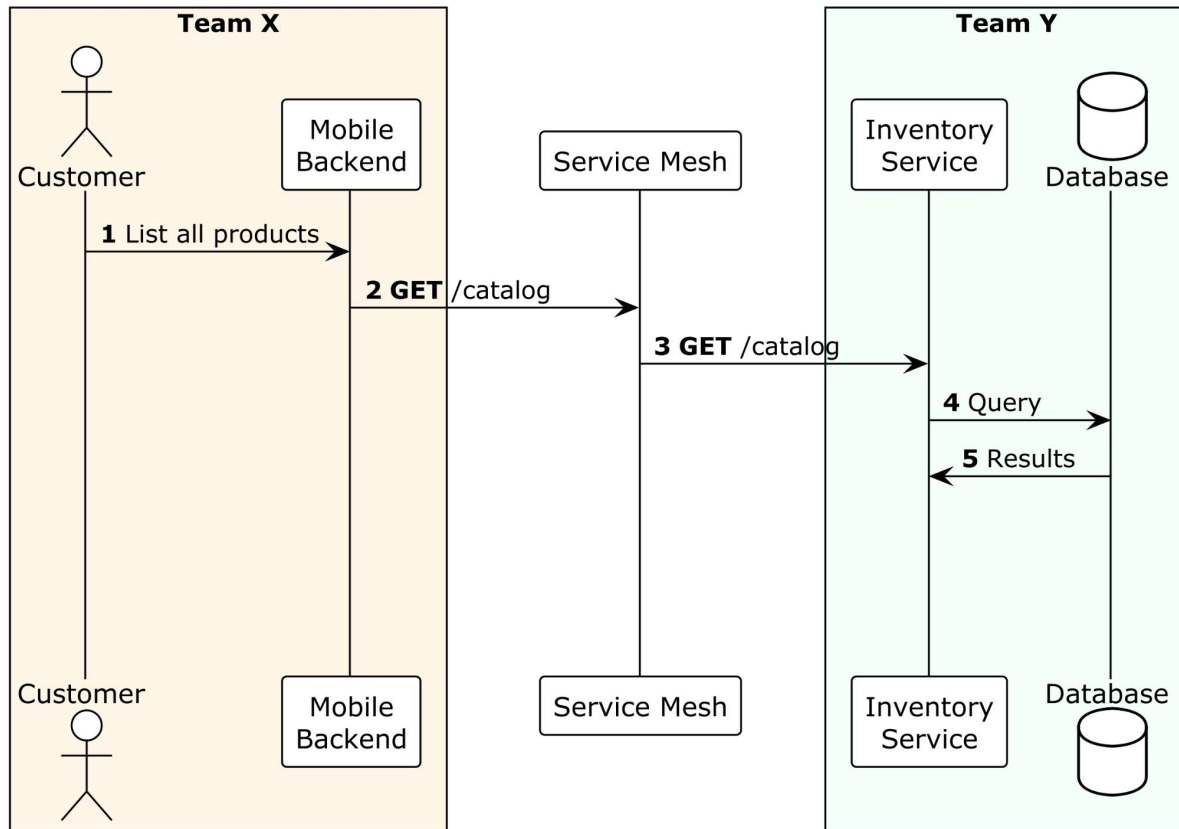


Localized Metrics Miss The Big Picture



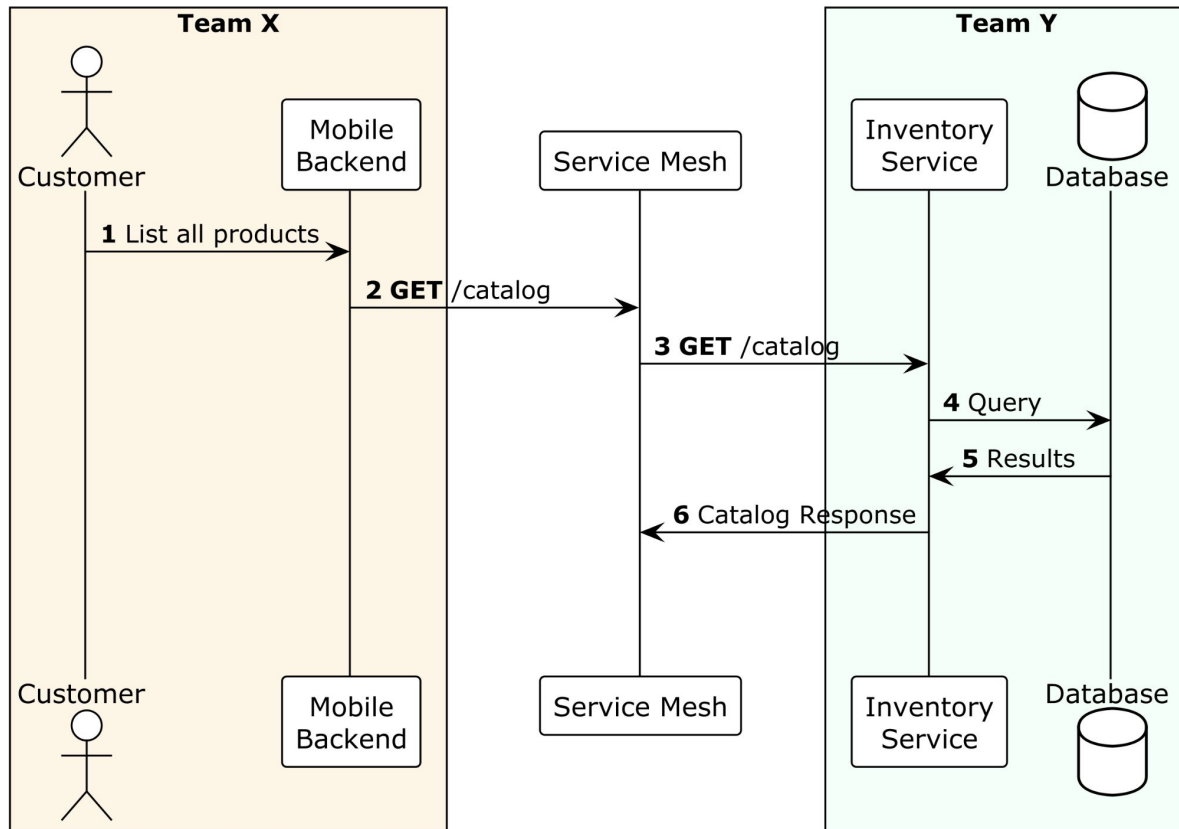


Localized Metrics Miss The Big Picture



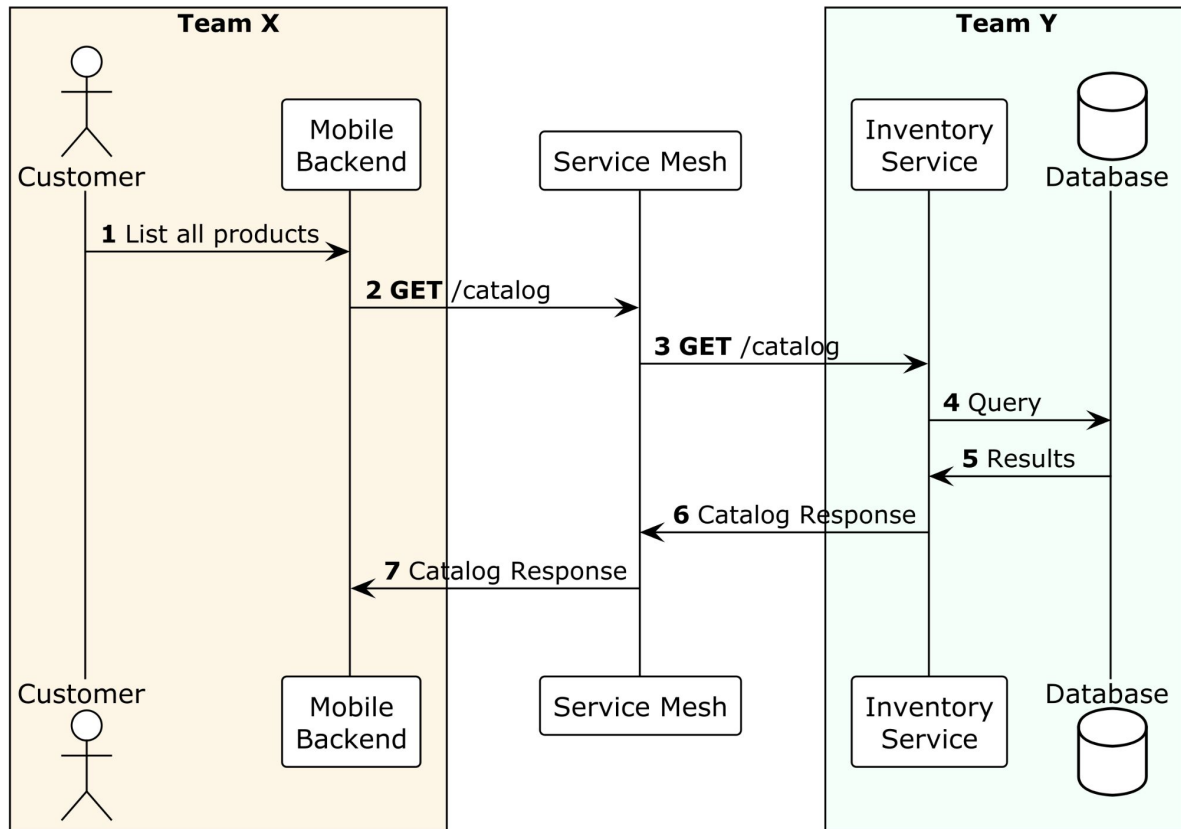


Localized Metrics Miss The Big Picture



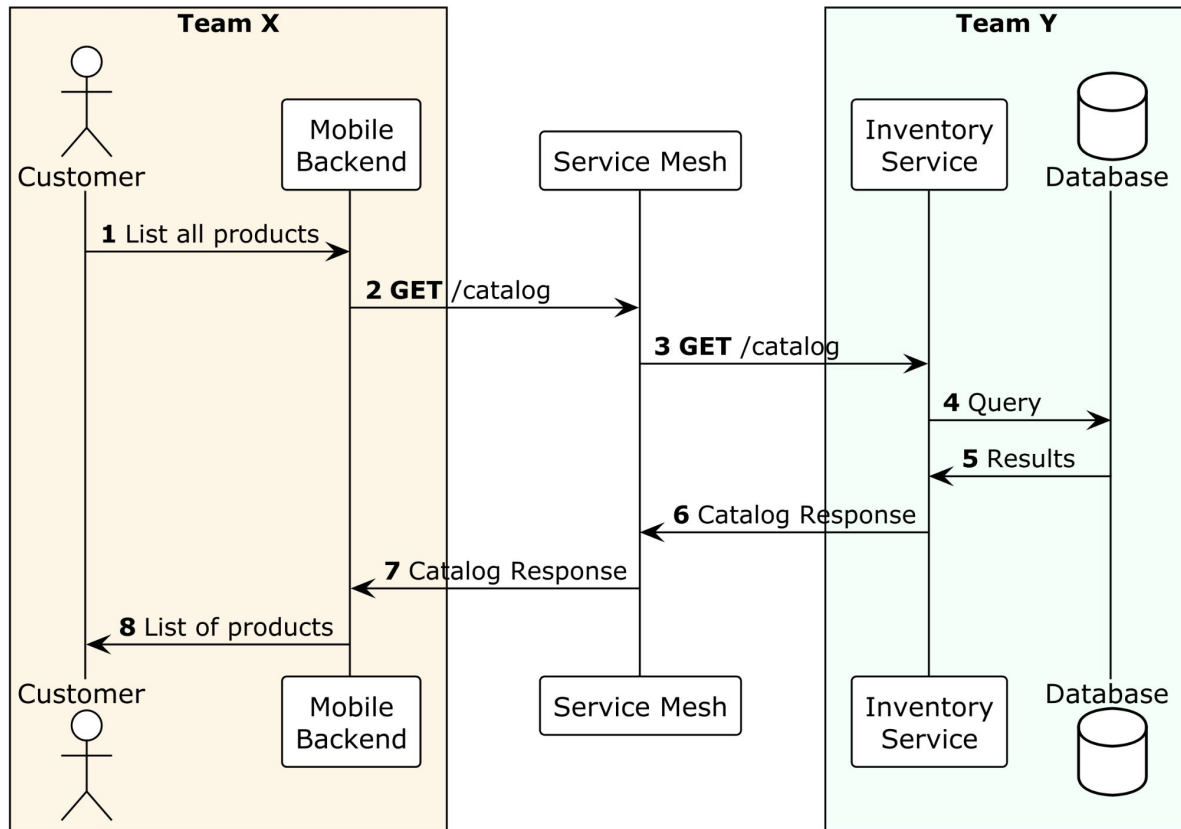


Localized Metrics Miss The Big Picture



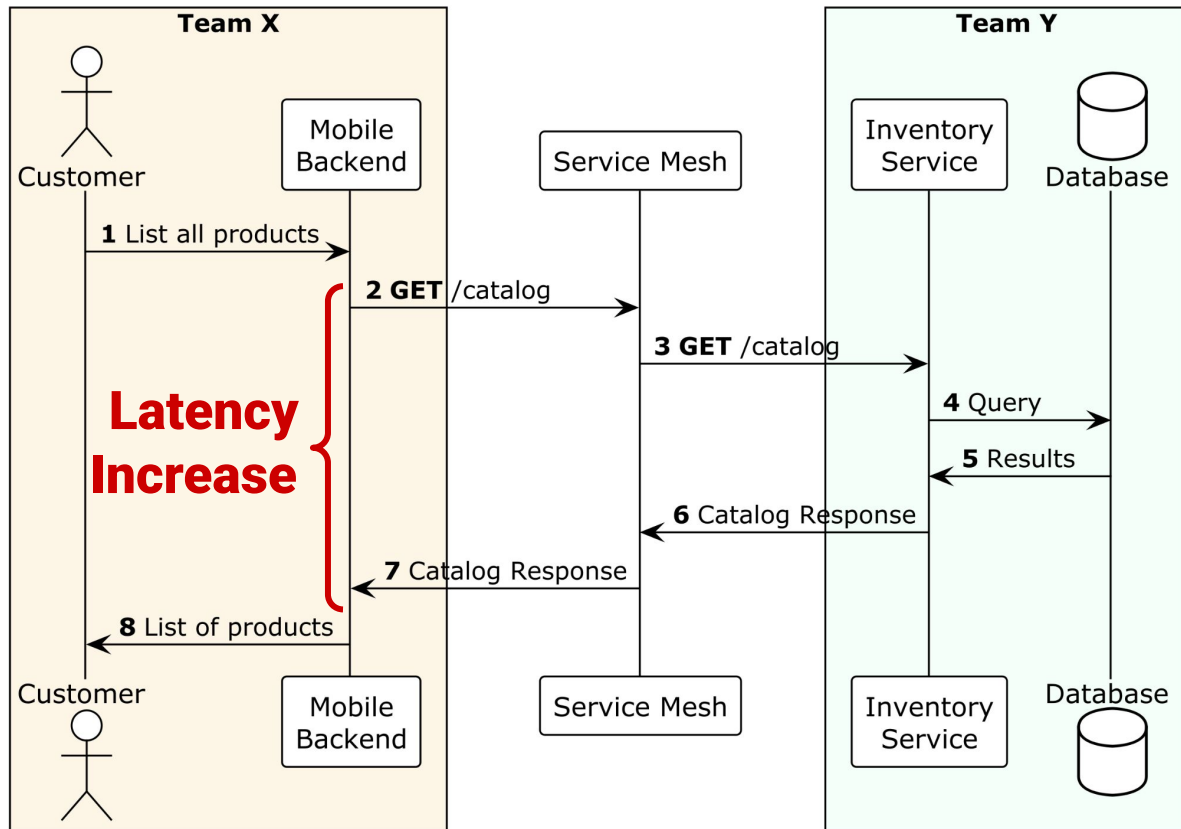


Localized Metrics Miss The Big Picture



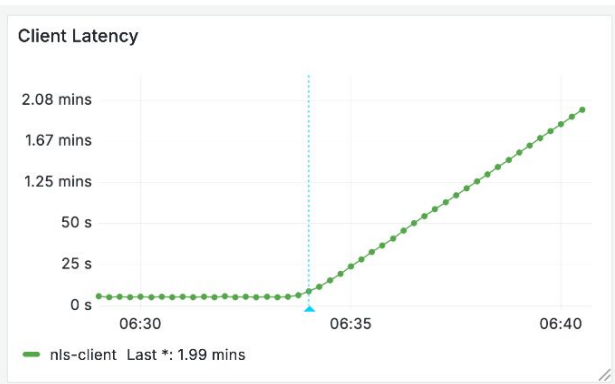


Localized Metrics Miss The Big Picture





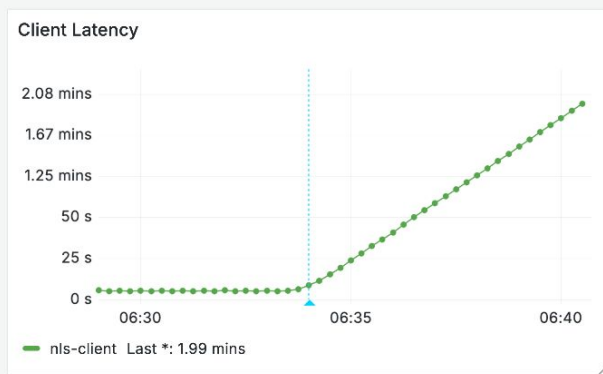
Localized Metrics Miss The Big Picture



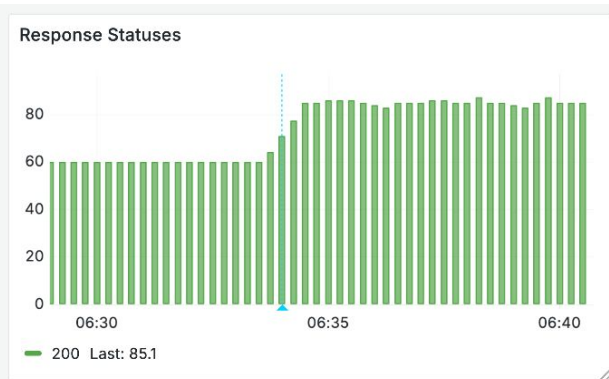
Client latency
increase



Localized Metrics Miss The Big Picture



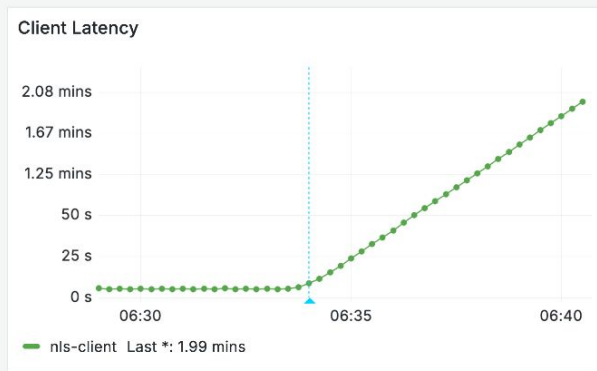
**Client latency
increase**



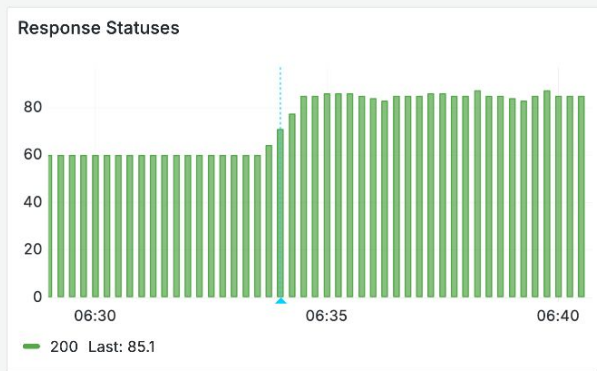
**Healthy responses
from server**



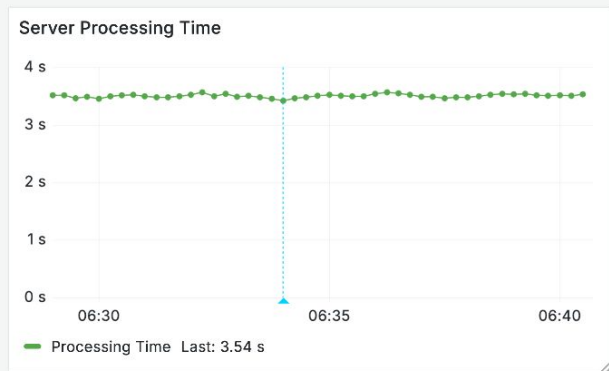
Localized Metrics Miss The Big Picture



Client latency
increase



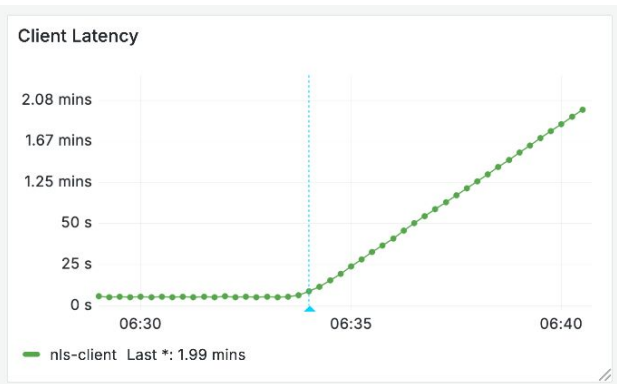
Healthy responses
from server



Server **unaware** of the
degradation



Localized Metrics Miss The Big Picture



Client latency
increase

← **Disconnect** →



Server **unaware** of the
degradation



Localized Metrics Miss The Big Picture

```
@RestController
```

```
public class Controller {
```

```
}
```



Localized Metrics Miss The Big Picture

@RestController

```
public class Controller {
```

```
    public Map<String, String> hello() {
```

```
        return doSomeComputation();
```

```
    }
```

```
}
```



Localized Metrics Miss The Big Picture

```
@RestController
public class Controller {
    @Timed(value = "conduit.http.server.processing.time")
    @GetMapping("/hello")
    public Map<String, String> hello() {
        return doSomeComputation();
    }
}
```



Localized Metrics Miss The Big Picture

```
@RestController
```

```
public class Controller {
```

```
    @Timed(value = "conduit.http.server.processing.time")
```

```
    @GetMapping("/hello")
```

```
    public Map<String, String> hello() {
```

```
        return doSomeComputation();
```

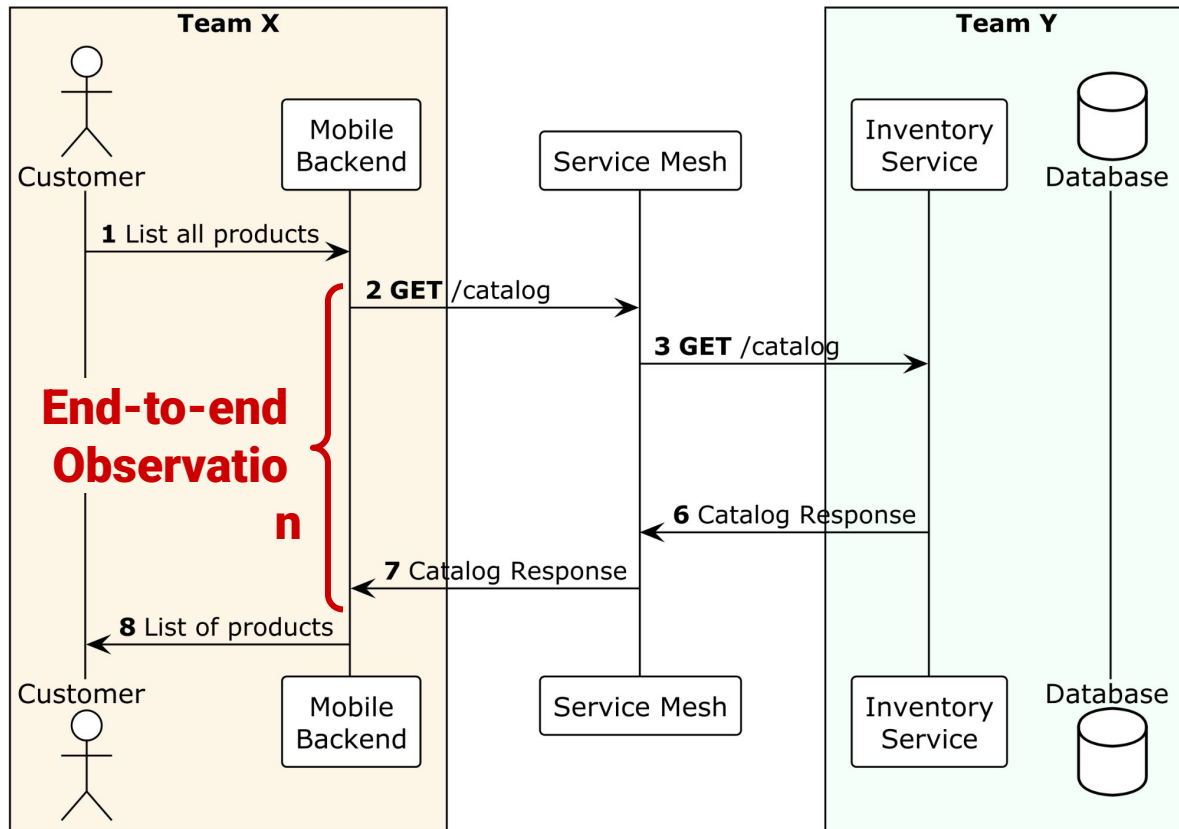
```
    }
```

```
}
```

**Measurement localized to
this method execution**

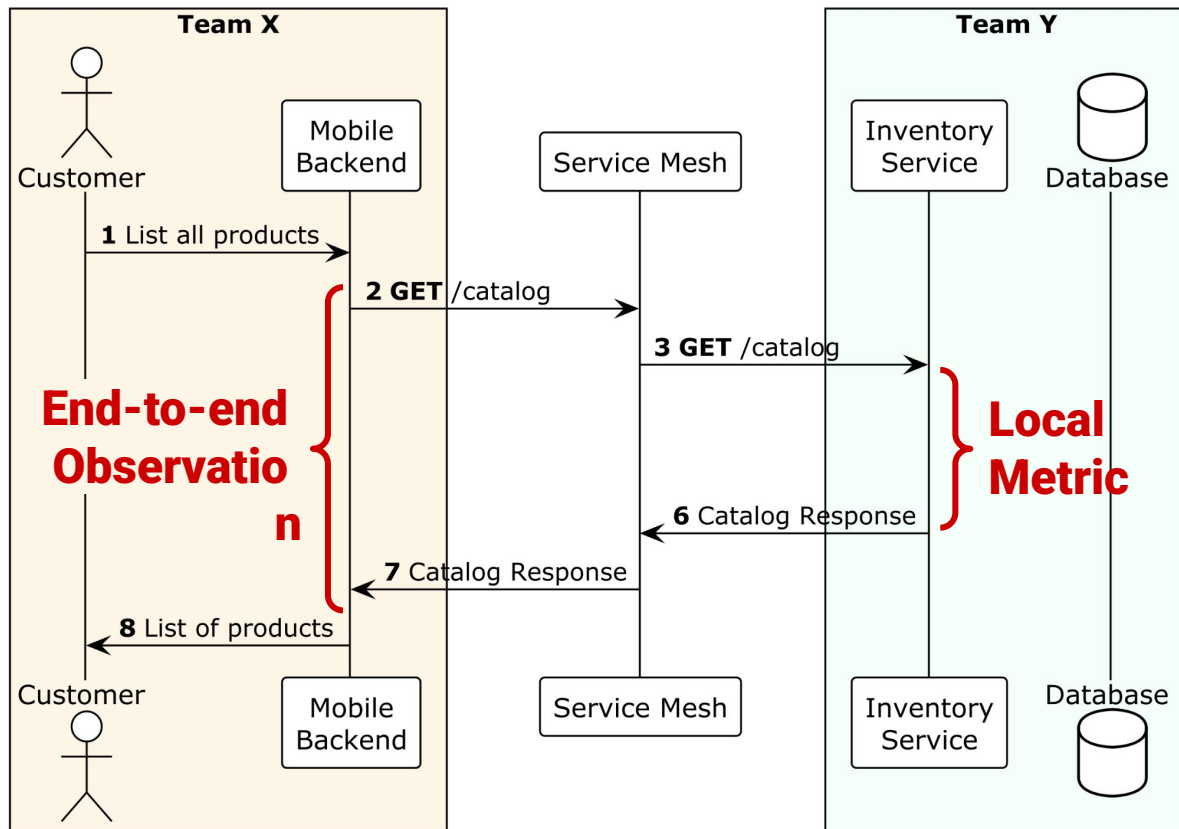


Localized Metrics Miss The Big Picture





Localized Metrics Miss The Big Picture



Overcoming the **Localized View** :

Flow of Requests



Overcoming the **Localized View** :

Flow of **Requests**

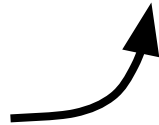
**What matters to
the customer**



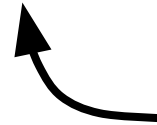
Overcoming the **Localized View** :

Flow of Requests

**End-to-end
value stream**



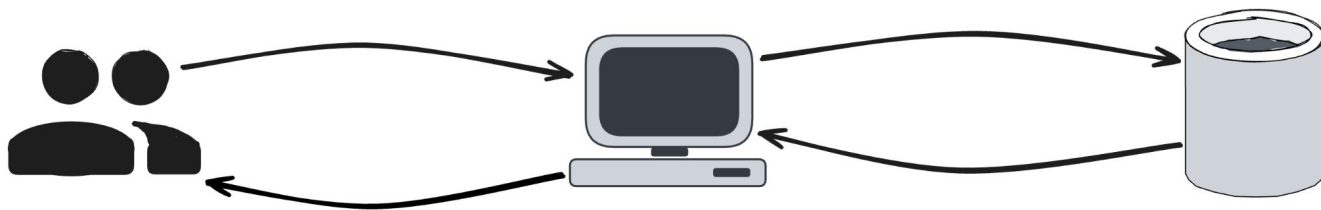
**What matters to
the customer**



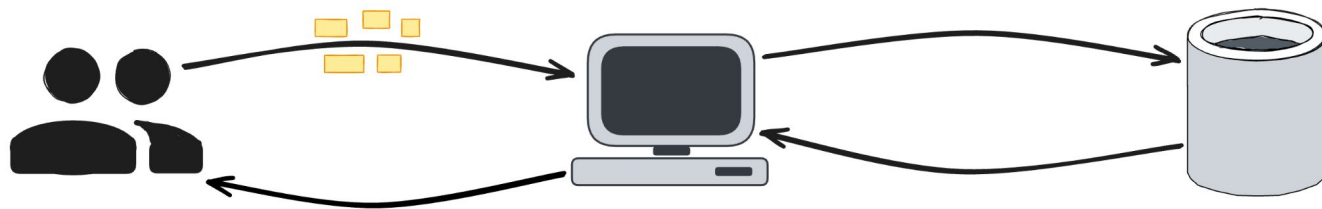


Measuring the Flow of Requests

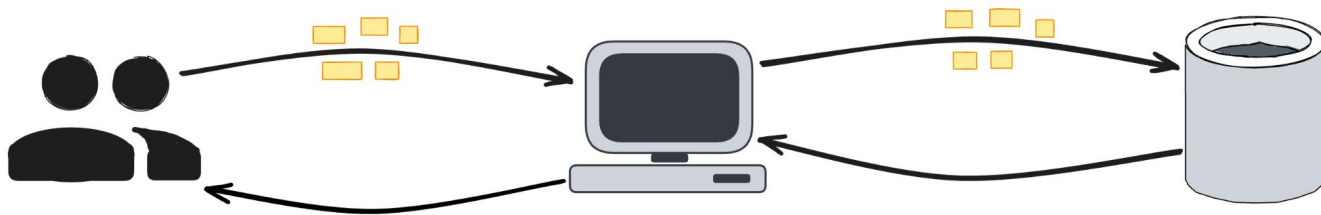
Measuring the Flow of Requests



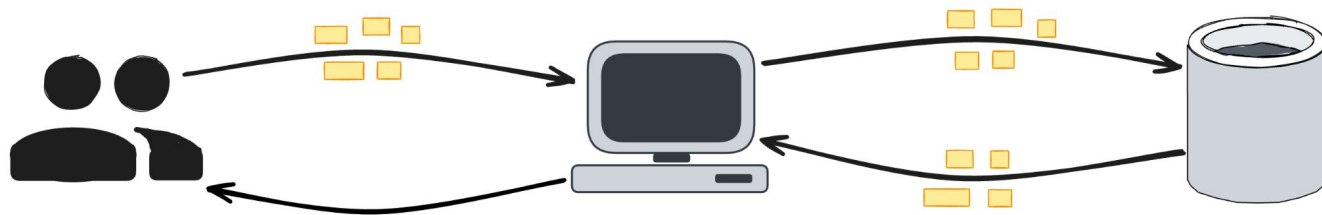
Measuring the Flow of Requests



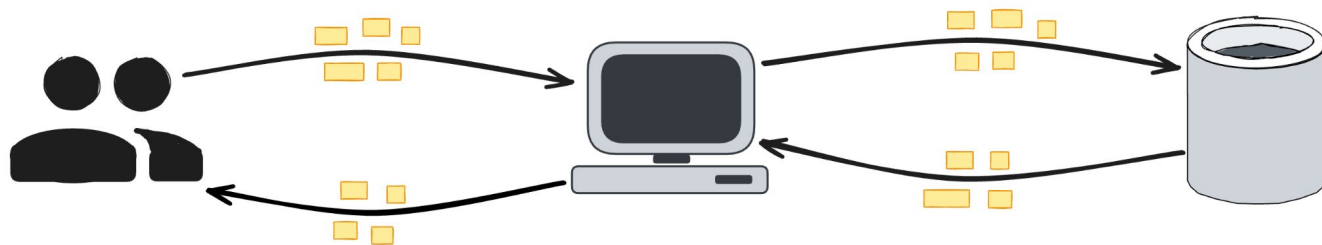
Measuring the Flow of Requests



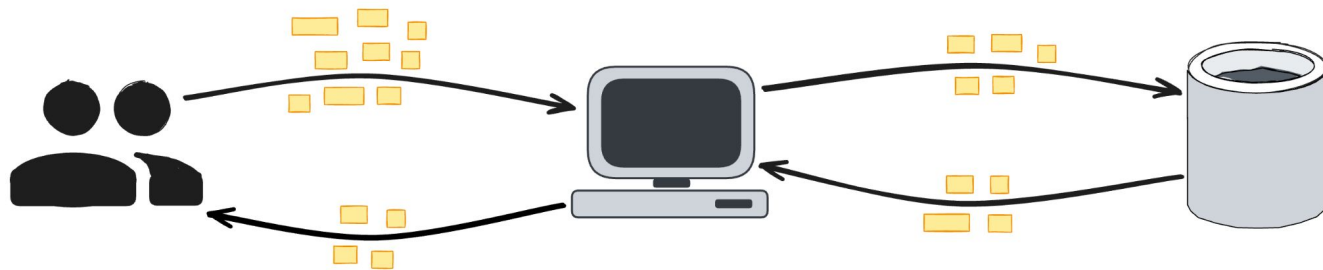
Measuring the Flow of Requests



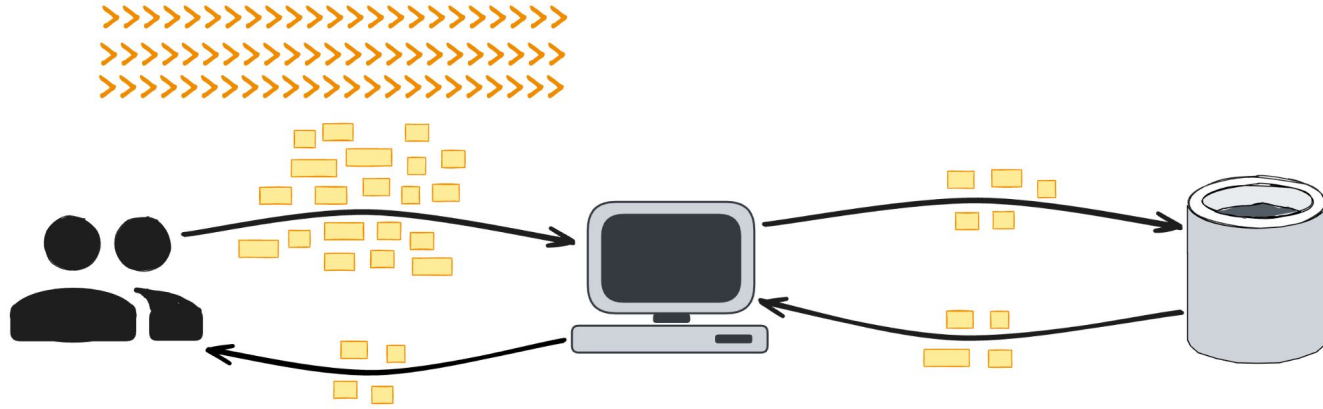
Measuring the Flow of Requests



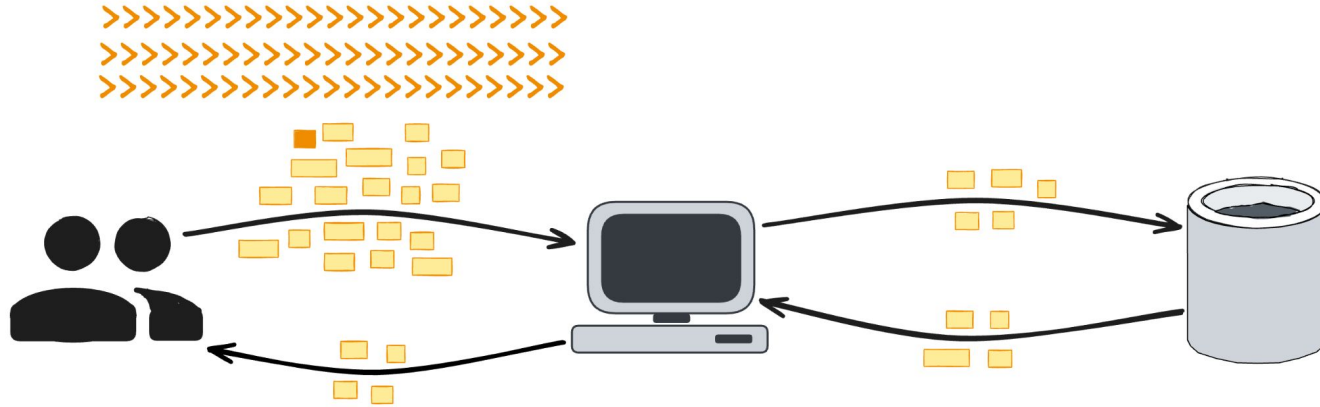
Measuring the Flow of Requests



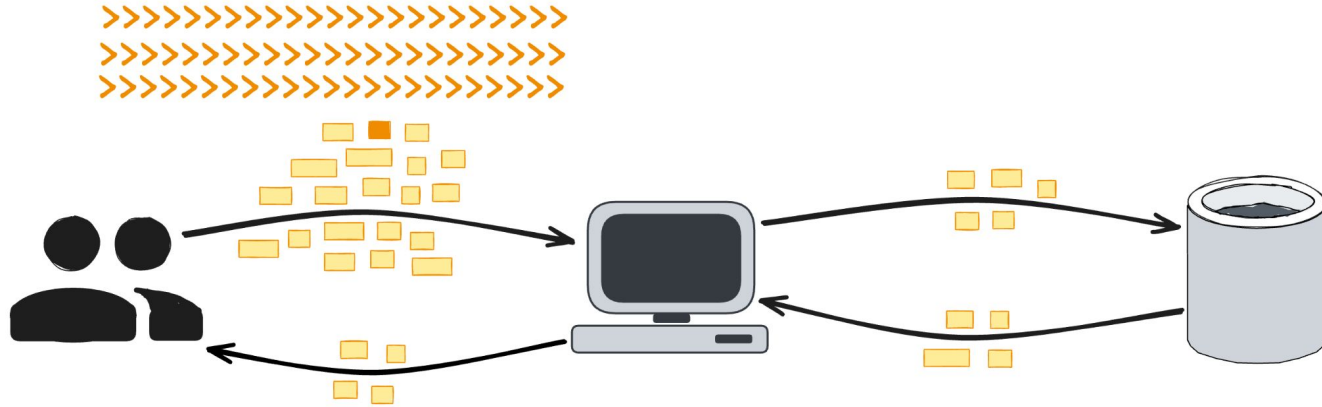
Measuring the Flow of Requests



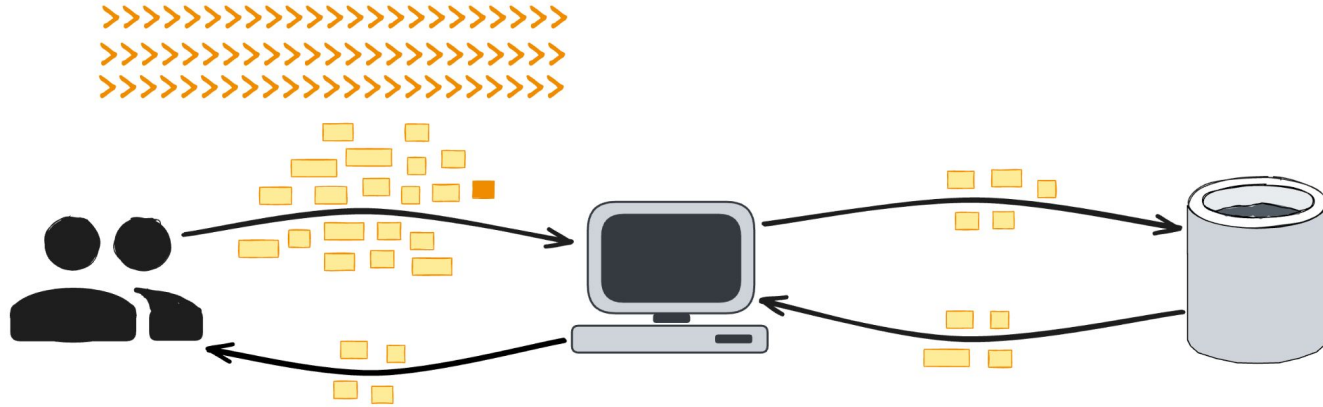
Measuring the Flow of Requests



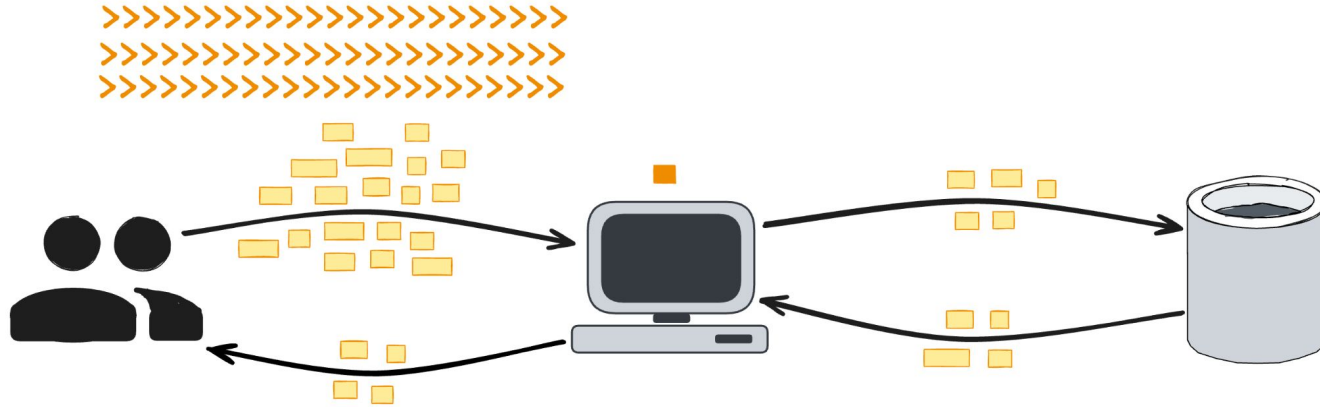
Measuring the Flow of Requests



Measuring the Flow of Requests

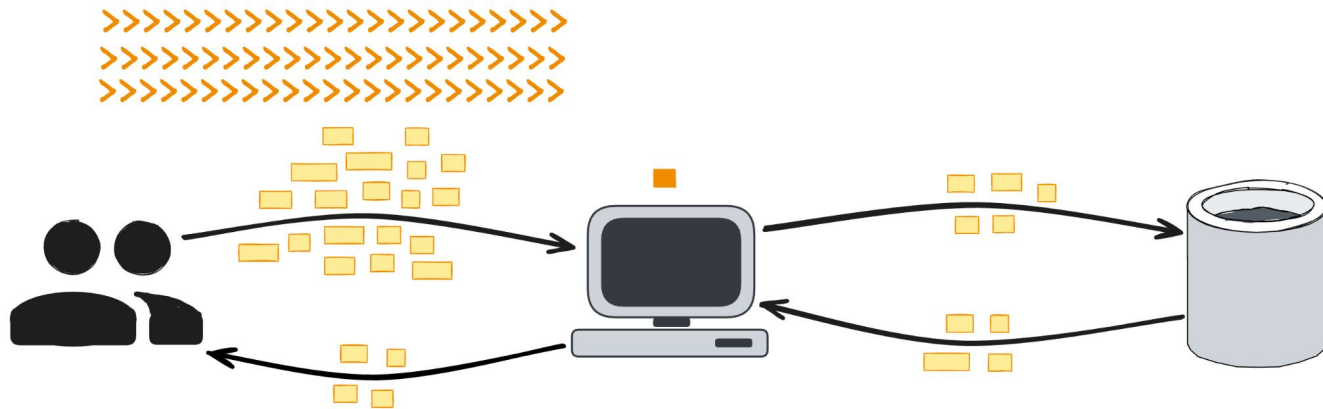


Measuring the Flow of Requests

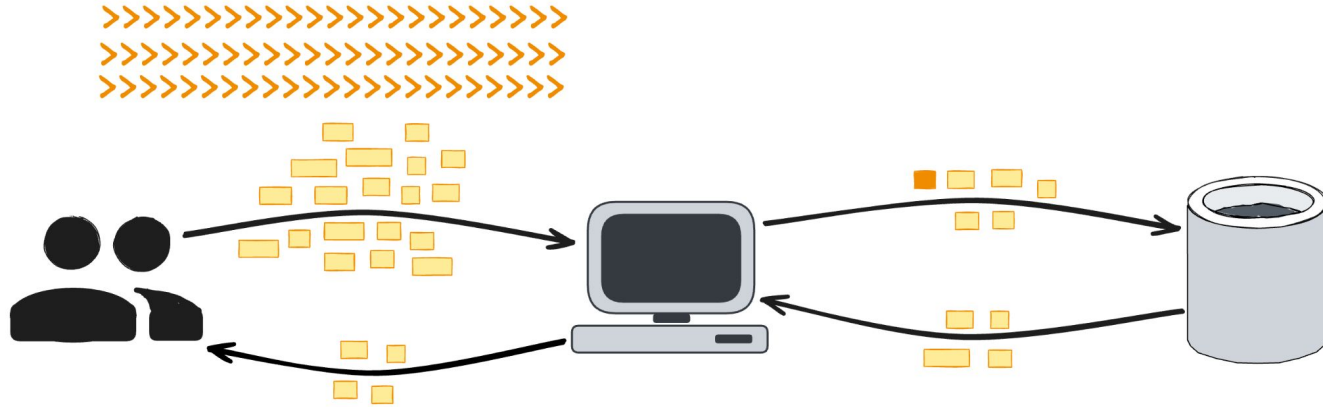


Measurement starts

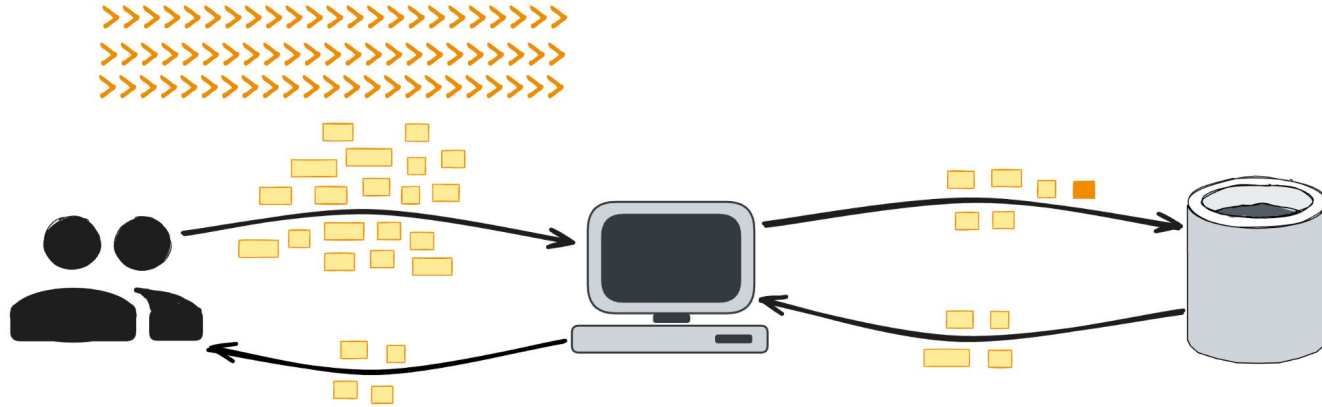
Measuring the Flow of Requests



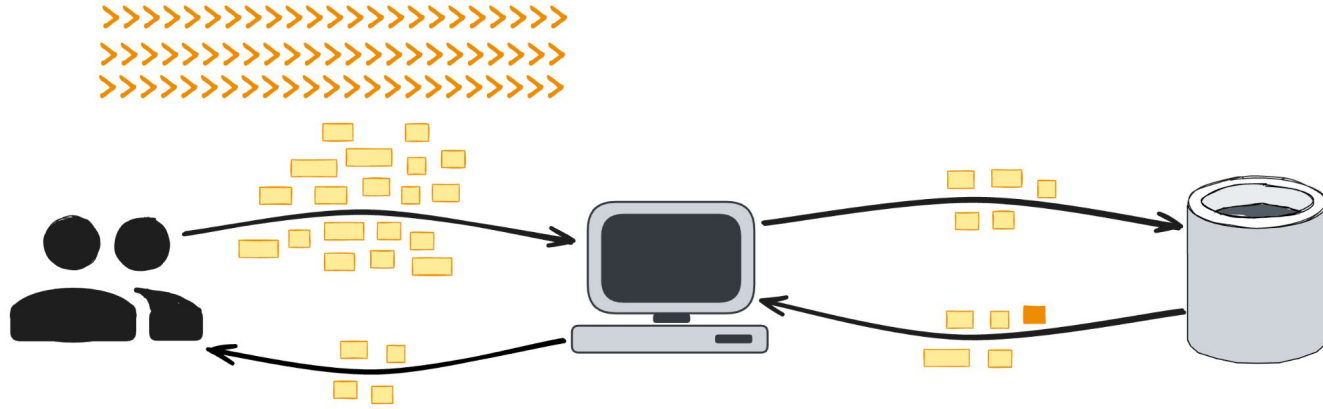
Measurement starts
The request's age has increased



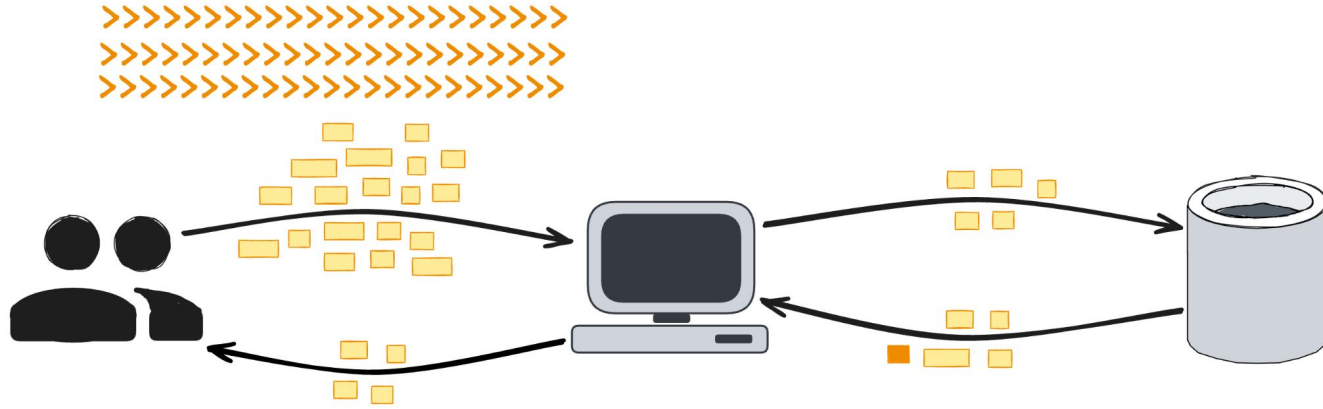
Measuring the Flow of Requests



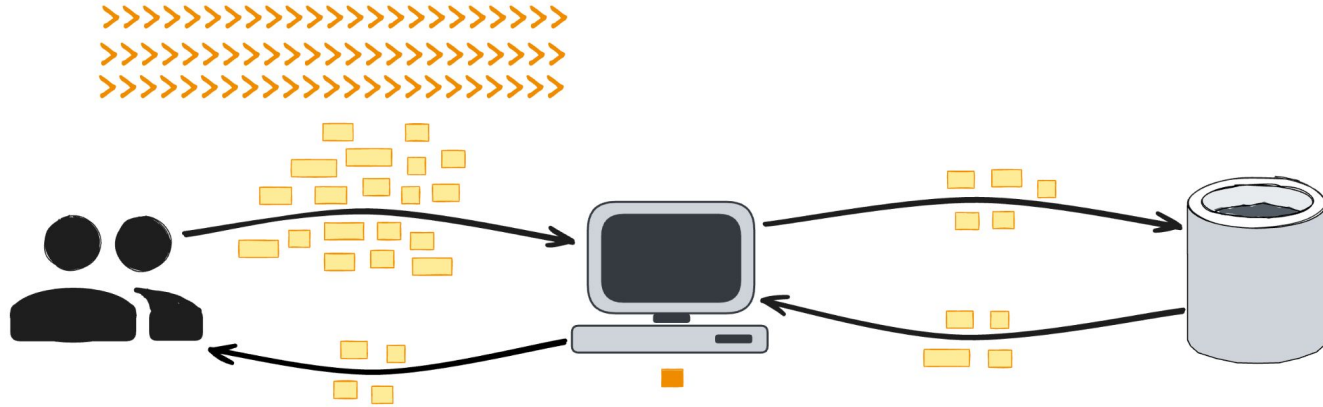
Measuring the Flow of Requests



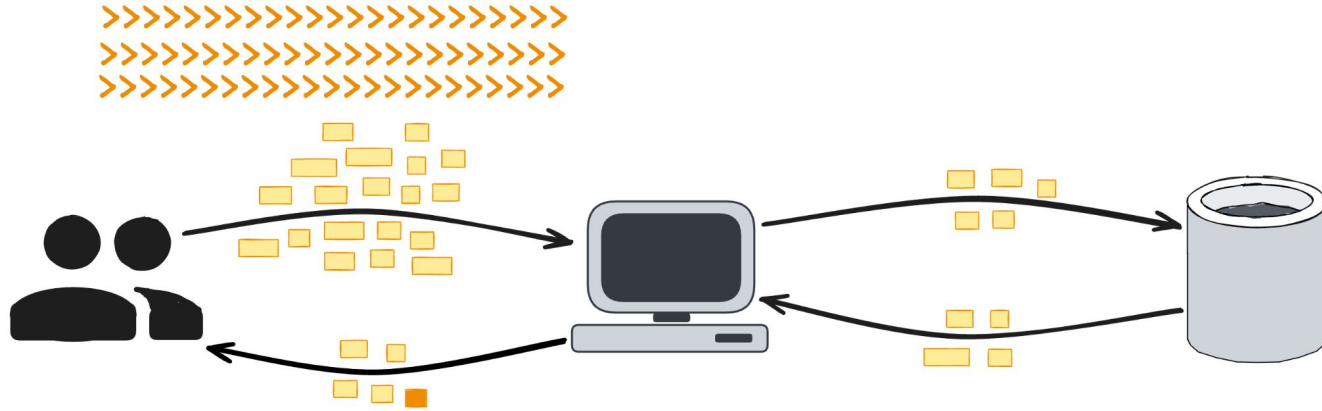
Measuring the Flow of Requests



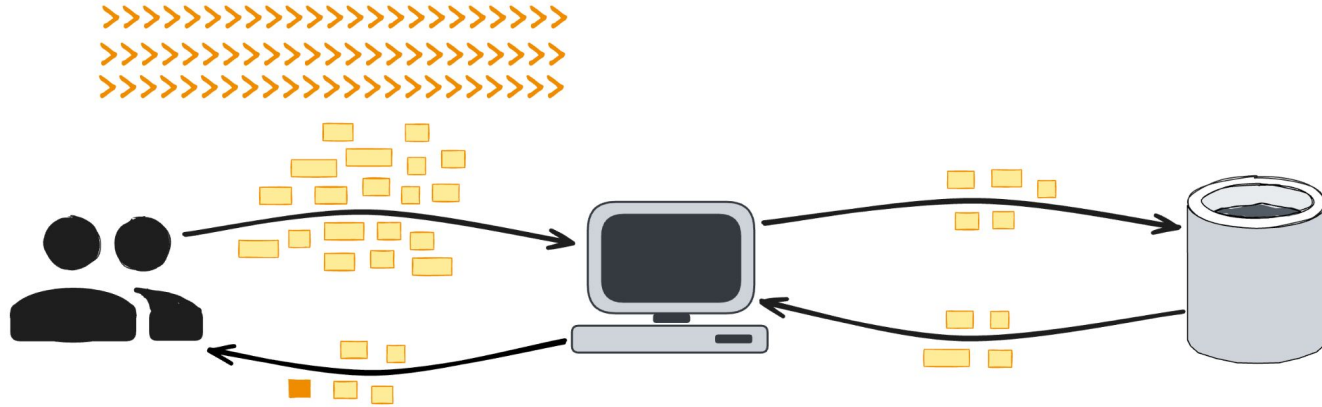
Measuring the Flow of Requests



Measuring the Flow of Requests

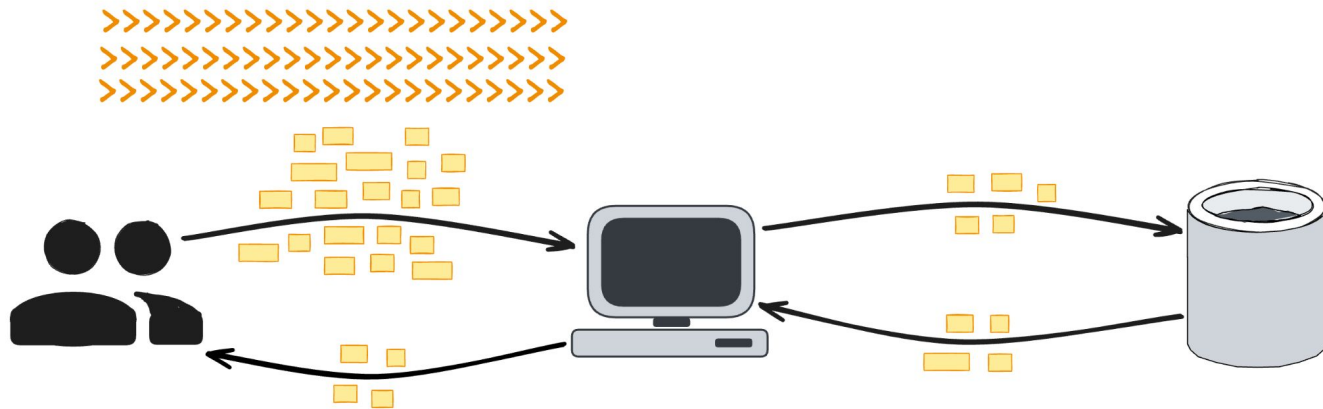


Measuring the Flow of Requests



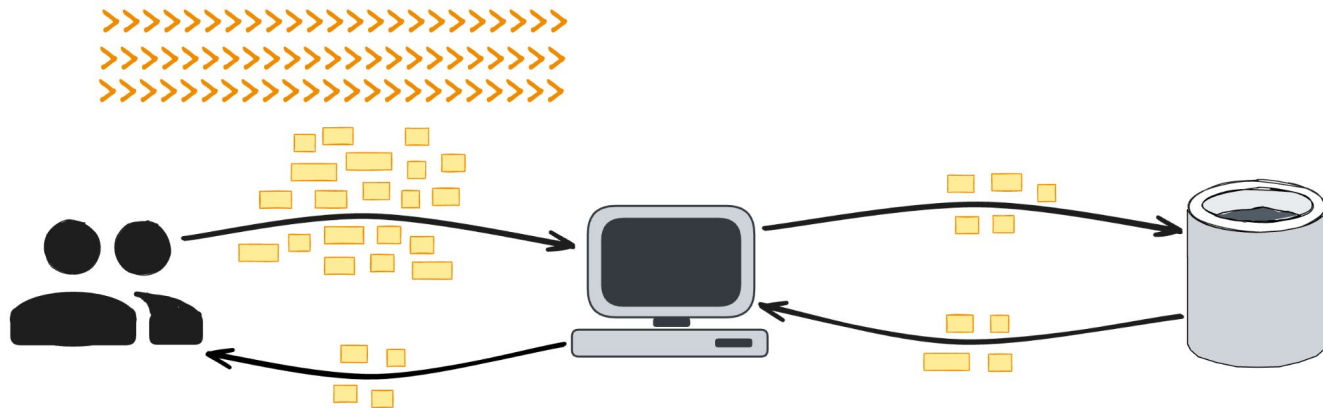


Measuring the Flow of Requests



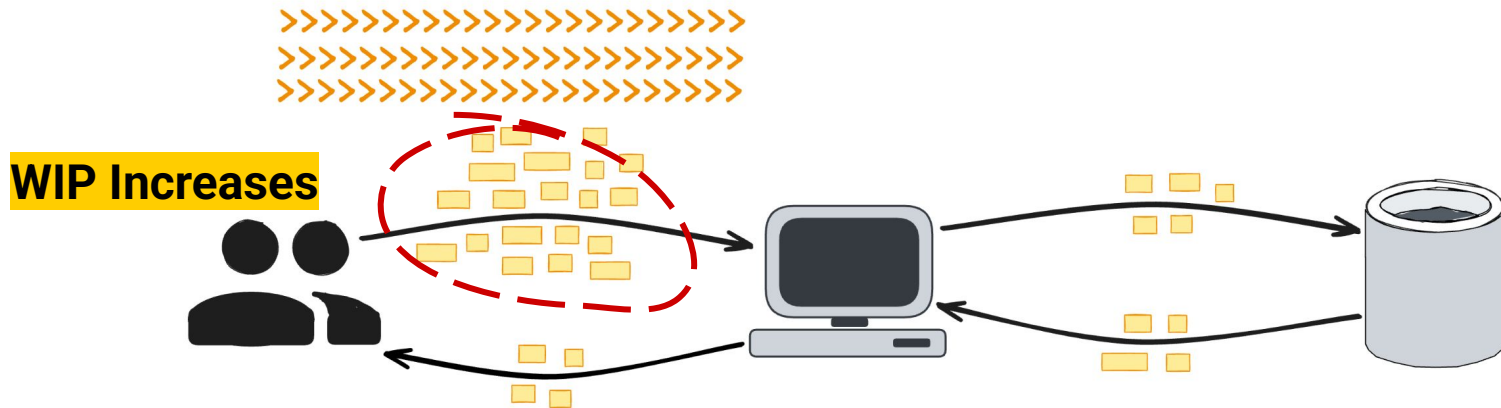
Measuring the Flow of Requests

When there is a **request spike**

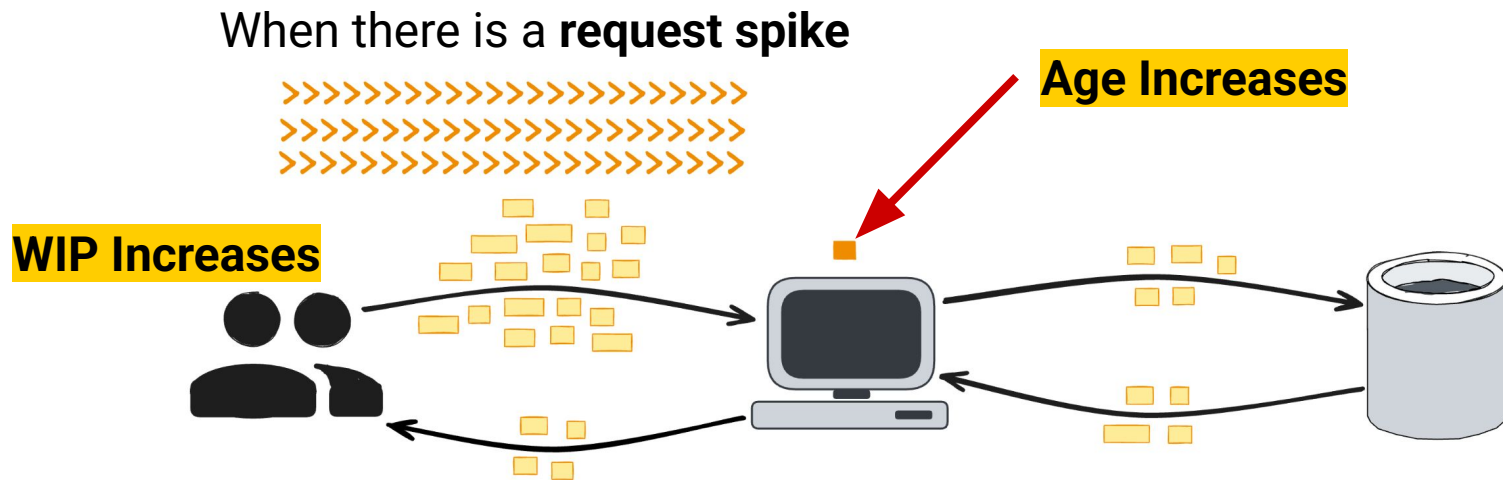


Measuring the Flow of Requests

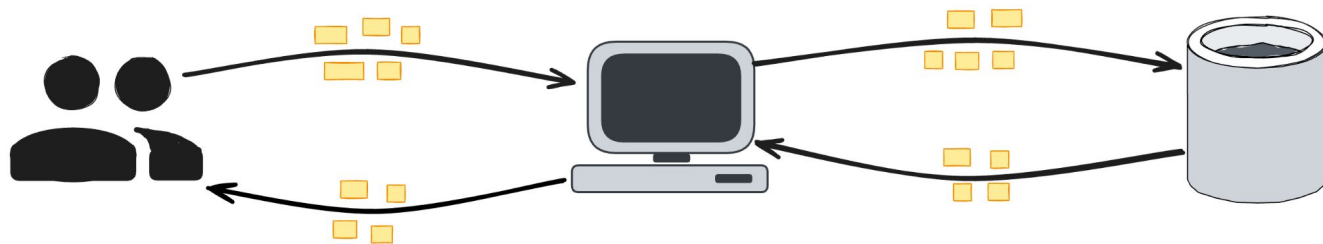
When there is a **request spike**



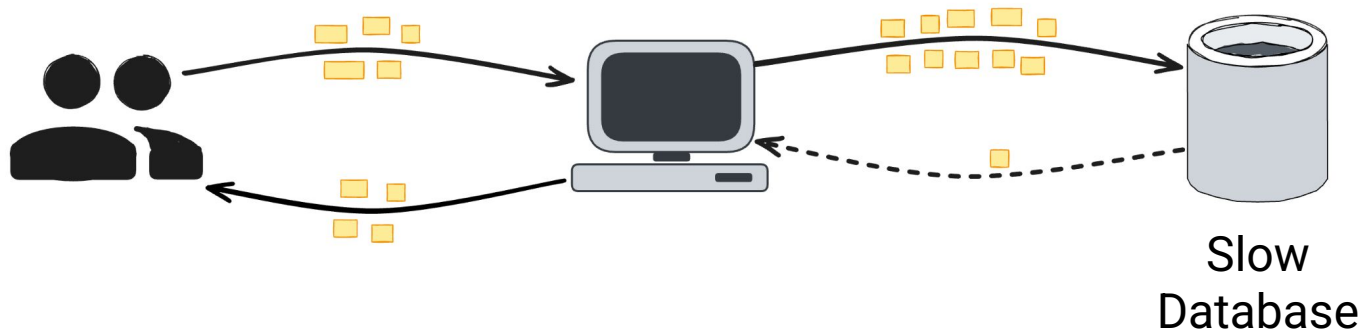
Measuring the Flow of Requests

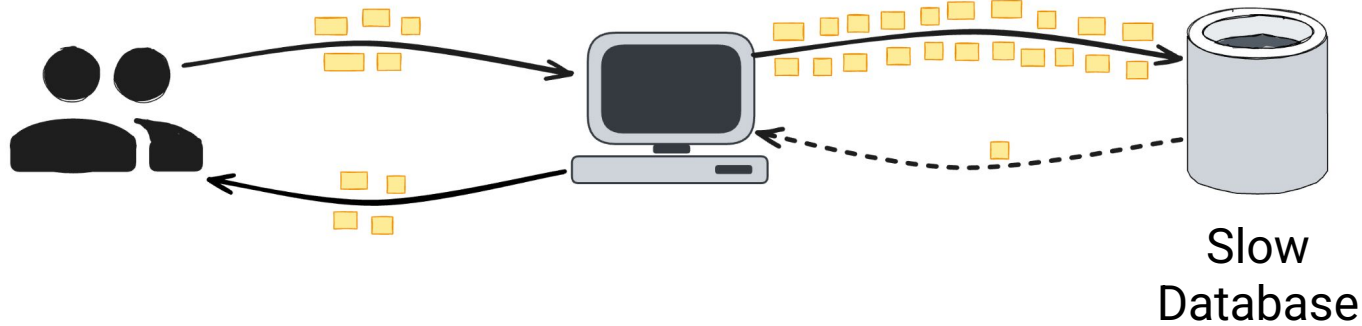


Measuring the Flow of Requests



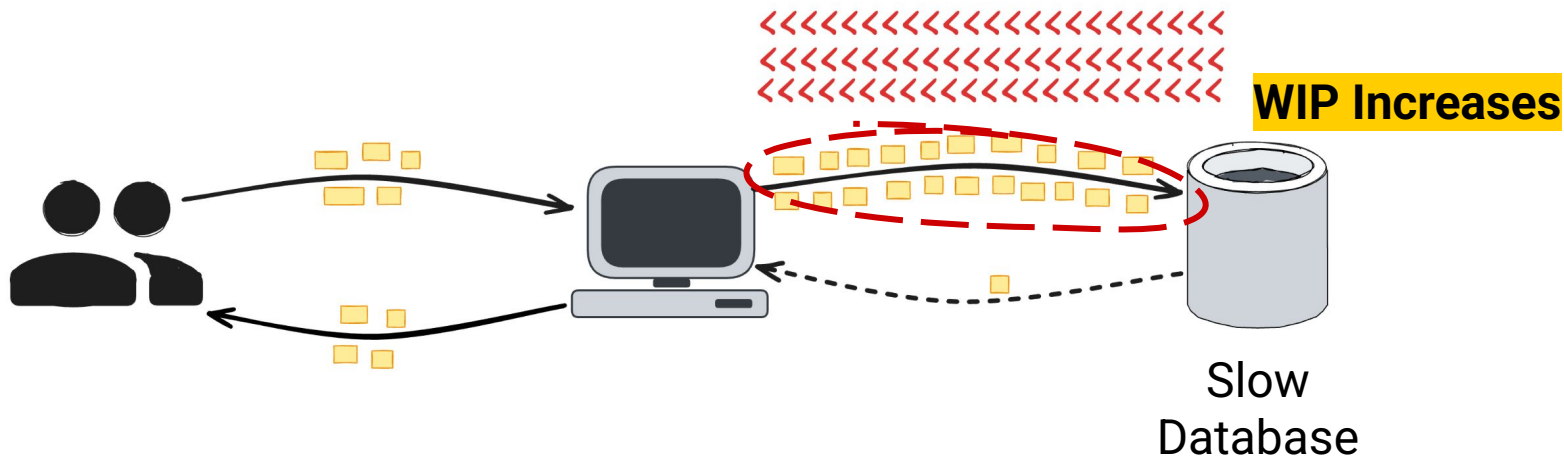
Measuring the Flow of Requests





Measuring the Flow of Requests

When there is a **slow dependency**





Measuring the Flow of Requests

- **Regardless** of the cause
 - Spike in requests
 - Slow dependency



Measuring the Flow of Requests

- **Regardless** of the cause
 - Spike in requests
 - Slow dependency
- The effect is an increase in
 - **WIP** (In-flight requests)
 - **Age** of requests

Measuring the Flow of Requests

- **Regardless** of the cause

- Spike in requests
- Slow dependency

- The effect is an increase in

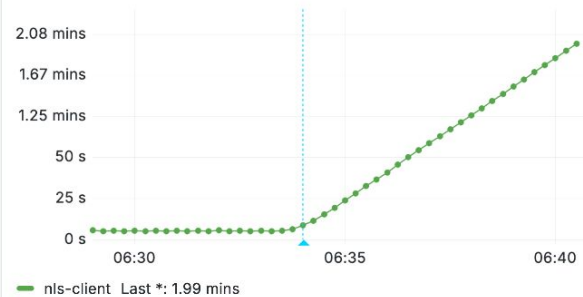
- **WIP** (In-flight requests)
- **Age** of requests

Leading Indicators

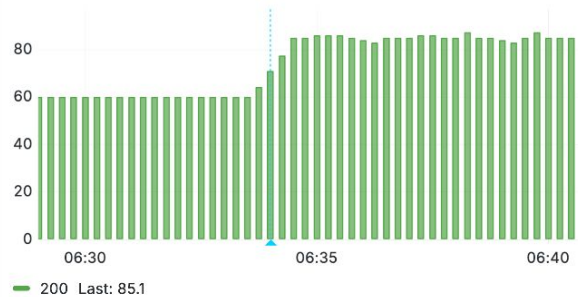


Measuring the Flow of Requests

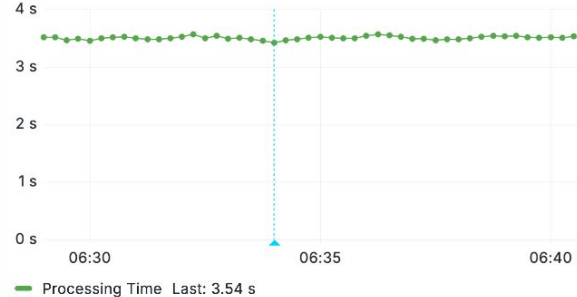
Client Latency



Response Statuses

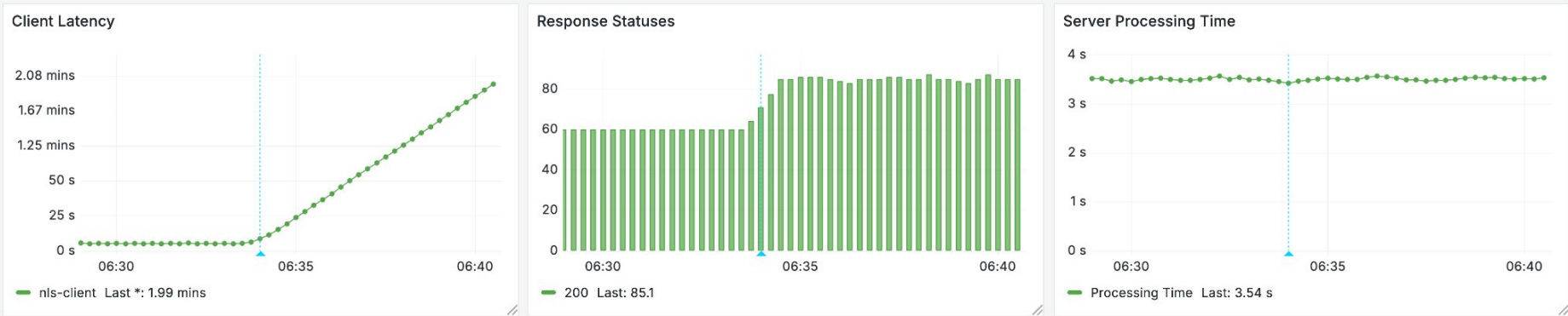


Server Processing Time





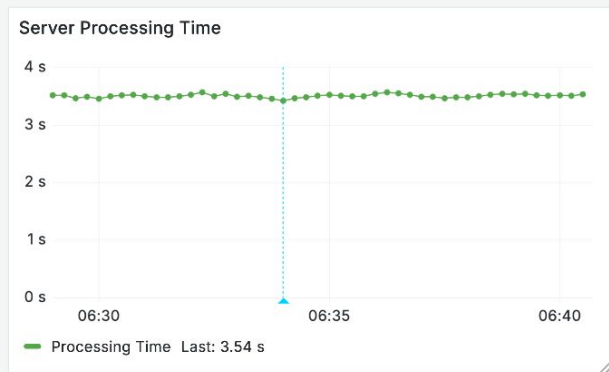
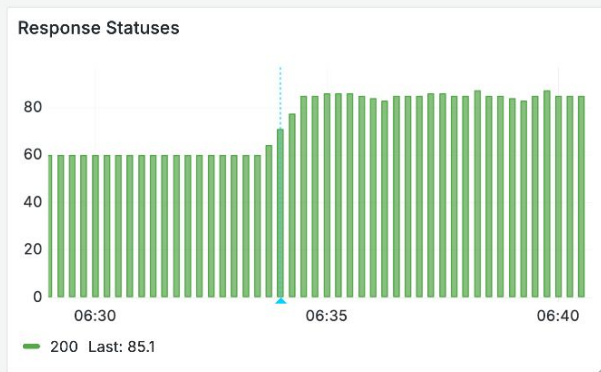
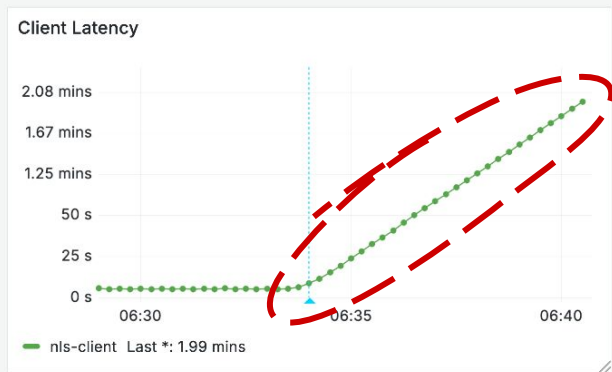
Measuring the Flow of Requests



Measured by the client



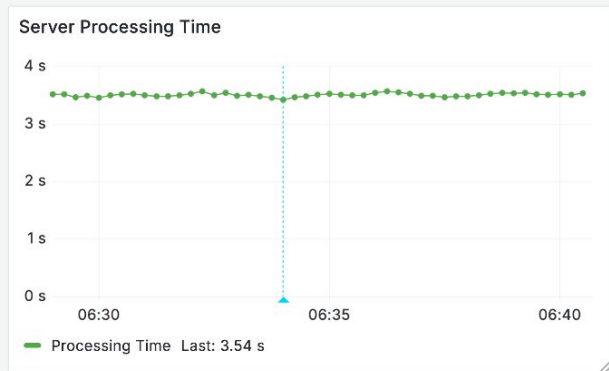
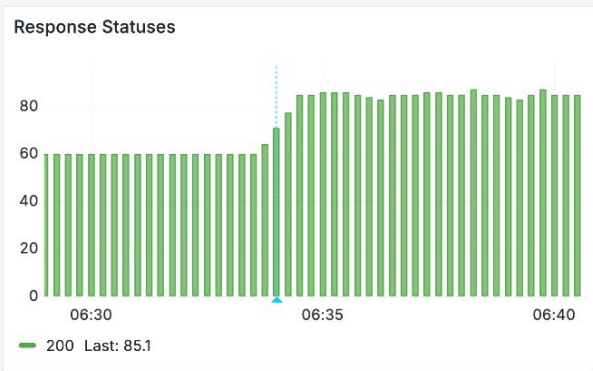
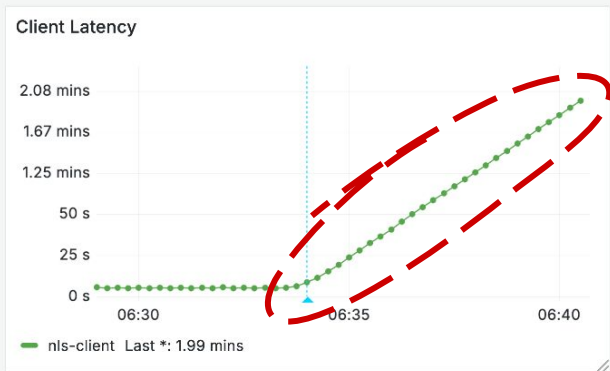
Measuring the Flow of Requests



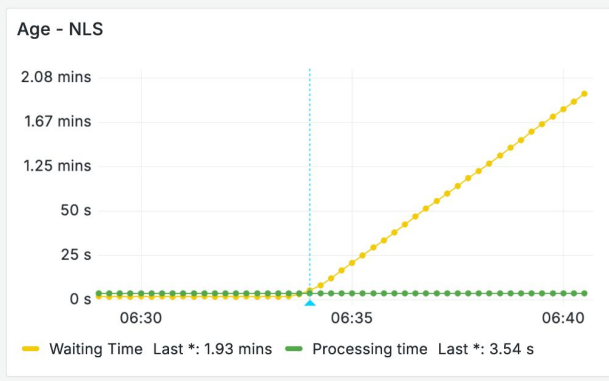
Measured by the client
after the fact



Measuring the Flow of Requests

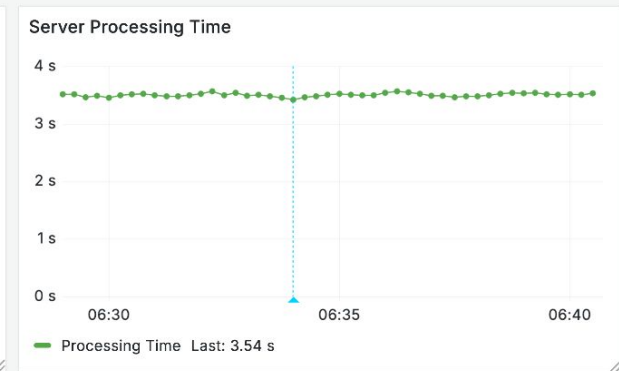
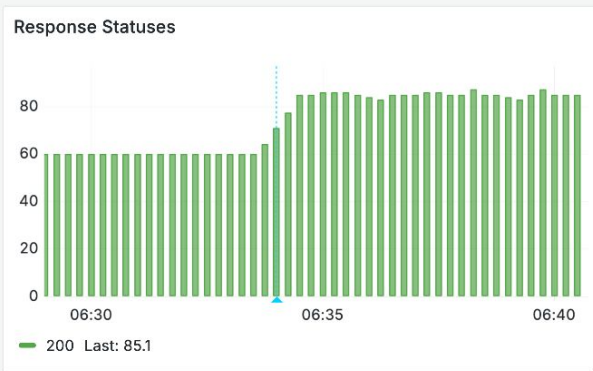
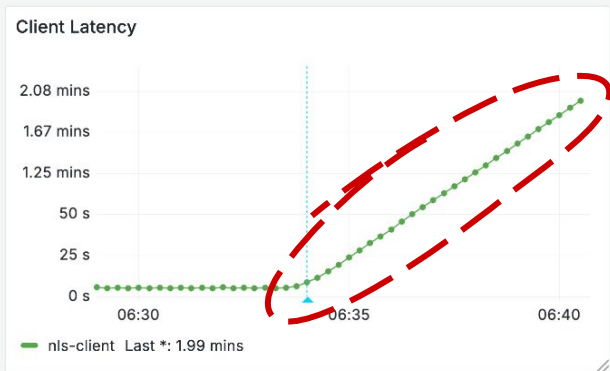


Measured by the client
after the fact



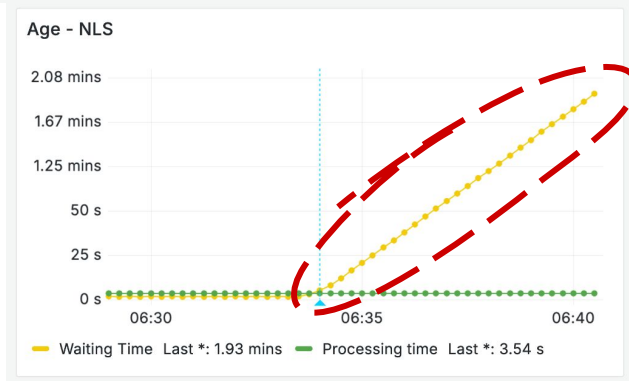


Measuring the Flow of Requests



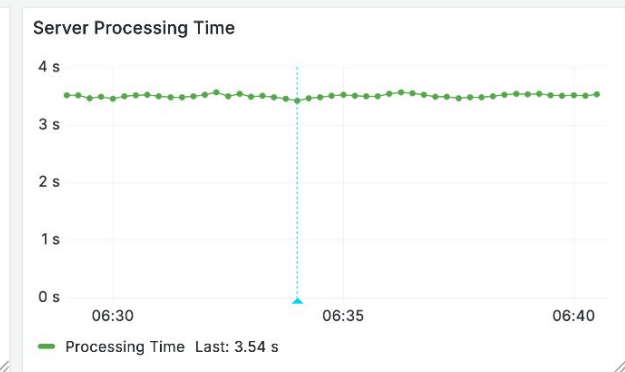
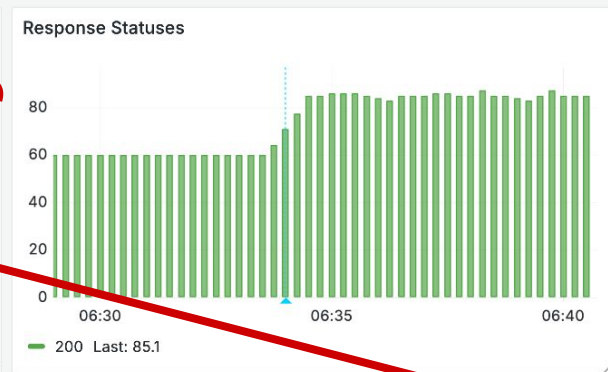
Measured by the client
after the fact

Measured by the server
on the fly



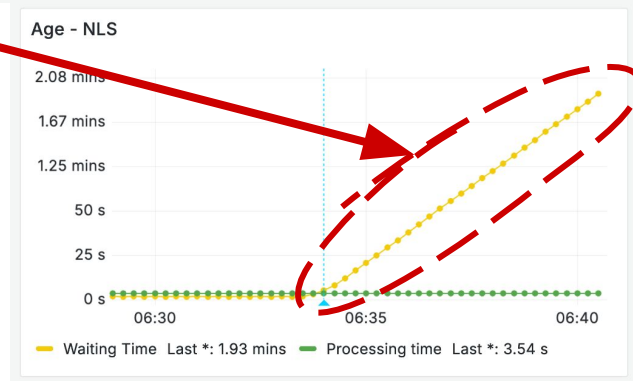


Measuring the Flow of Requests



Measured by the client
after the fact

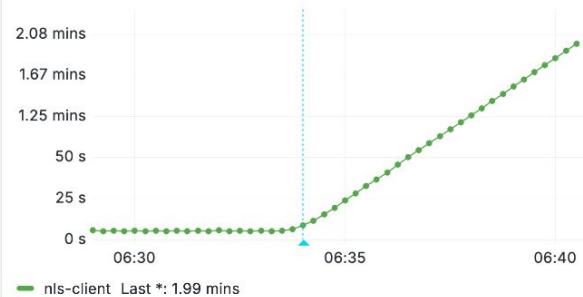
Measured by the server
on the fly



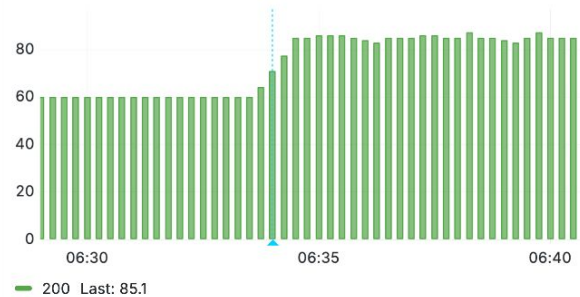


Measuring the Flow of Requests

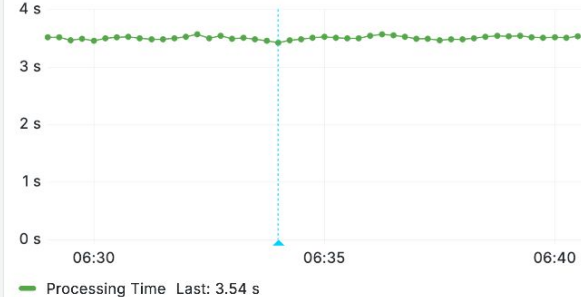
Client Latency



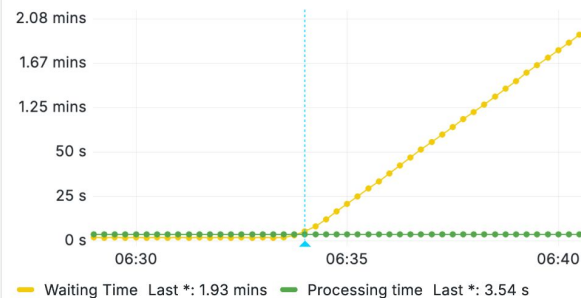
Response Statuses



Server Processing Time



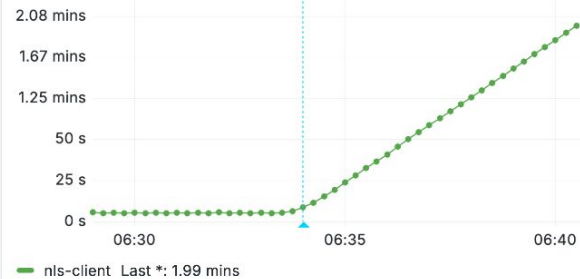
Age - NLS



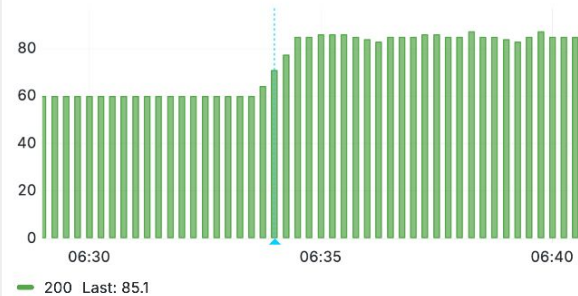


Measuring the Flow of Requests

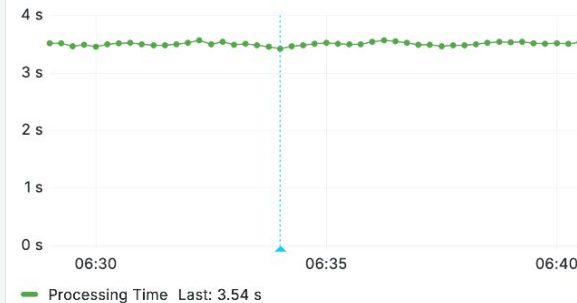
Client Latency



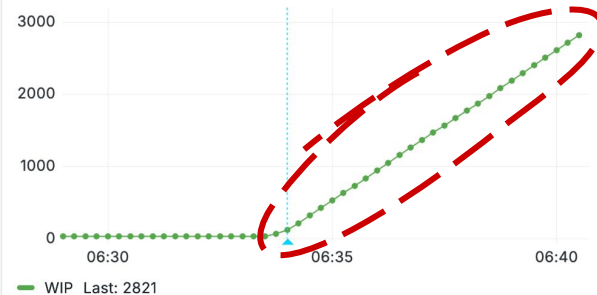
Response Statuses



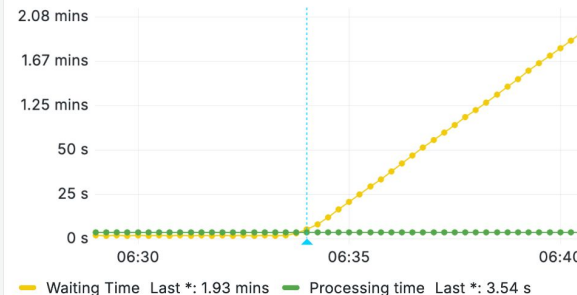
Server Processing Time



In flight Requests (WIP) - NLS



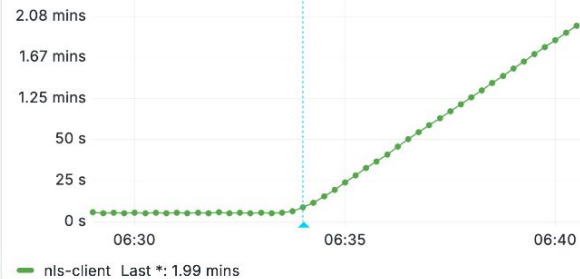
Age - NLS



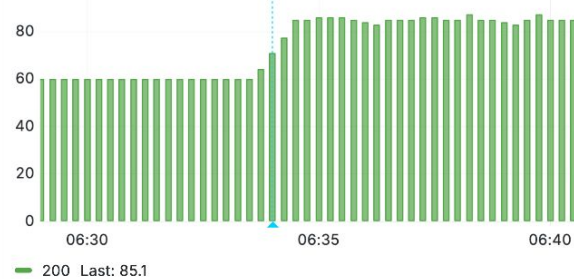


Measuring the Flow of Requests

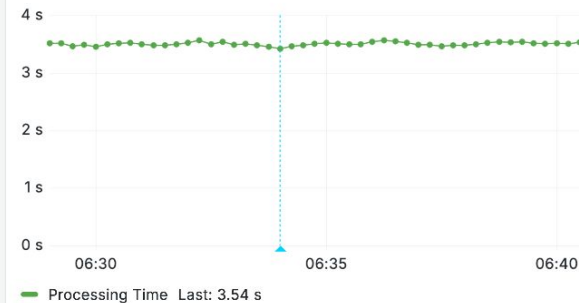
Client Latency



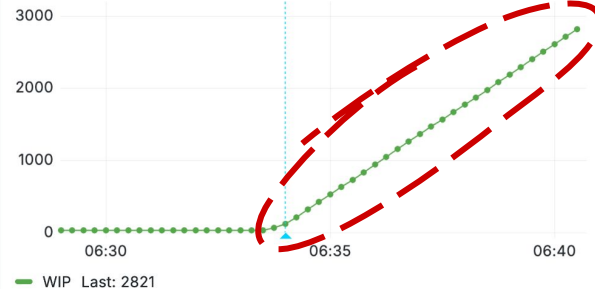
Response Statuses



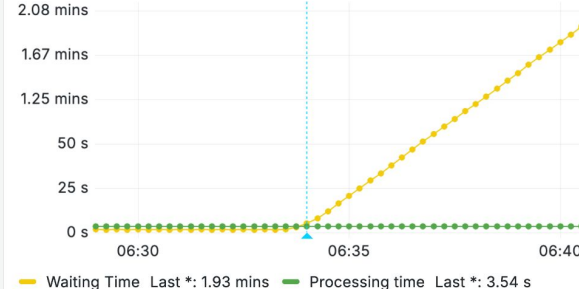
Server Processing Time



In flight Requests (WIP) - NLS



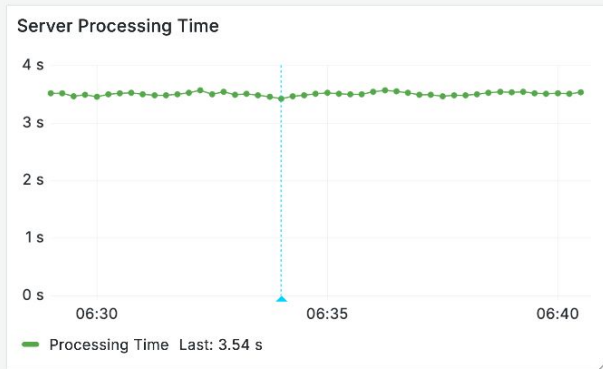
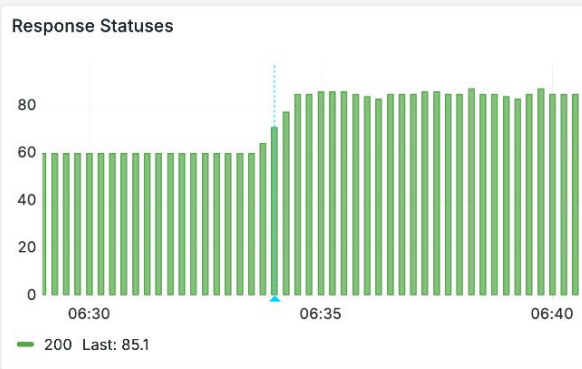
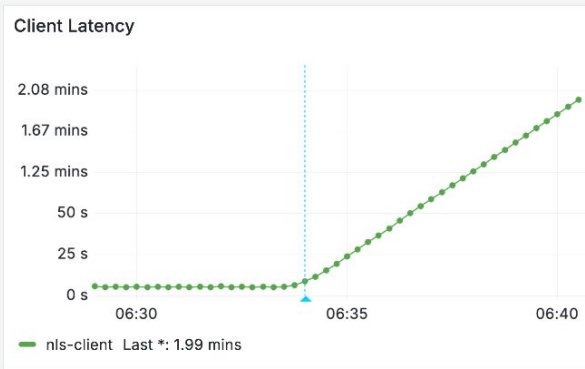
Age - NLS



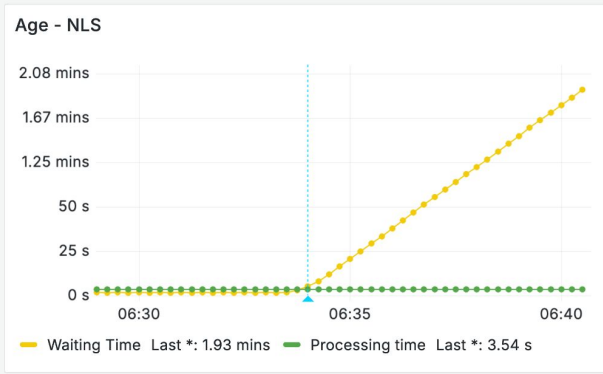
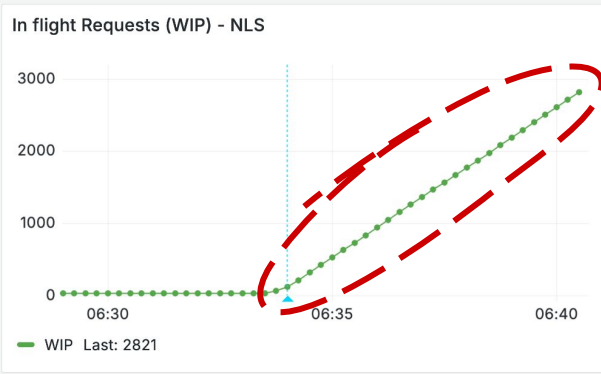
Too much work piling up



Measuring the Flow of Requests

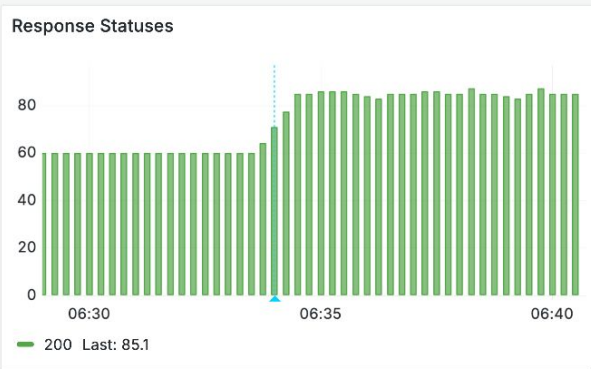
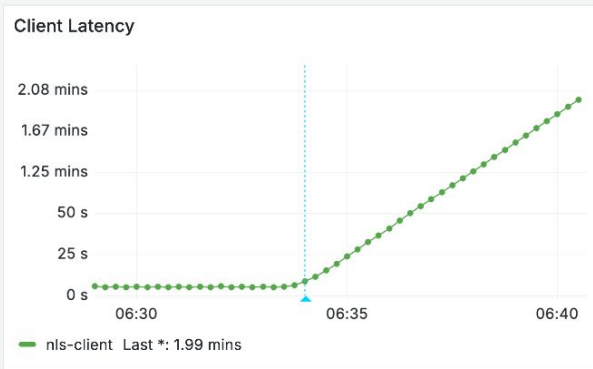


Too much work piling up
Congestion

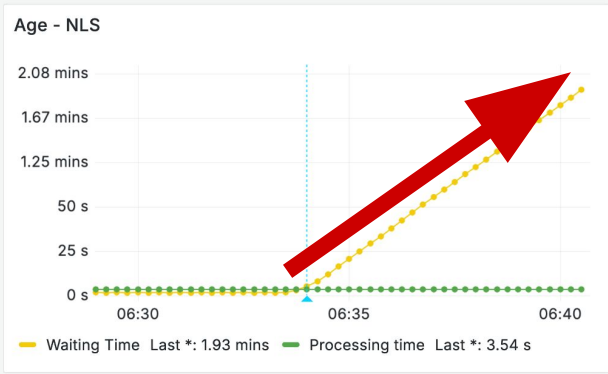
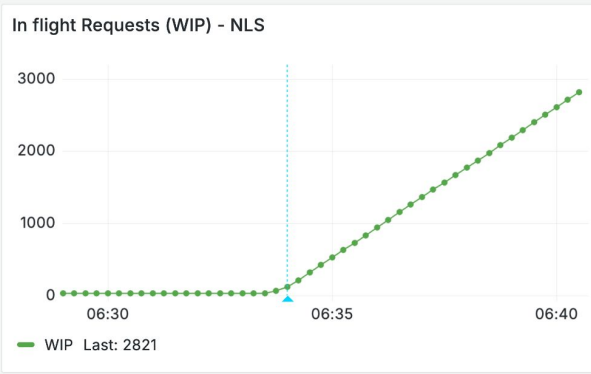




Measuring the Flow of Requests



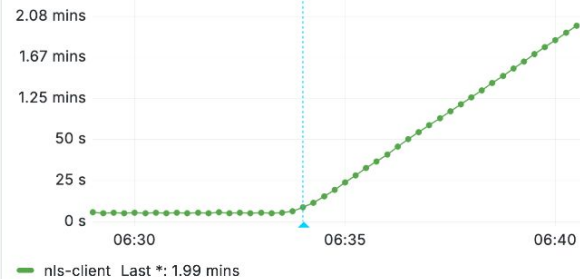
Too much work piling up
Congestion



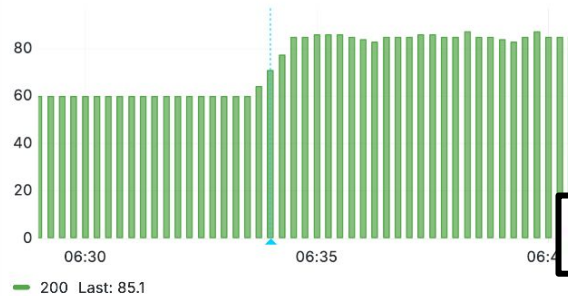


Measuring the Flow of Requests

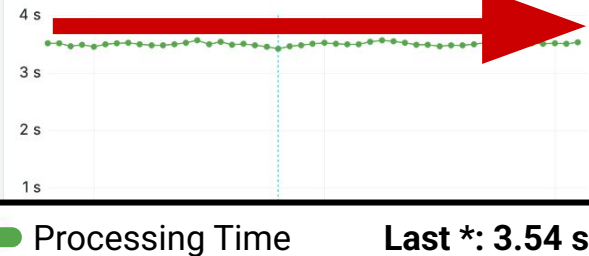
Client Latency



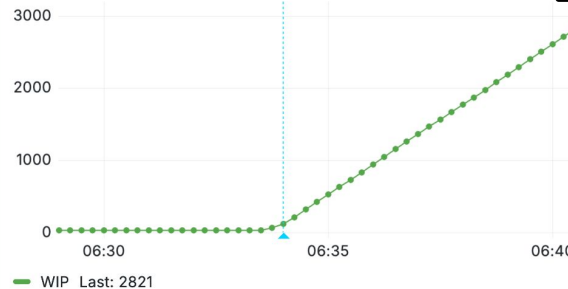
Response Statuses



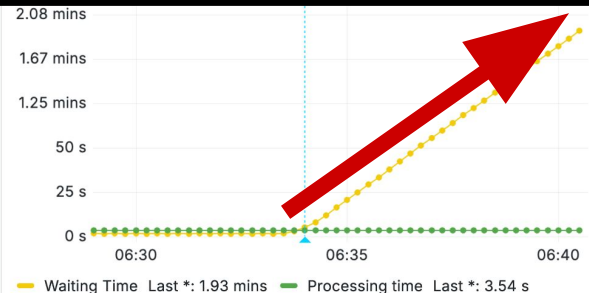
Server Processing Time



In flight Requests (WIP) - NLS



Waiting Time Last *: 1.93 mins



Too much work piling up
Congestion

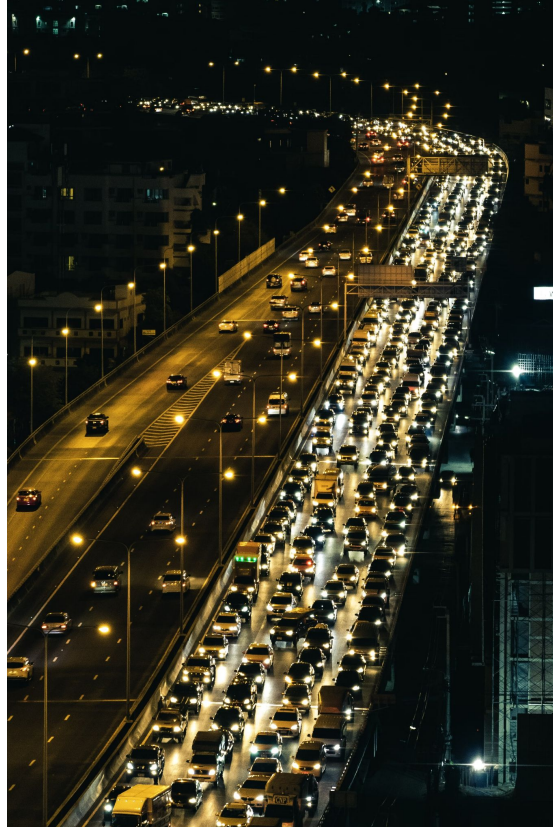
What is Congestion?





What is Congestion?

Congestion is a state of **sustained network overload**.

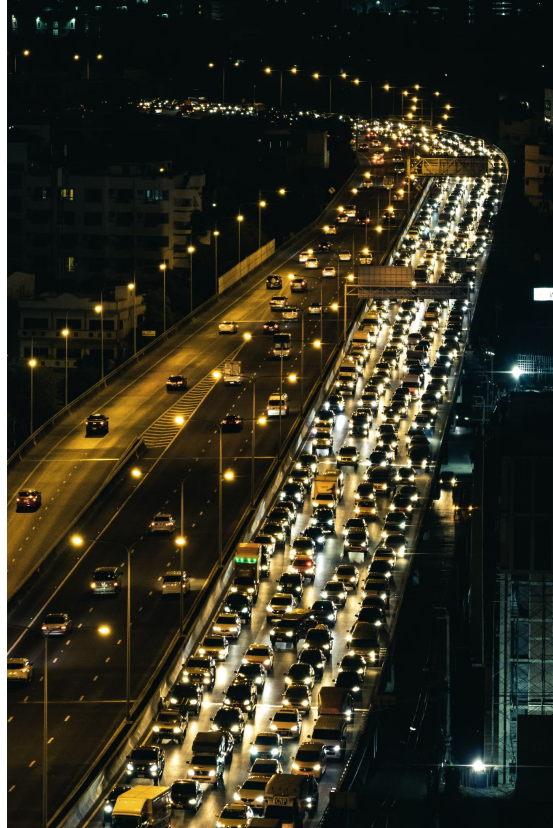




What is Congestion?

Congestion is a state of **sustained network overload**.

When the **demands for resources exceed the supply**.

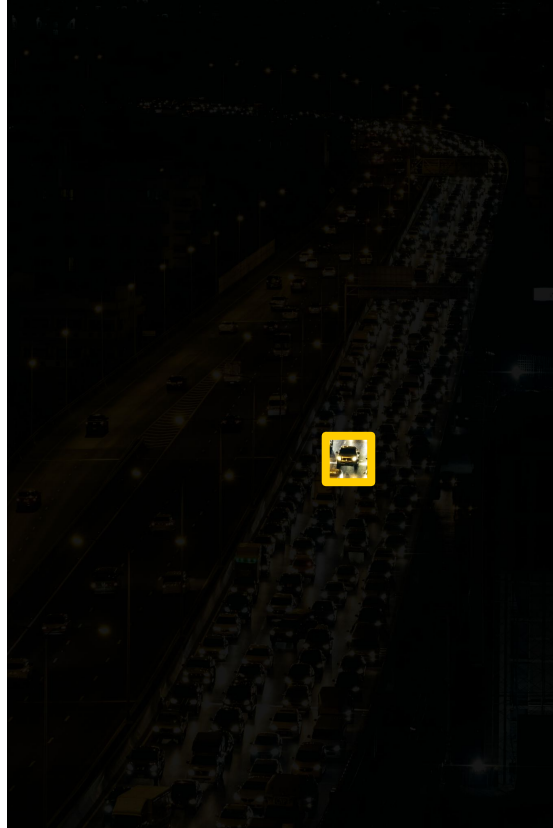




What is Congestion?

Congestion is a state of **sustained network overload**.

When the **demands for resources exceed the supply**.

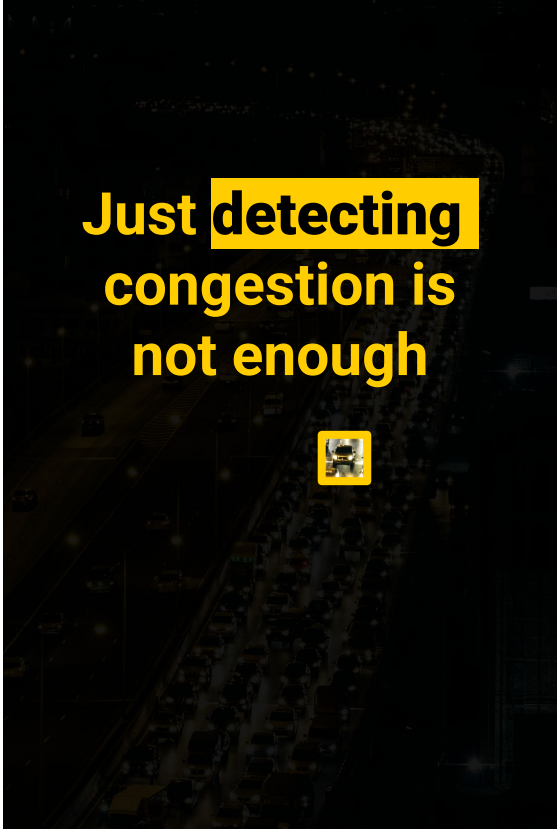




What is Congestion?

Congestion is a state of **sustained network overload**.

When the **demands for resources exceed the supply**.



**Just detecting
congestion is
not enough**



Resilience of the Flow of Requests

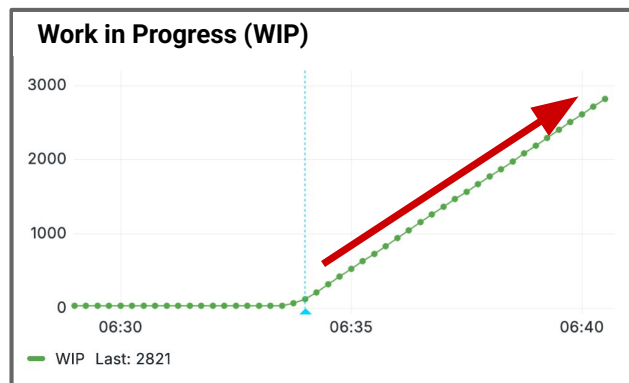




Acting on the Leading Indicators

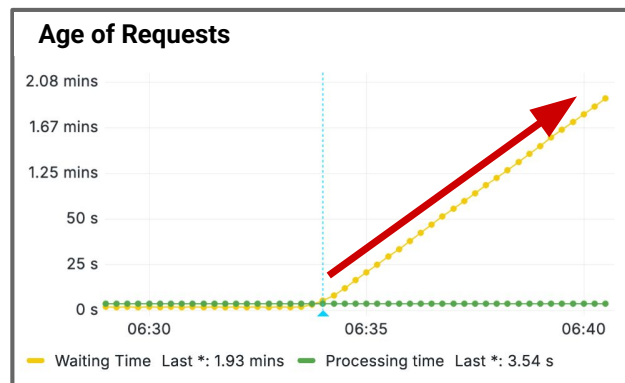
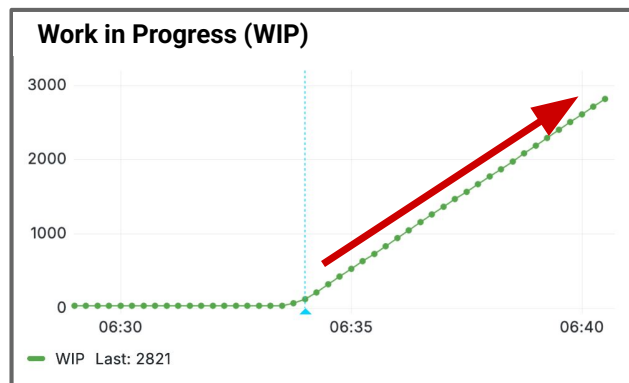
Acting on the Leading Indicators

- During congestion
 - **WIP** increases



Acting on the Leading Indicators

- During congestion
 - **WIP** increases
 - **Age** increases

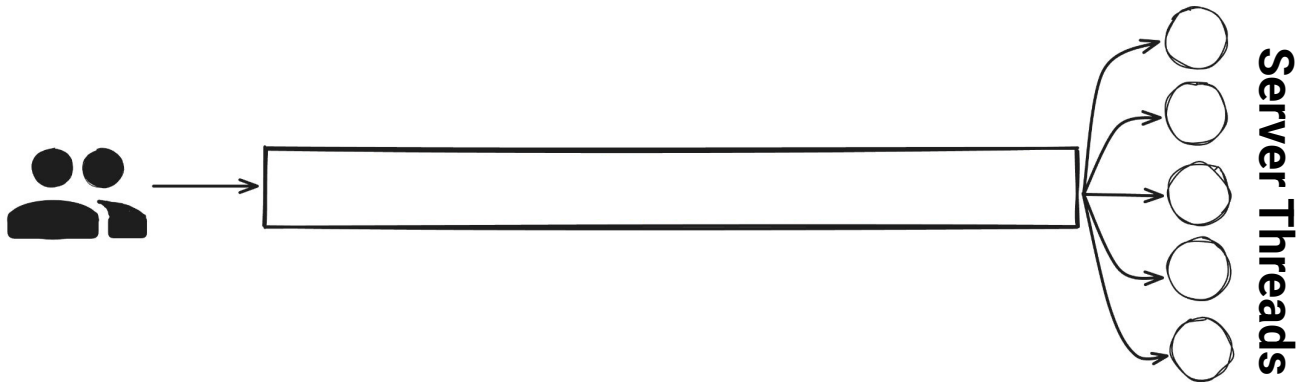




Limiting WIP and Age

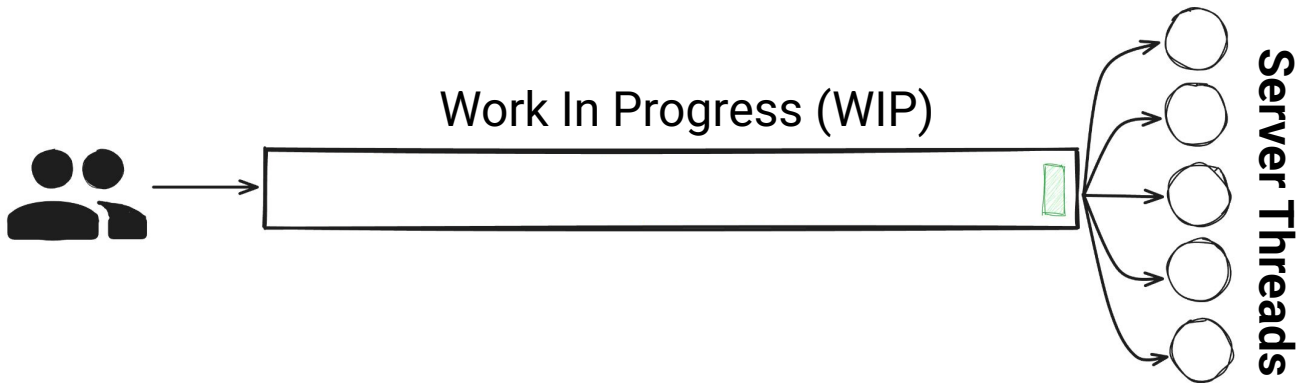


Limiting WIP and Age



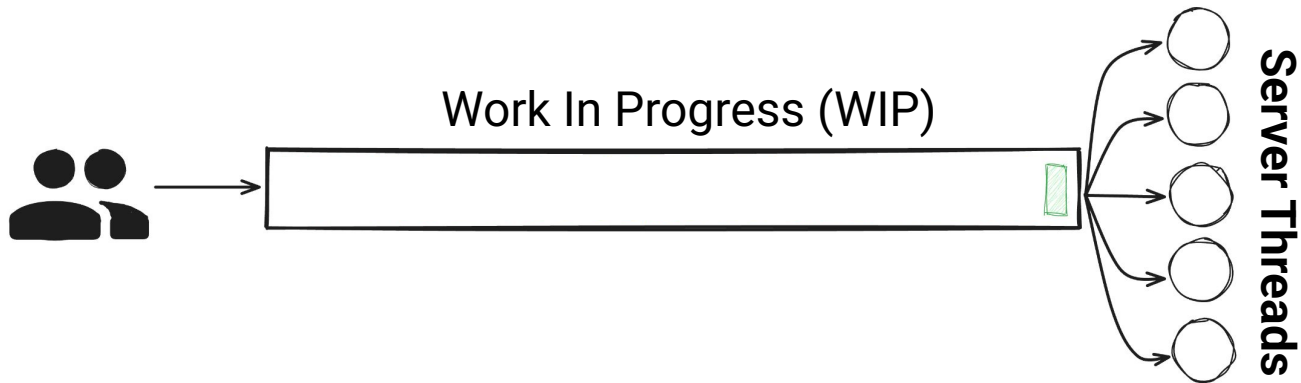


Limiting WIP and Age





Limiting WIP and Age

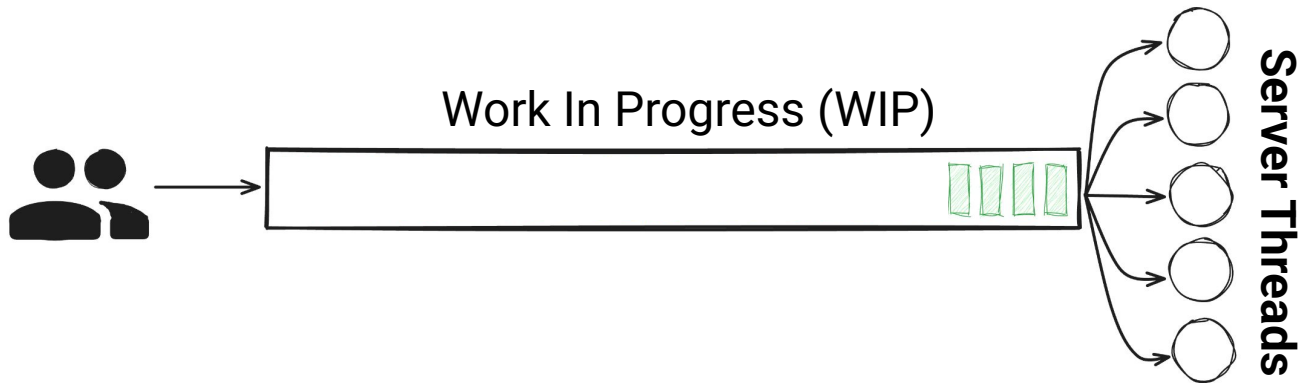


Response Time

Processing Time



Limiting WIP and Age

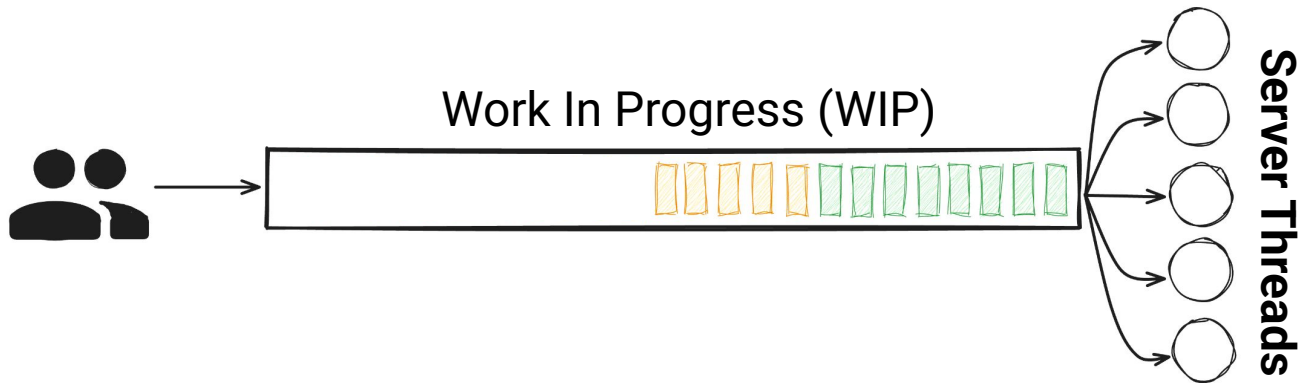


Response Time

Processing Time



Limiting WIP and Age



Response Time

=

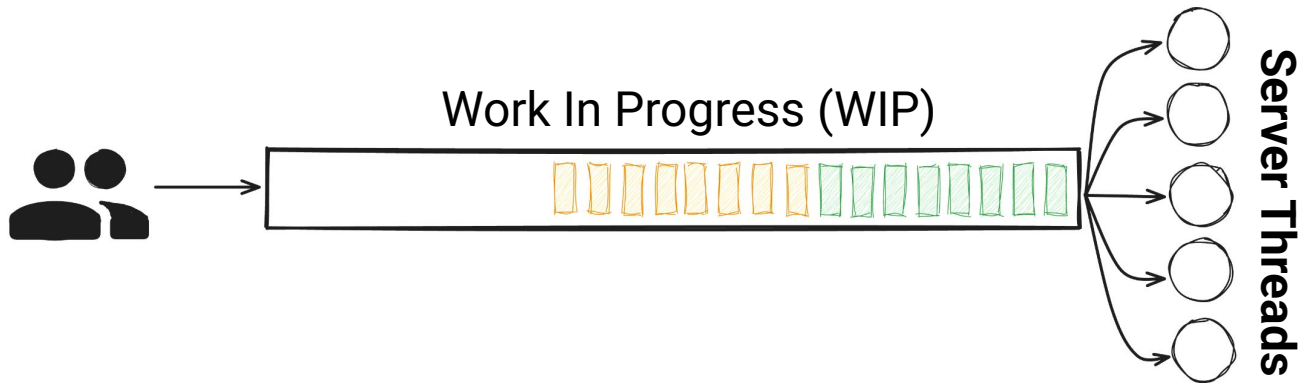
Waiting Time

+

Processing Time



Limiting WIP and Age



Response Time

=

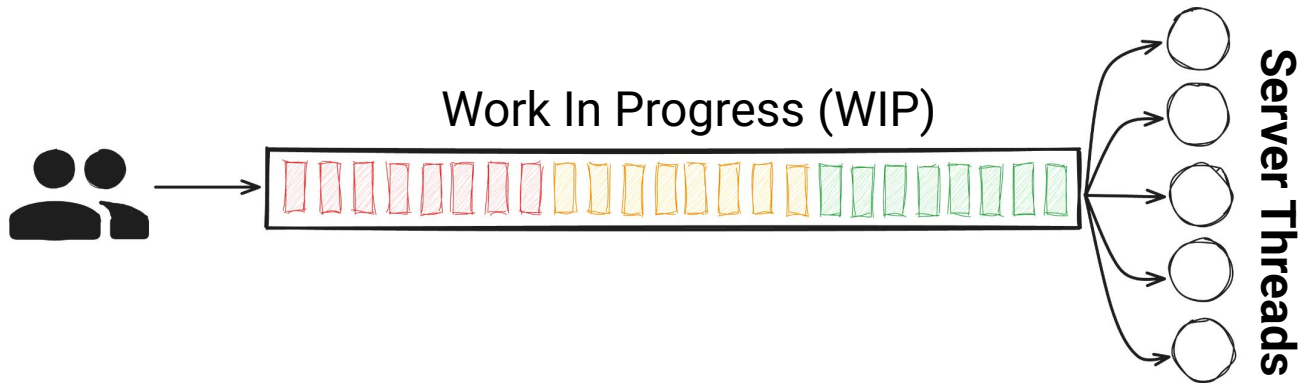
Waiting Time

+

Processing Time



Limiting WIP and Age



Response Time

=

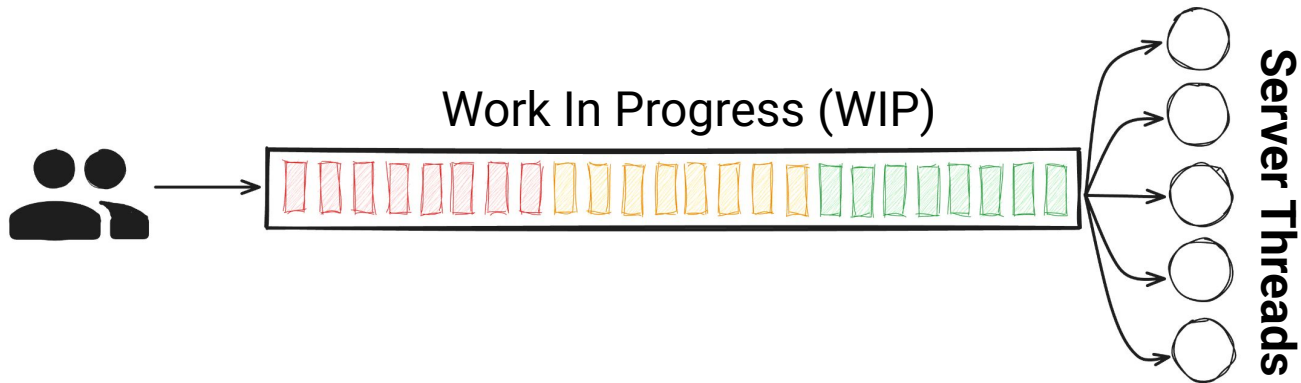
Waiting Time

+

Processing Time



Limiting WIP and Age



Response Time

=

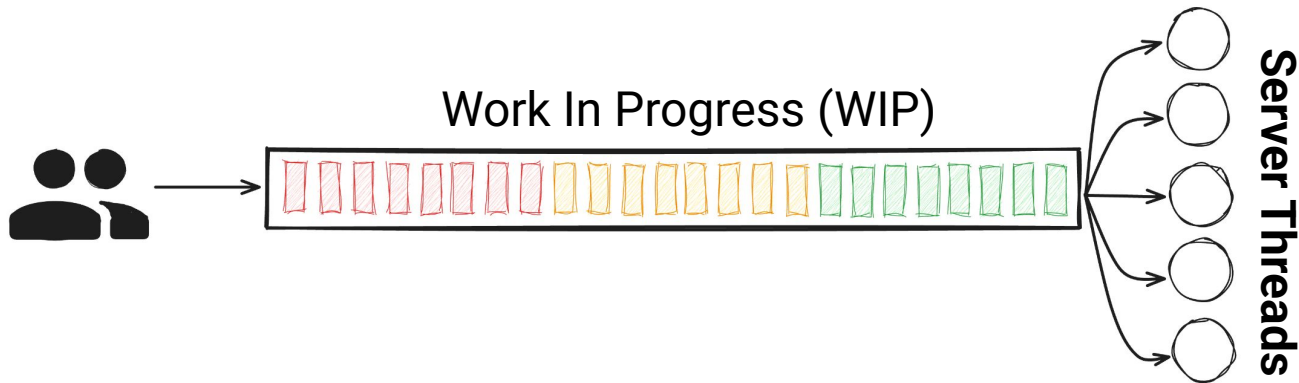
Waiting Time

+

Processing Time
(Constant)



Limiting WIP and Age



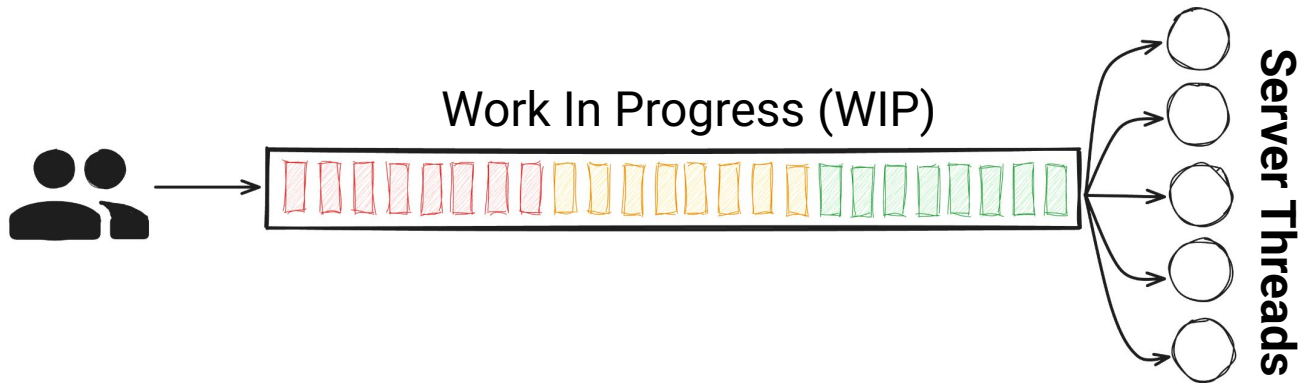
Response Time

α

Waiting Time



Limiting WIP and Age



Response Time

α

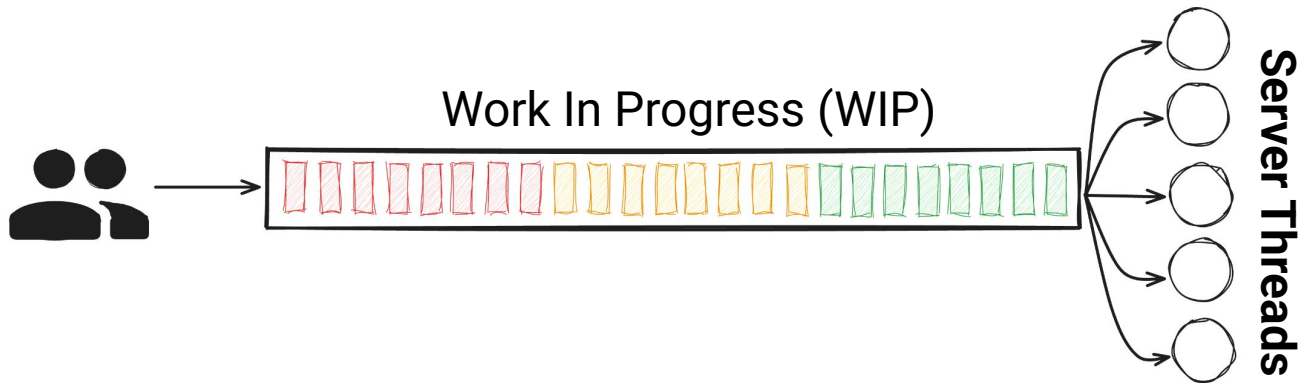
Waiting Time

α

Queue Length



Limiting WIP and Age



Response Time

α

Waiting Time

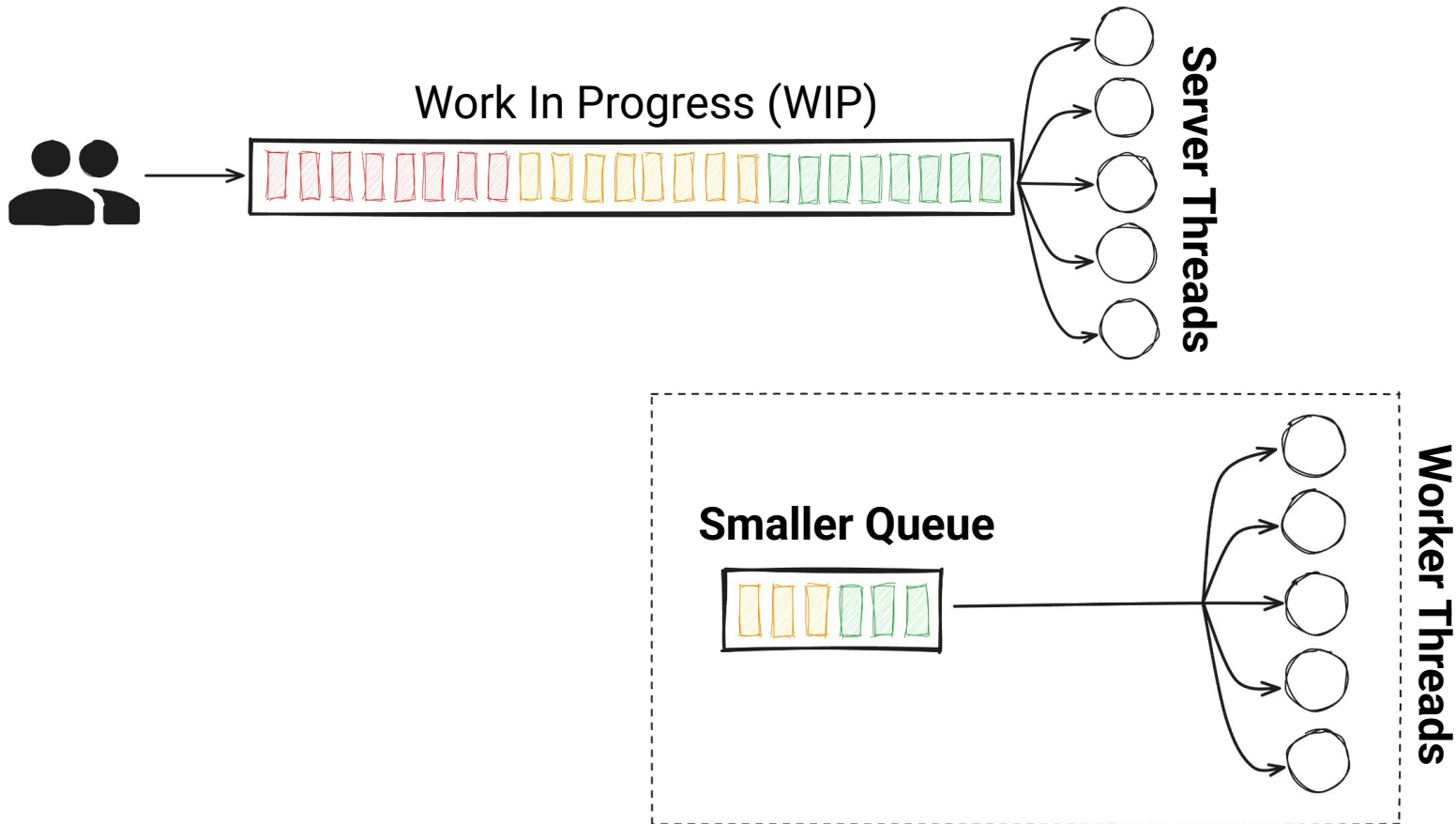
α

Queue Length

Longer Queue Length $\Rightarrow$ Higher Response Time

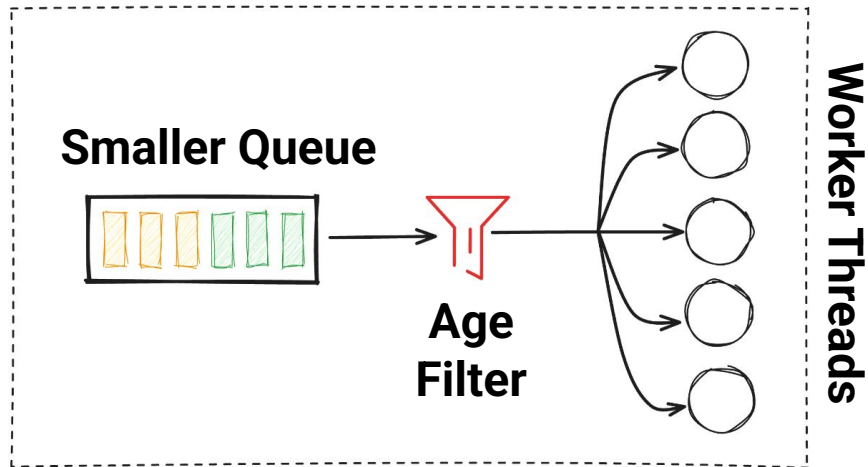
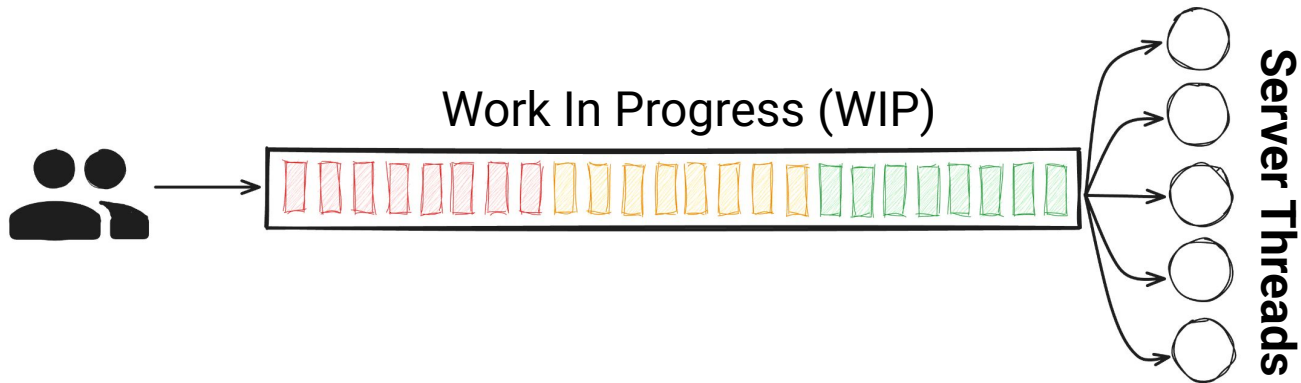


Limiting WIP and Age



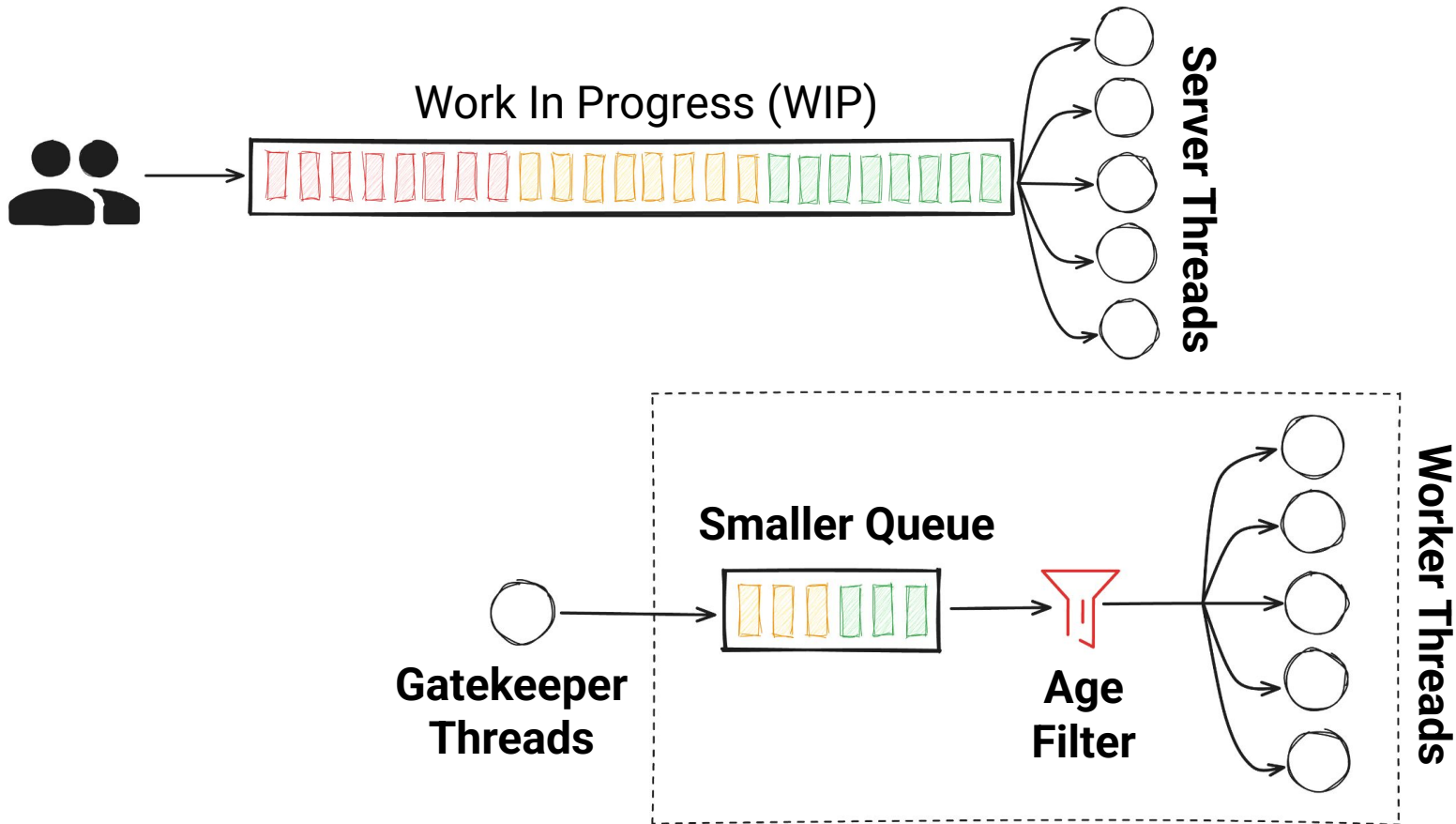


Limiting WIP and Age



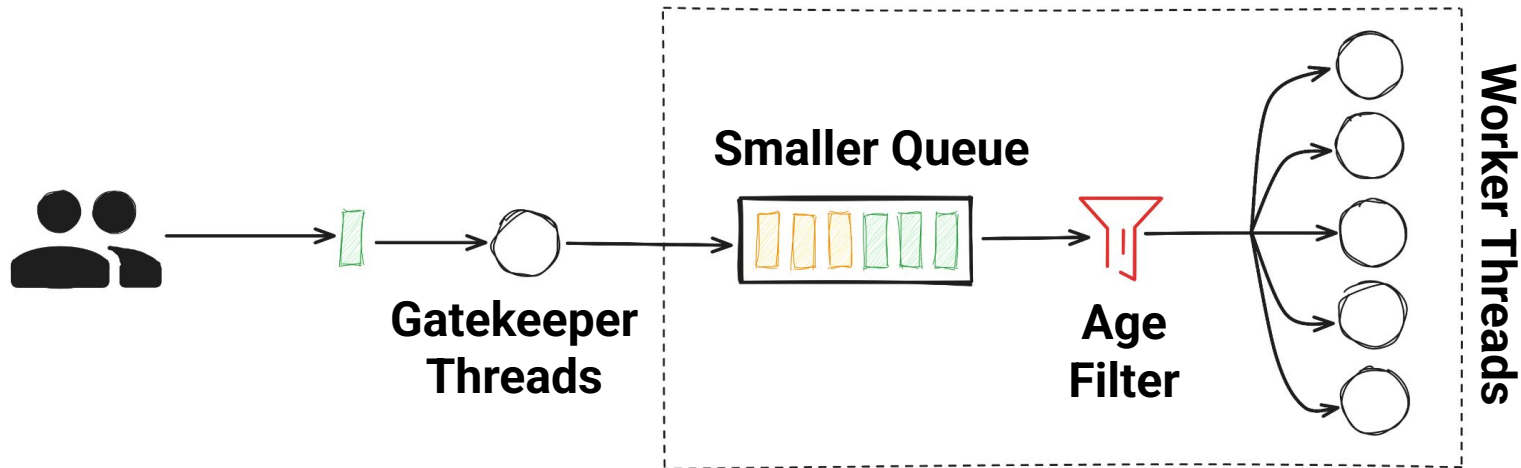
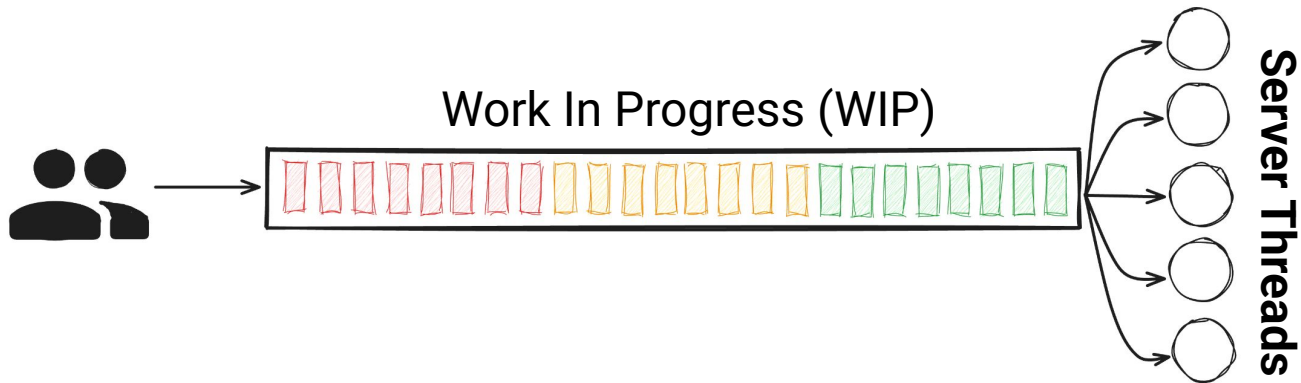


Limiting WIP and Age

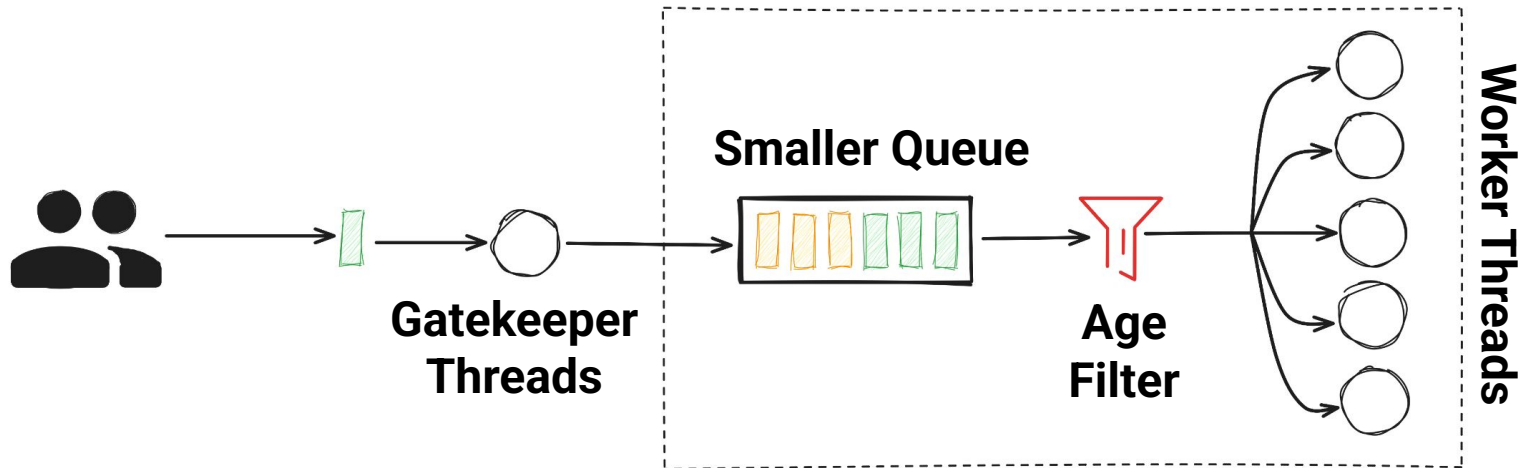




Limiting WIP and Age

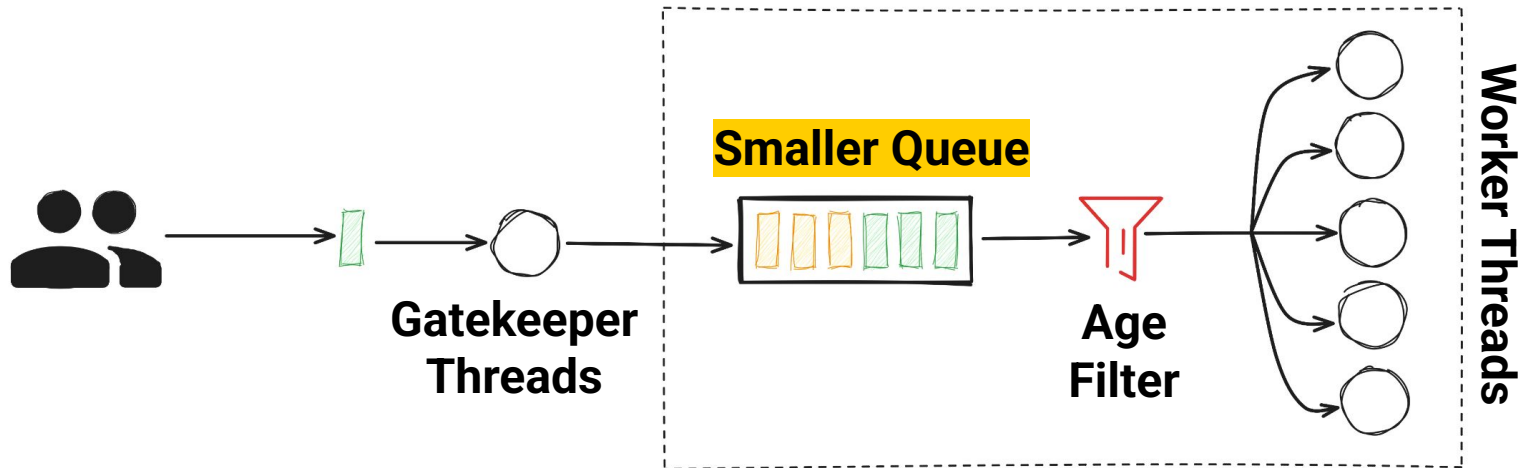


Limiting WIP and Age



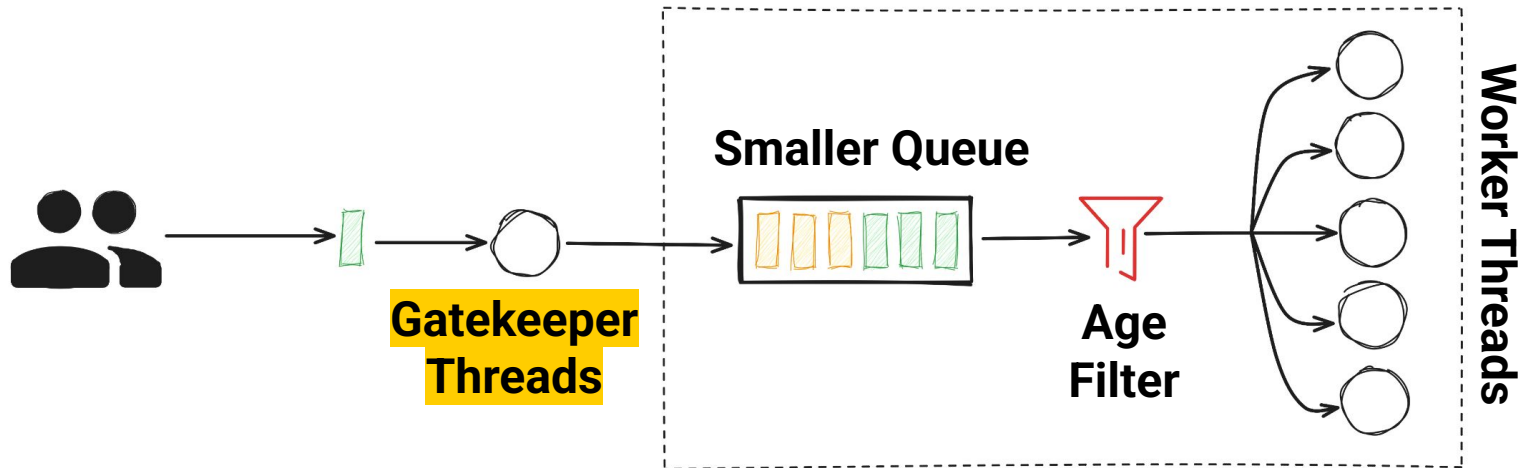
Limiting WIP and Age

- **Bounded wait-time** for clients



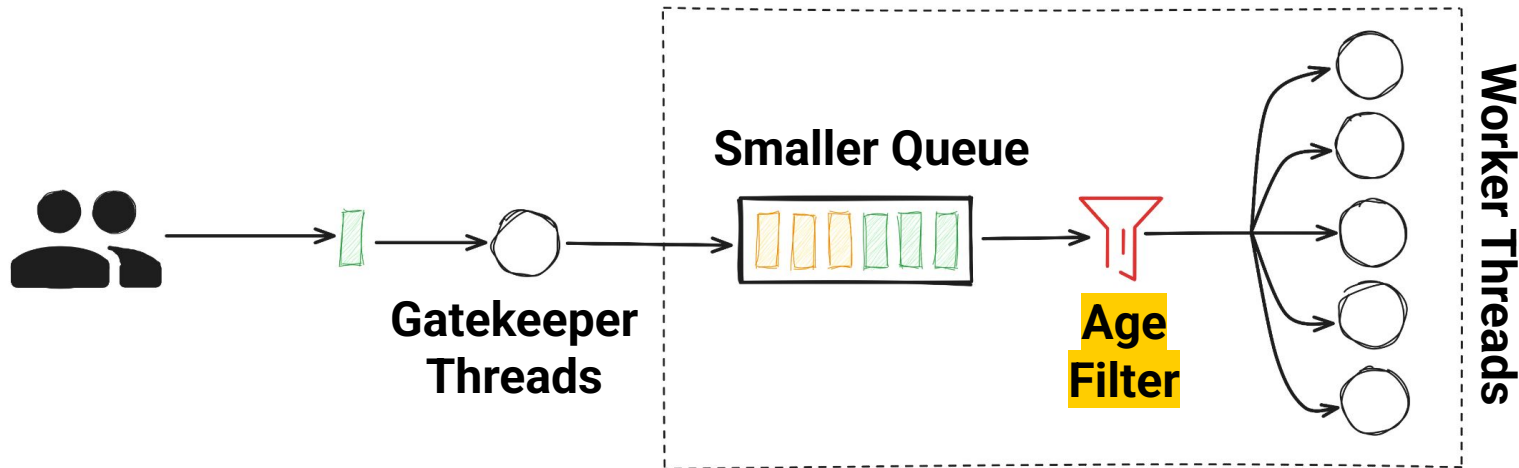
Limiting WIP and Age

- **Bounded wait-time** for clients
- **Effective communication** when there is congestion

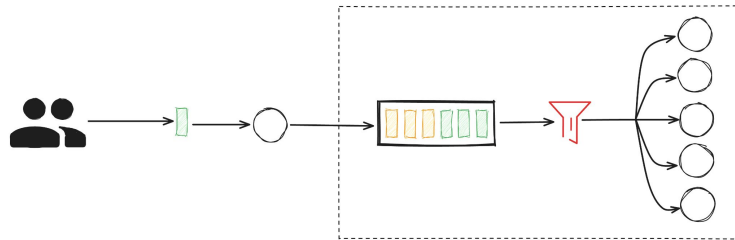


Limiting WIP and Age

- **Bounded wait-time** for clients
- **Effective communication** when there is congestion
- **Quicker** recovery



Implementation





Implementation

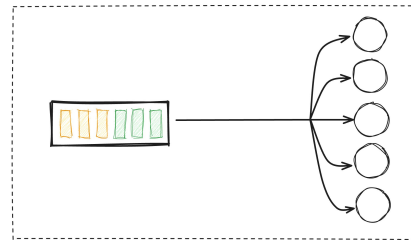
```
new LinkedBlockingQueue<>(30) // WIP Limit
```



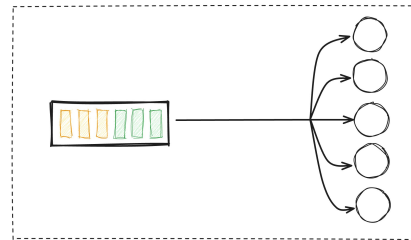


Implementation

```
threadPool = new ThreadPoolExecutor(  
    5,  
    5,  
    0L,  
    TimeUnit.MILLISECONDS,  
    new LinkedBlockingQueue<>(30) // WIP Limit  
);
```



Implementation

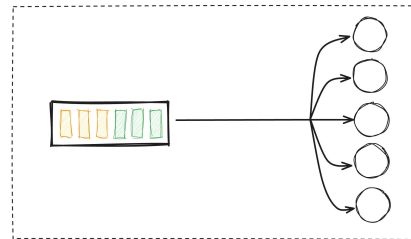




Implementation

```
threadPool.execute((() -> {
```

```
});
```



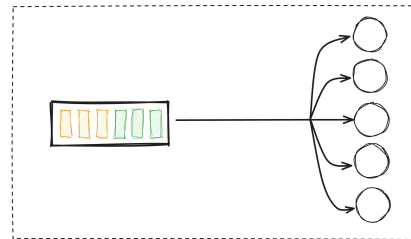


Implementation

```
long enqueueTime = System.currentTimeMillis();
```

```
threadPool.execute((() -> {
```

```
});
```





Implementation

```
long enqueueTime = System.currentTimeMillis();
```

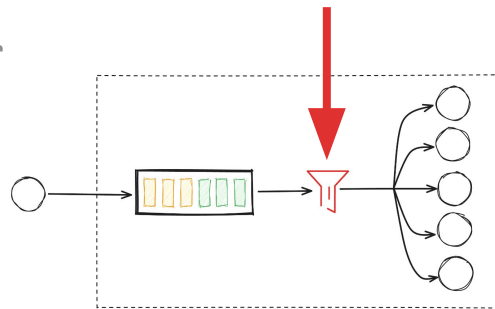
```
threadPool.execute(() -> {
```

```
    if (System.currentTimeMillis() - enqueueTime > 5000) { // age filter
```

```
        return;
```

```
    }
```

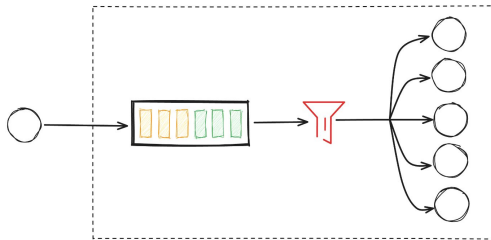
```
});
```





Implementation

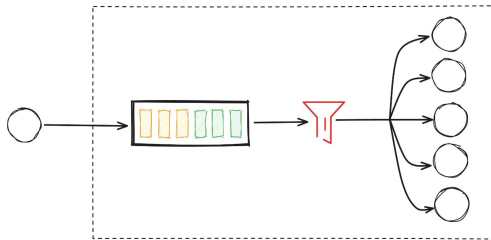
```
long enqueueTime = System.currentTimeMillis();  
var response = new DeferredResult<ResponseEntity<Map<String, String>>>(5000);  
threadPool.execute(() -> {  
    if (System.currentTimeMillis() - enqueueTime > 5000) { // age filter  
  
        return;  
    }  
  
});
```





Implementation

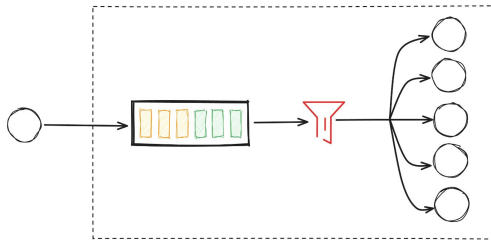
```
long enqueueTime = System.currentTimeMillis();  
var response = new DeferredResult<ResponseEntity<Map<String, String>>>(5000);  
threadPool.execute(() -> {  
    if (System.currentTimeMillis() - enqueueTime > 5000) { // age filter  
        response.setResult(tryAgainLater());  
        return;  
    }  
});
```





Implementation

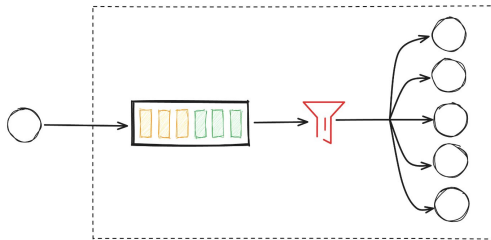
```
long enqueueTime = System.currentTimeMillis();  
var response = new DeferredResult<ResponseEntity<Map<String, String>>>(5000);  
threadPool.execute(() -> {  
    if (System.currentTimeMillis() - enqueueTime > 5000) { // age filter  
        response.setResult(tryAgainLater());  
        return;  
    }  
    response.setResult(ResponseEntity.ok(doSomeComputation()));  
});
```





Implementation

```
public DeferredResult<ResponseEntity<Map<String, String>>> sayHello() {  
    long enqueueTime = System.currentTimeMillis();  
    var response = new DeferredResult<ResponseEntity<Map<String, String>>>(5000);  
    threadPool.execute(() -> {  
        if (System.currentTimeMillis() - enqueueTime > 5000) { // age filter  
            response.setResult(tryAgainLater());  
            return;  
        }  
        response.setResult(ResponseEntity.ok(doSomeComputation()));  
    });  
    return response;  
}
```

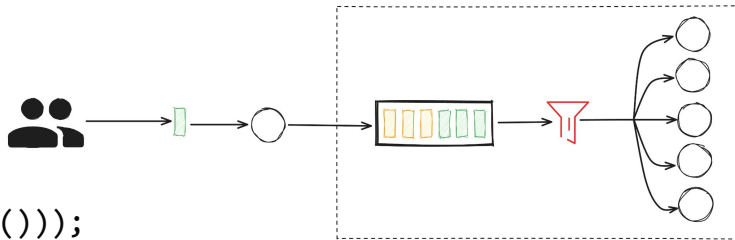




Implementation

```
@GetMapping("/hello")
```

```
public DeferredResult<ResponseEntity<Map<String, String>>> sayHello() {  
    long enqueueTime = System.currentTimeMillis();  
    var response = new DeferredResult<ResponseEntity<Map<String, String>>>(5000);  
    threadPool.execute(() -> {  
        if (System.currentTimeMillis() - enqueueTime > 5000) { // age filter  
            response.setResult(tryAgainLater());  
            return;  
        }  
        response.setResult(ResponseEntity.ok(doSomeComputation()));  
    });  
    return response;  
}
```



Results

github.com/mourjo/**resilience-with-flow-metri**
cs





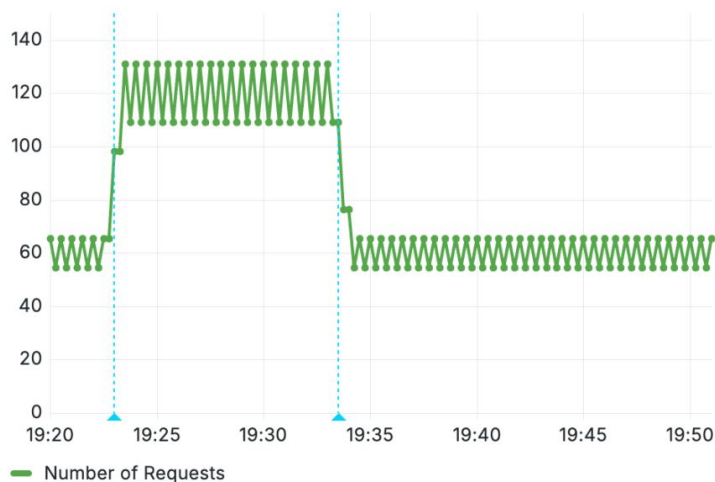
Result: Request Spike



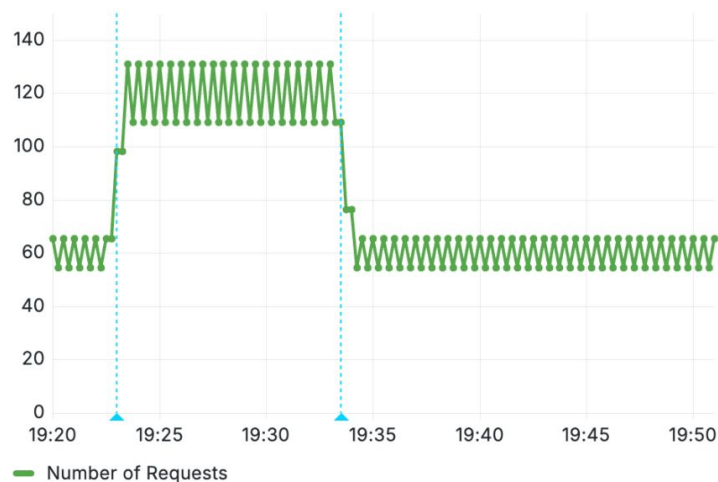
Result: Request Spike

Default Server

Requests Received - NLS



Requests Received - LS

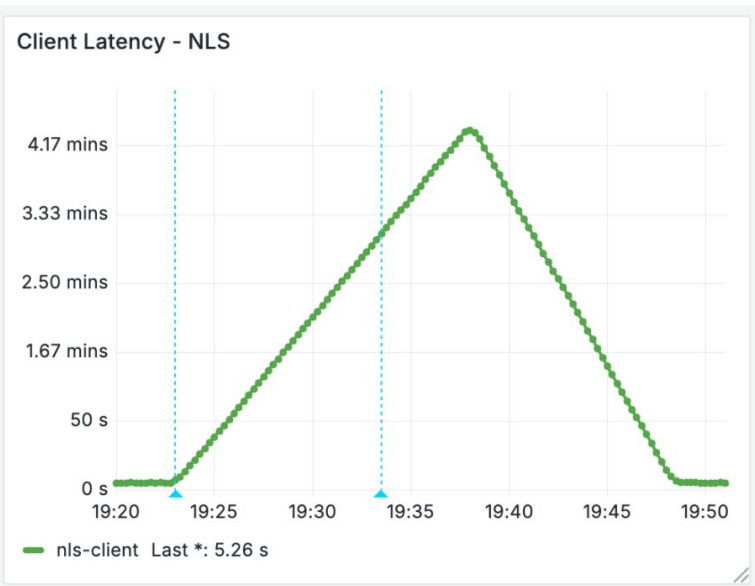


WIP-Age Limiting Server



Result: Request Spike

Default Server

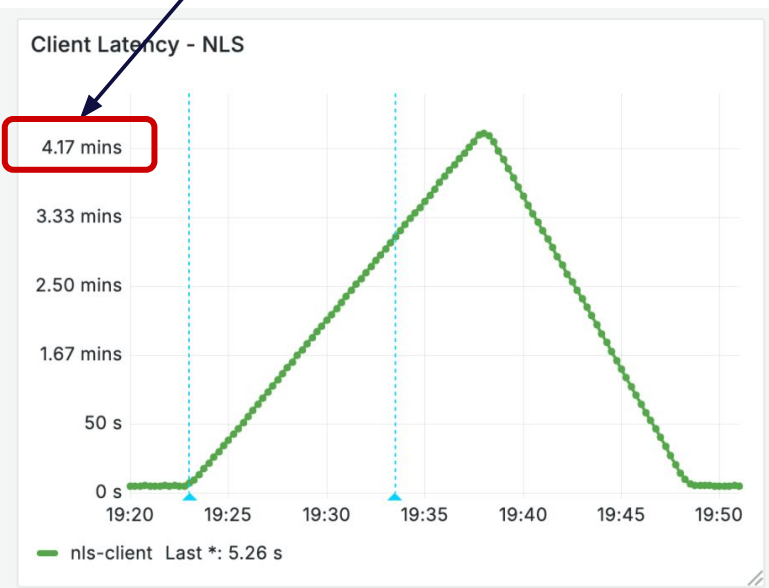




Result: Request Spike

Unresponsive

Default Server





Result: Request Spike

Unresponsive

Default Server

Client Latency - NLS

4.17 mins

3.33 mins

2.50 mins

1.67 mins

50 s

0 s

19:20 19:25 19:30 19:35 19:40 19:45 19:50

— nls-client Last *: 5.26 s

Client Latency - LS

6 s

5 s

4 s

3 s

2 s

1 s

0 s

19:20 19:25 19:30 19:35 19:40 19:45 19:50

— ls-client Last *: 5.24 s

WIP-Age Limiting Server

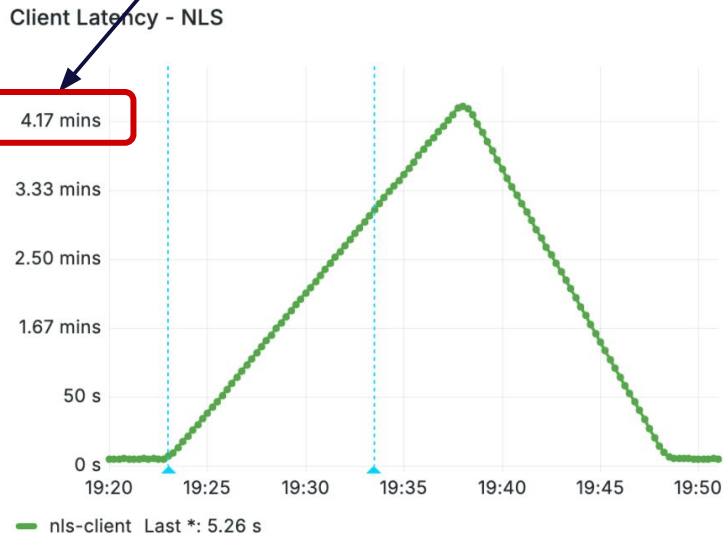


Result: Request Spike

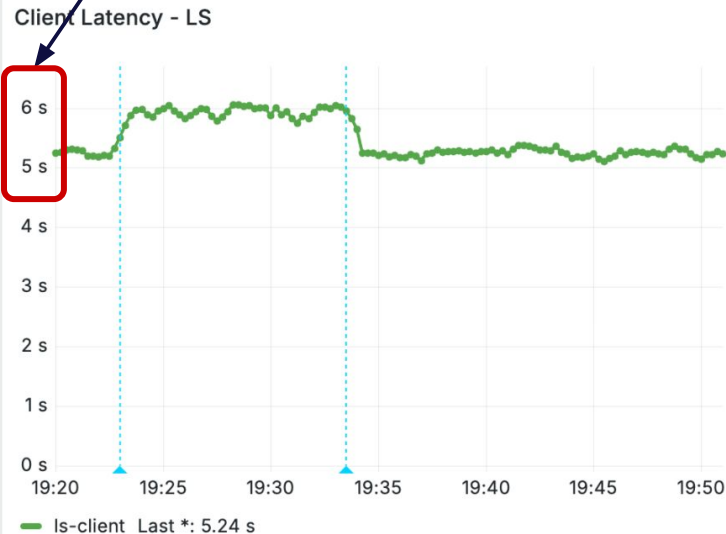
Unresponsive

Responds in time

Default Server



WIP-Age Limiting Server

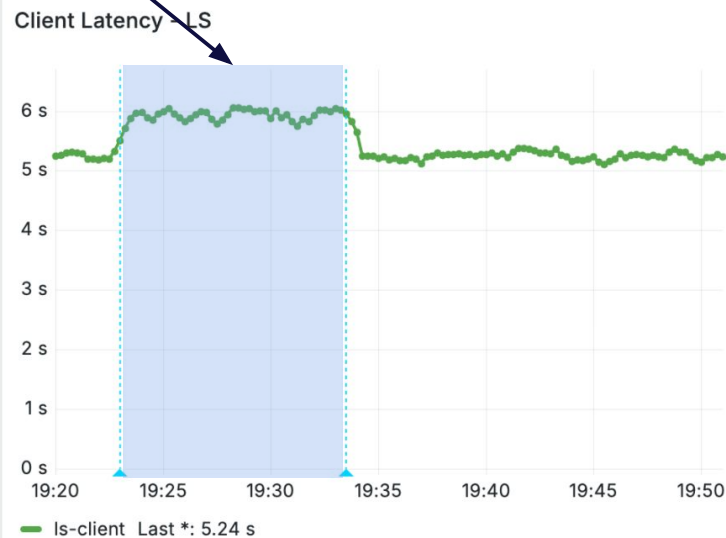
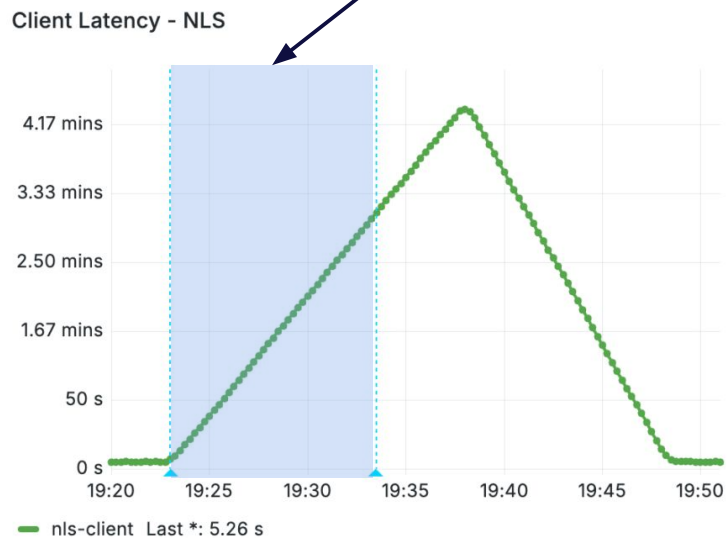




Result: Request Spike

Incident
Duration

Default Server



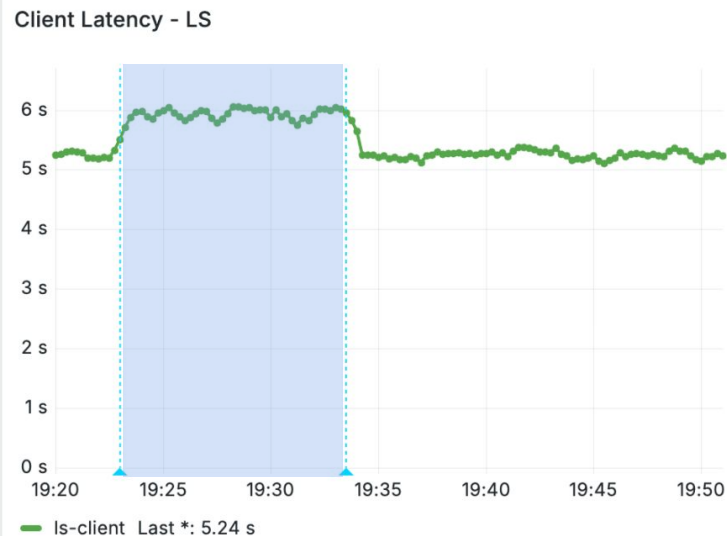
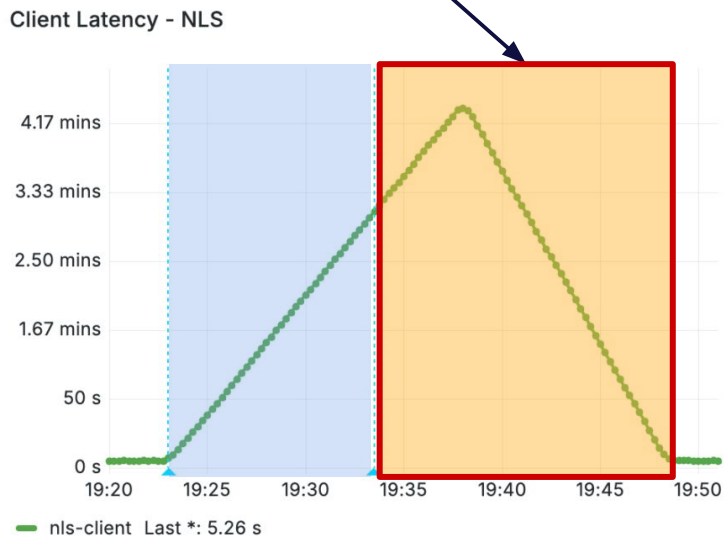
WIP-Age Limiting Server



Result: Request Spike

Time to recover is longer

Default Server



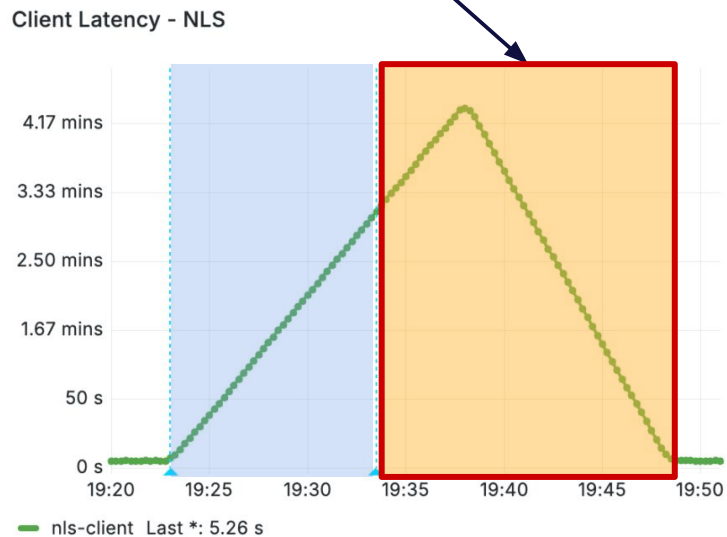
WIP-Age Limiting Server



Result: Request Spike

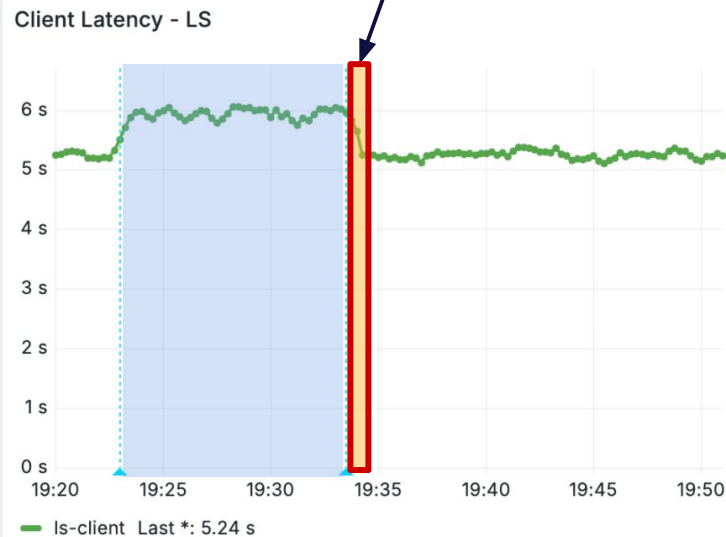
Time to recover is longer

Default Server



Recovers immediately

WIP-Age Limiting Server

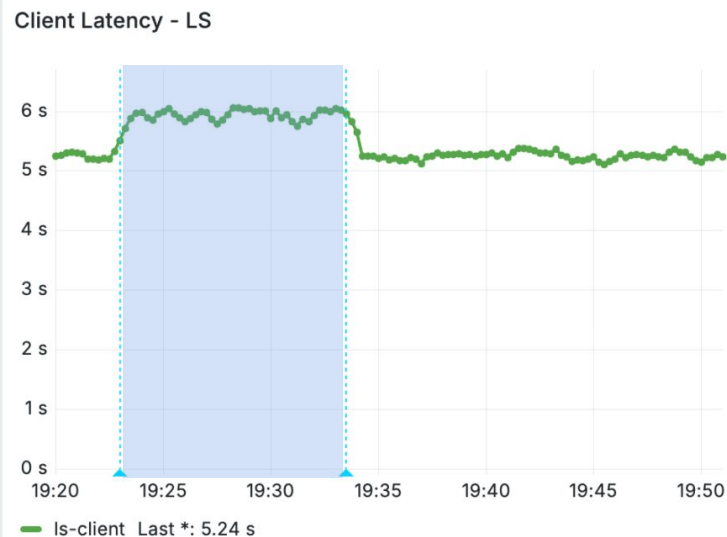
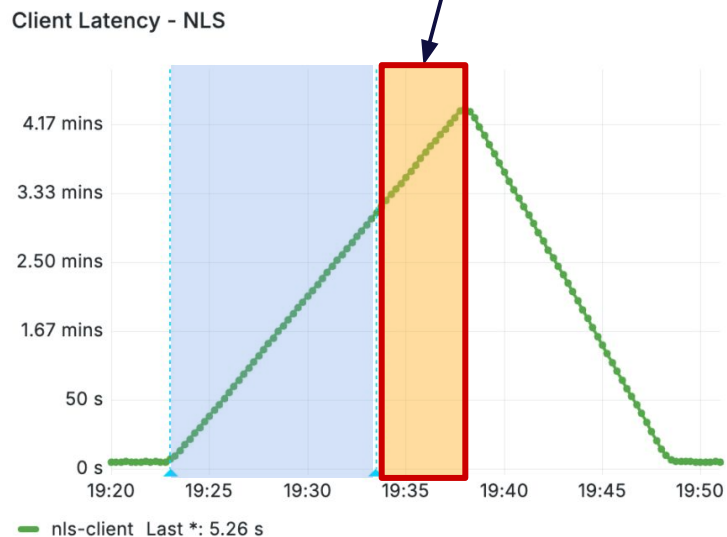




Result: Request Spike

Latency continues to
increase

Default Server

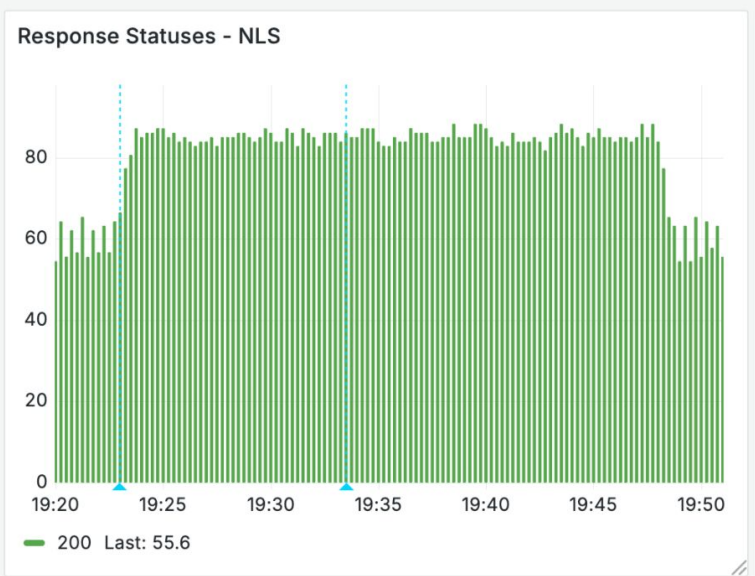


WIP-Age Limiting Server



Result: Request Spike

Default Server





Result: Request Spike

Pretends everything is 200 OK

Default Server

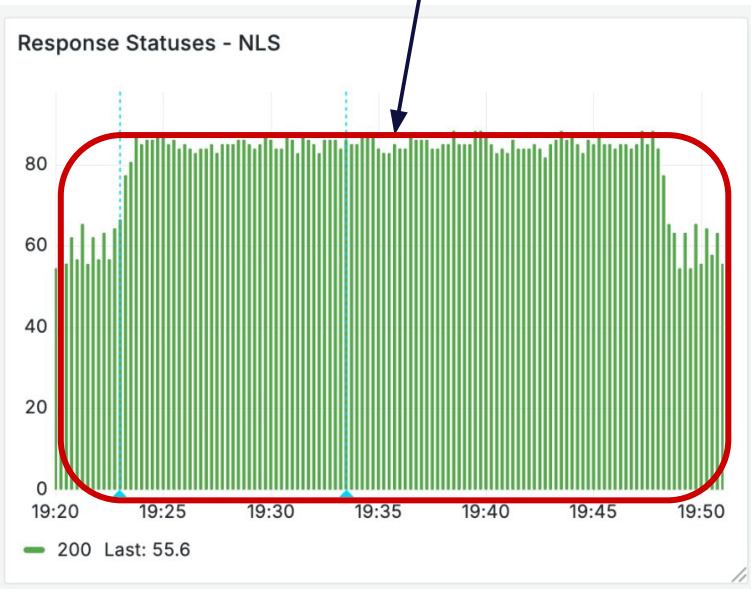




Result: Request Spike

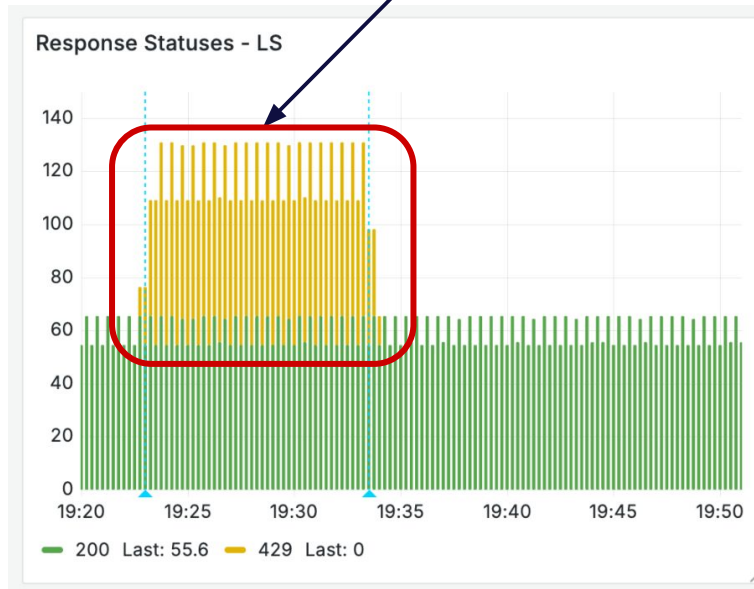
Pretends everything is 200 OK

Default Server



Gracefully degrades

WIP-Age Limiting Server

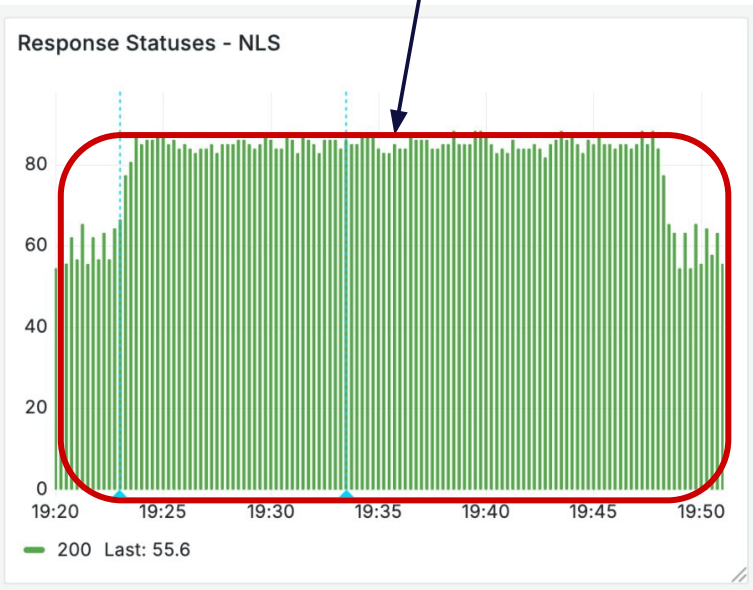




Result: Request Spike

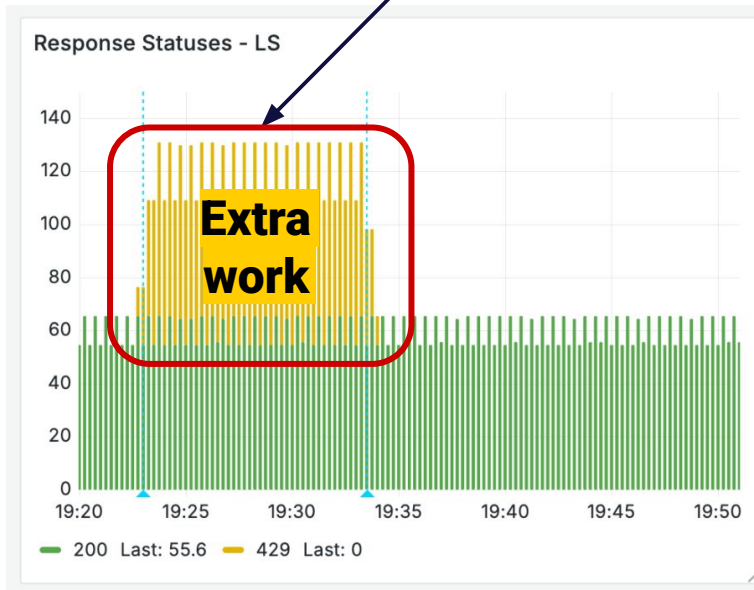
Pretends everything is 200 OK

Default Server



Gracefully degrades

WIP-Age Limiting Server





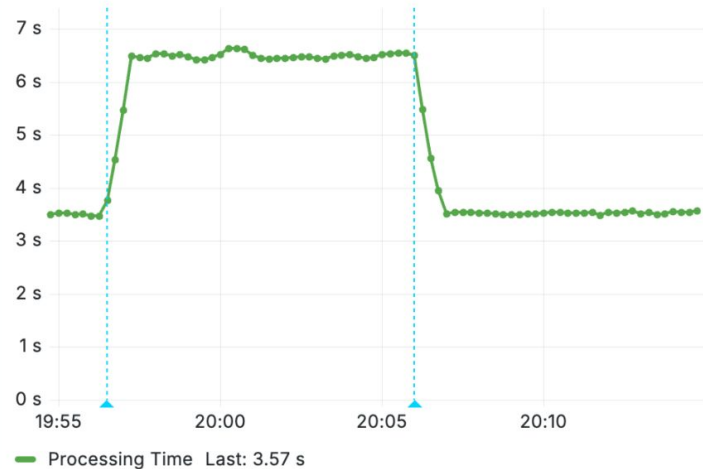
Result: Degraded Dependency



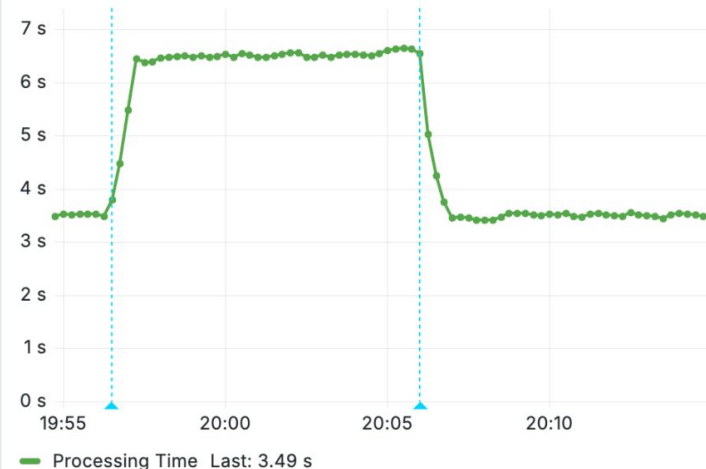
Result: Degraded Dependency

Default Server

Processing Time - NLS



Processing Time - LS



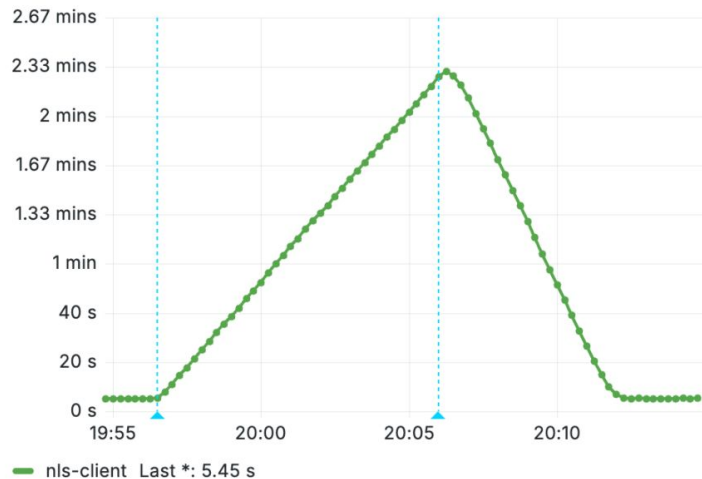
WIP-Age Limiting Server



Result: Degraded Dependency

Default Server

Client Latency - NLS





Result: Degraded Dependency

Unresponsive

Default Server

Client Latency - NLS

2.67 mins

2.33 mins

2 mins

1.67 mins

1.33 mins

1 min

40 s

20 s

0 s

19:55

20:00

20:05

20:10

nls-client Last *: 5.45 s



Result: Degraded Dependency

Unresponsive

Responds in time

Default Server

Client Latency - NLS

2.67 mins
2.33 mins
2 mins

1.67 mins
1.33 mins

1 min

40 s

20 s

0 s

19:55 20:00 20:05 20:10

— nls-client Last *: 5.45 s

Client Latency - LS

7 s
6 s
5 s

4 s

3 s

2 s

1 s

0 s

19:55 20:00 20:05 20:10

— ls-client Last *: 5.24 s

WIP-Age Limiting Server

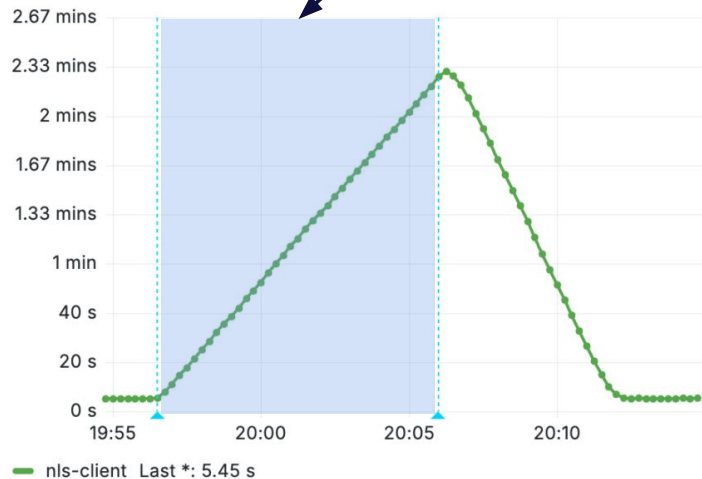


Result: Degraded Dependency

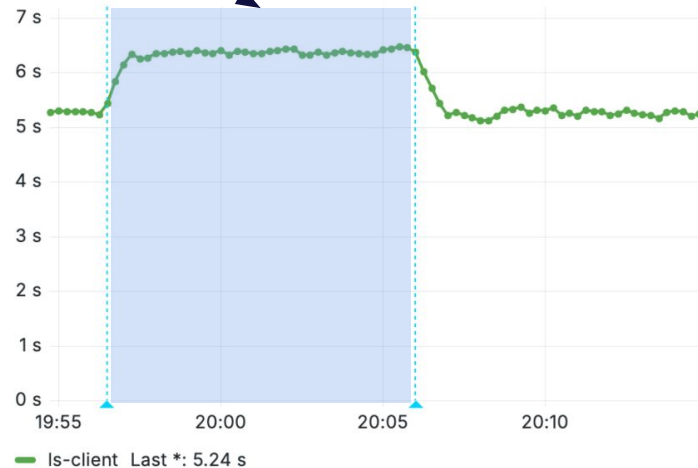
Incident
Duration

Default Server

Client Latency - NLS



Client Latency - LS



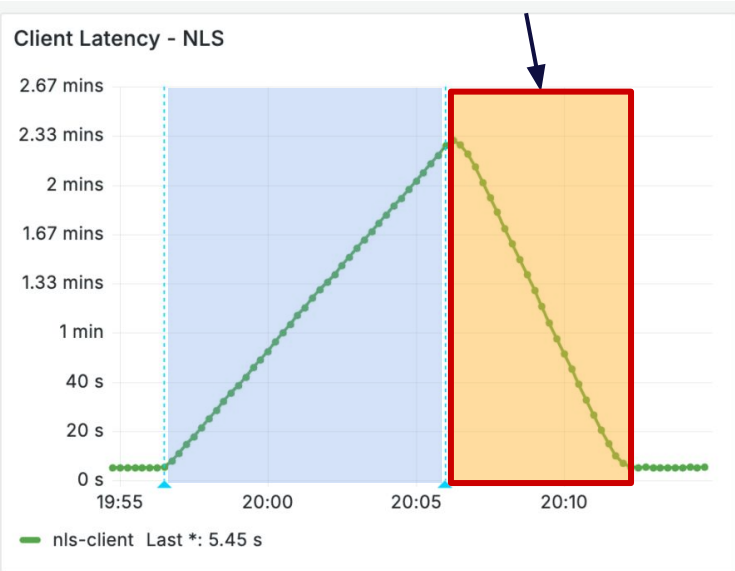
WIP-Age Limiting Server



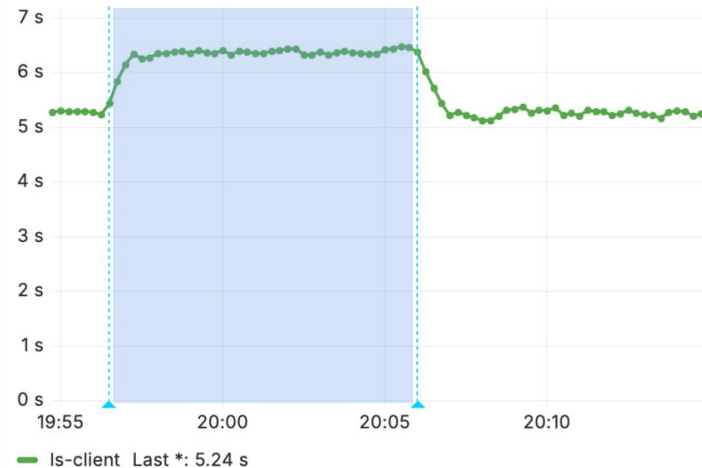
Result: Degraded Dependency

Time to recover is longer

Default Server



Client Latency - LS



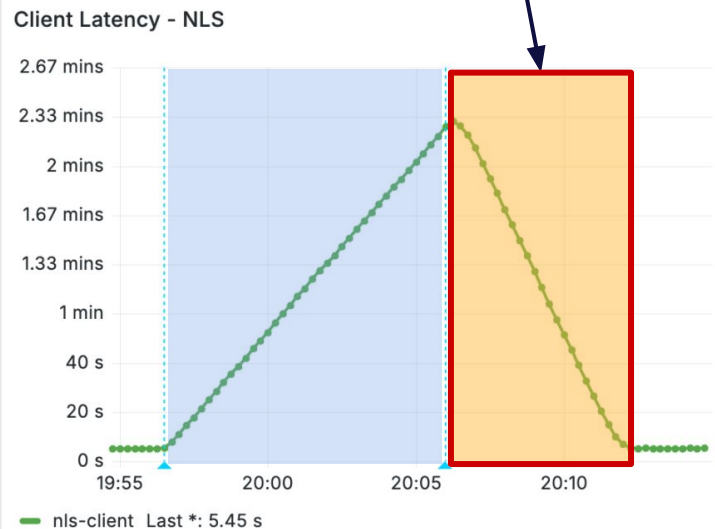
WIP-Age Limiting Server



Result: Degraded Dependency

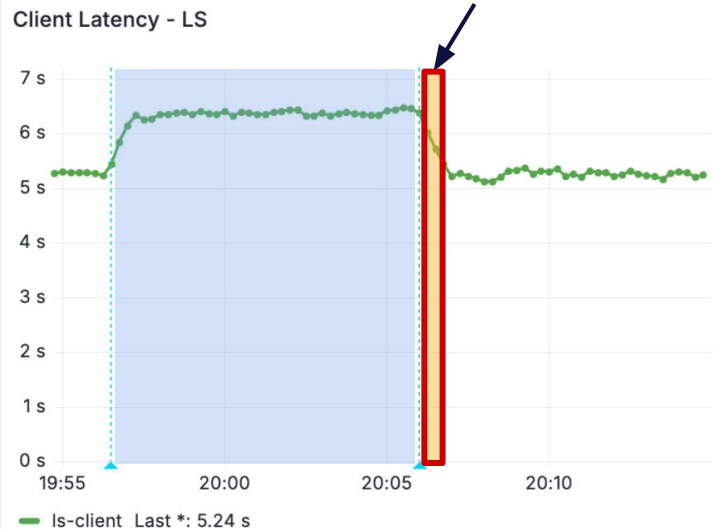
Time to recover is longer

Default Server



Recovers immediately

WIP-Age Limiting Server

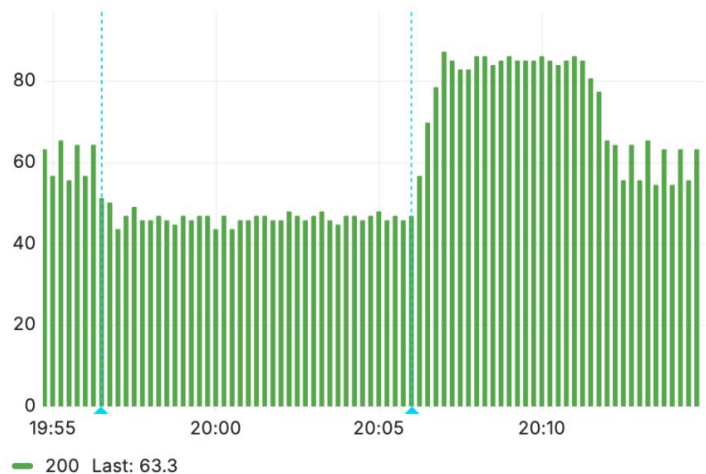




Result: Degraded Dependency

Default Server

Response Statuses - NLS



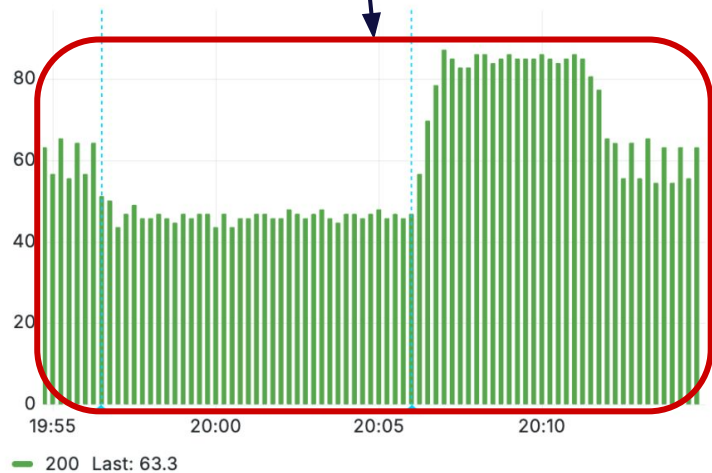


Result: Degraded Dependency

Pretends everything is 200 OK

Default Server

Response Statuses - NLS

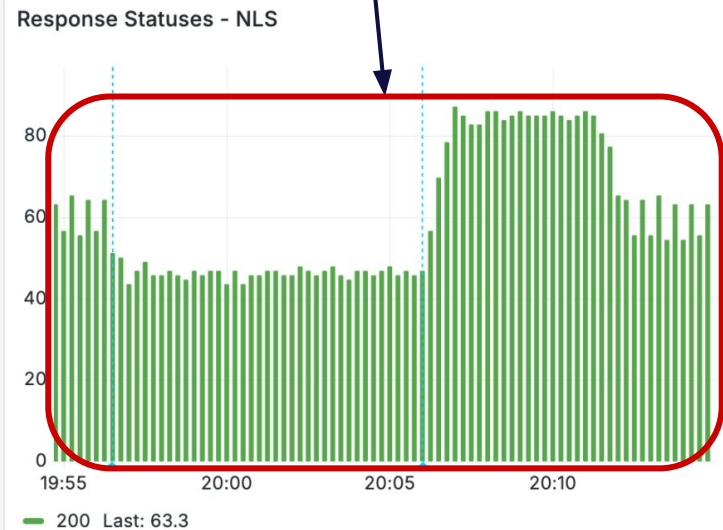




Result: Degraded Dependency

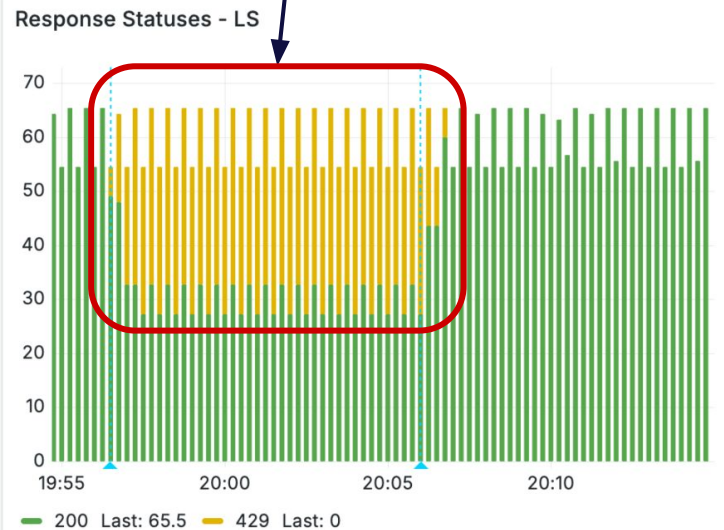
Pretends everything is 200 OK

Default Server



Gracefully degrades

WIP-Age Limiting Server

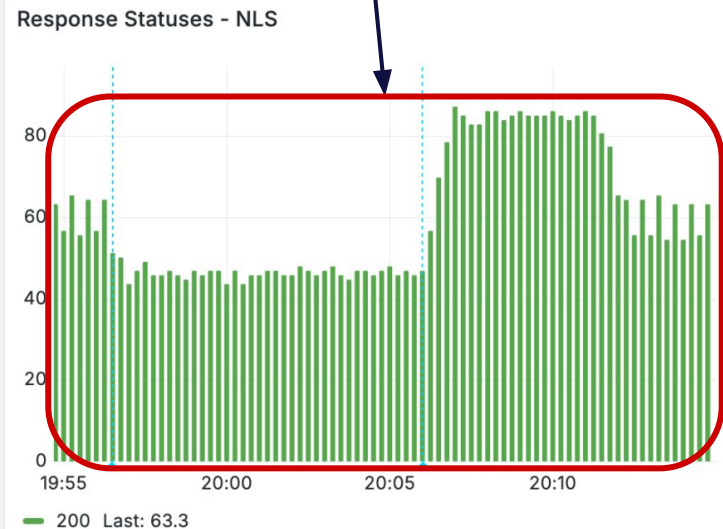




Result: Degraded Dependency

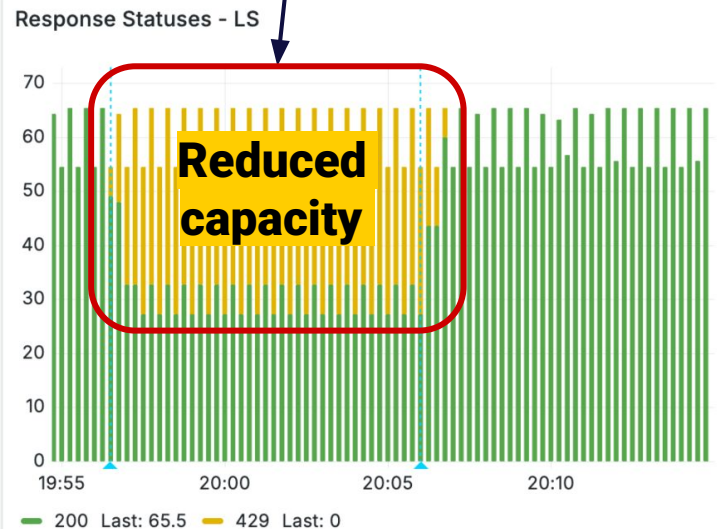
Pretends everything is 200 OK

Default Server



Gracefully degrades

WIP-Age Limiting Server



Applying Resilience Globally



Applying Resilience

Globally

Microservices have
multiple machines



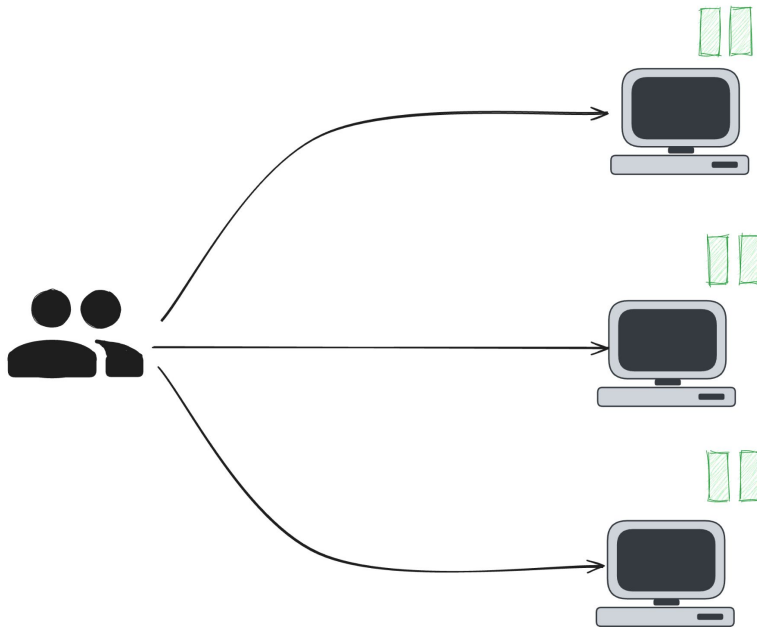


Local Measurements To Global Resilience



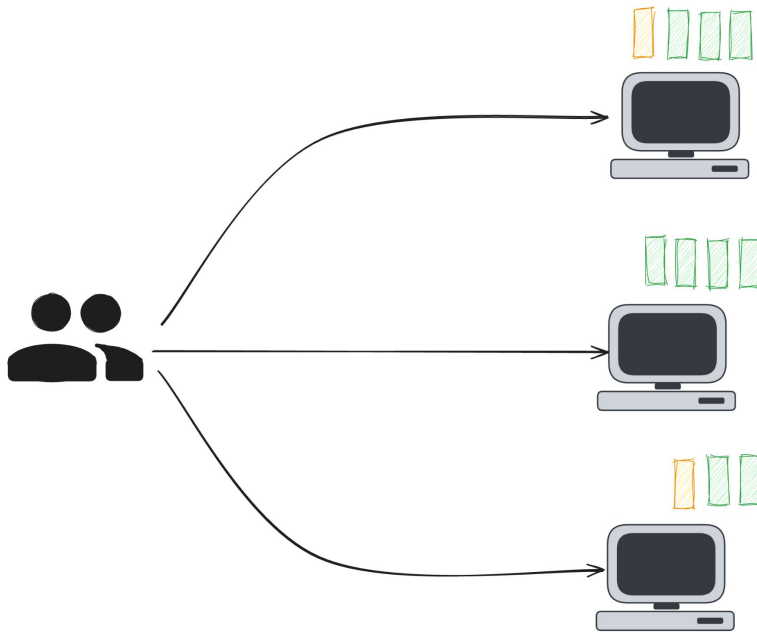


Local Measurements To Global Resilience



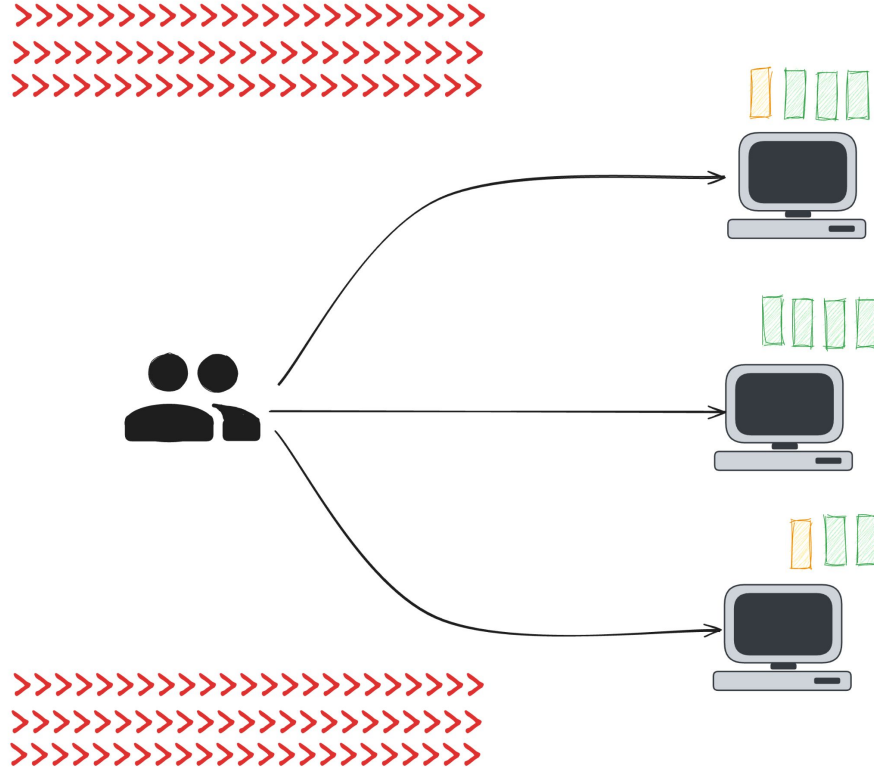


Local Measurements To Global Resilience



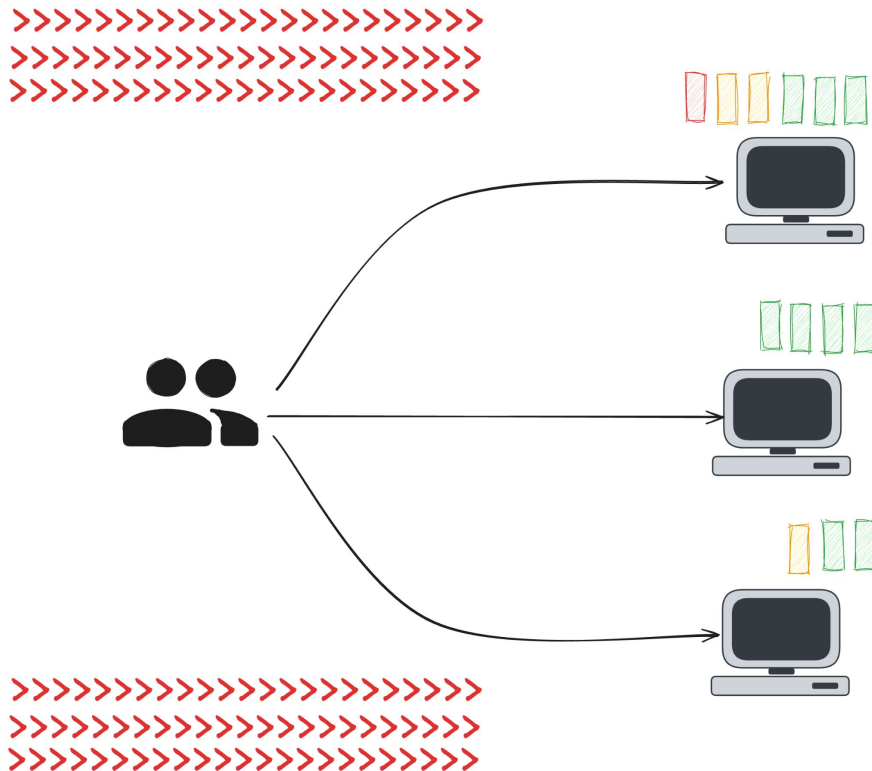


Local Measurements To Global Resilience



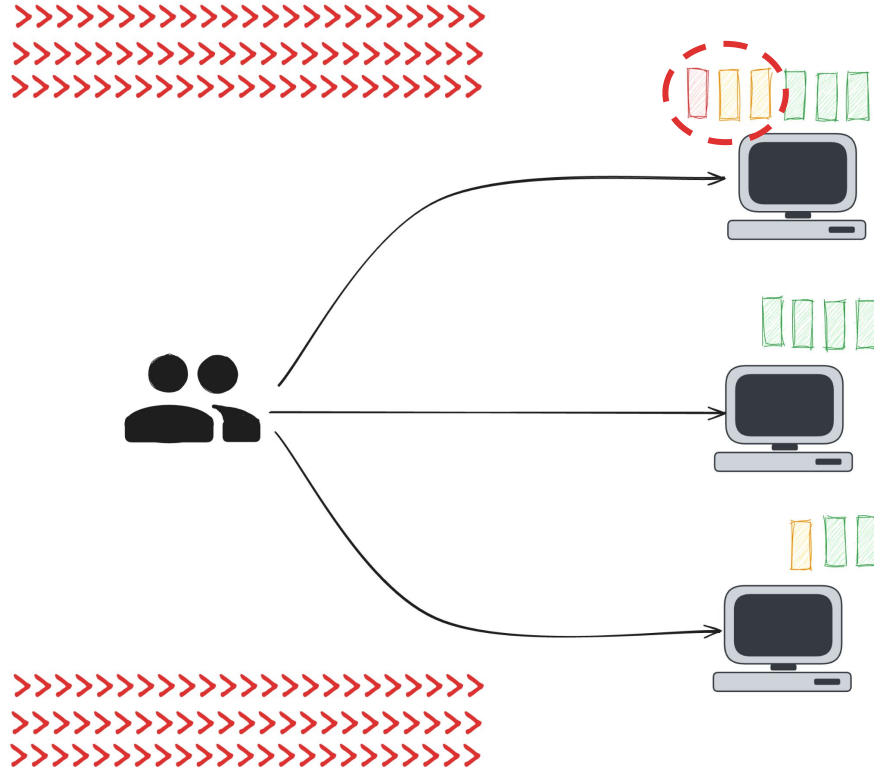


Local Measurements To Global Resilience



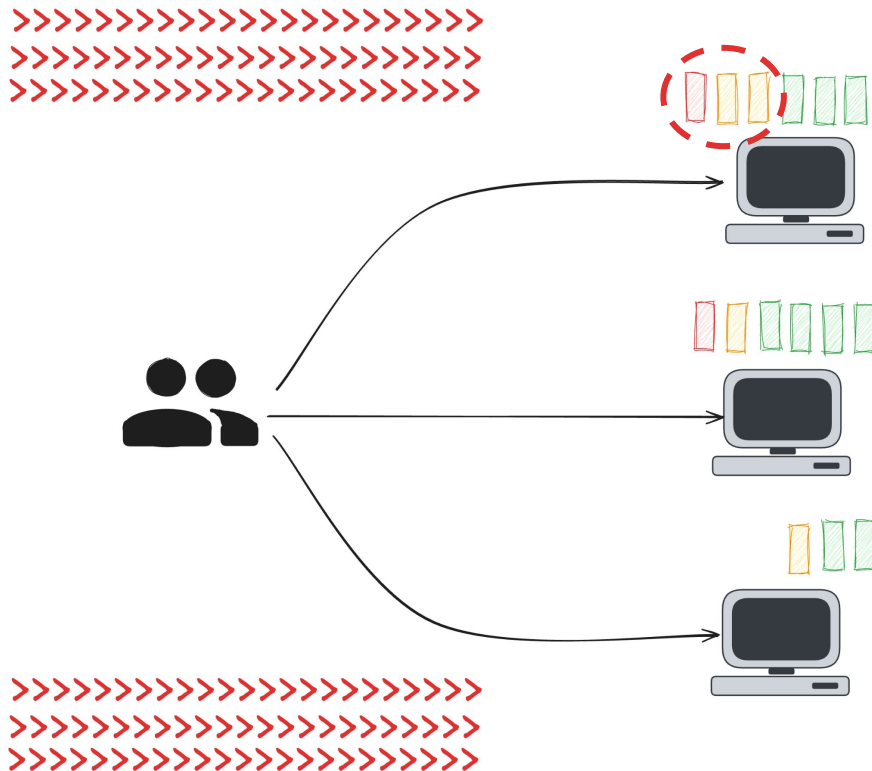


Local Measurements To Global Resilience



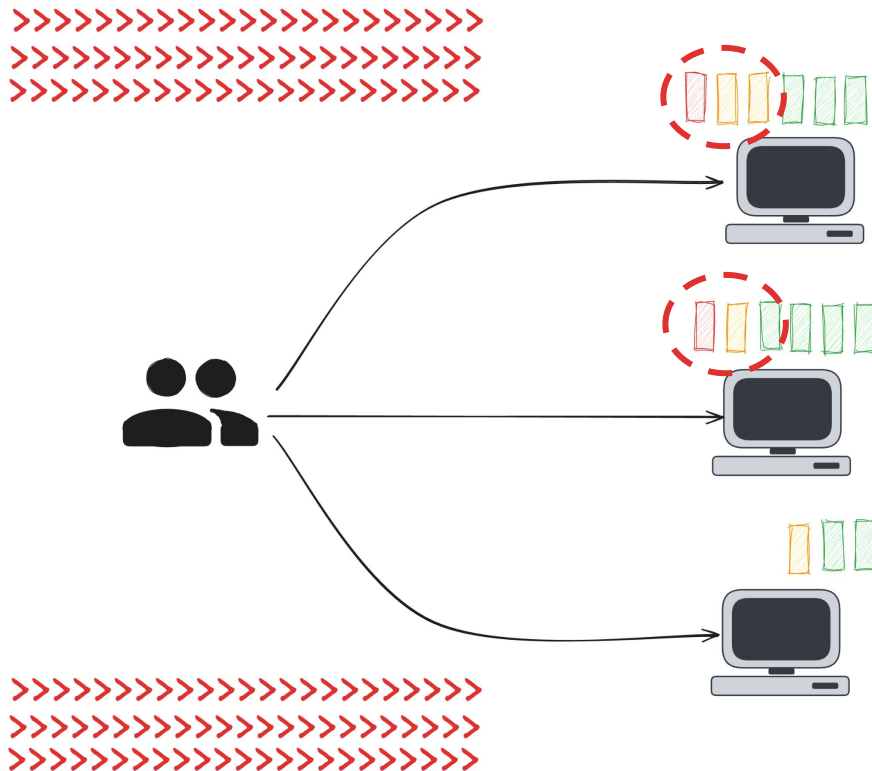


Local Measurements To Global Resilience



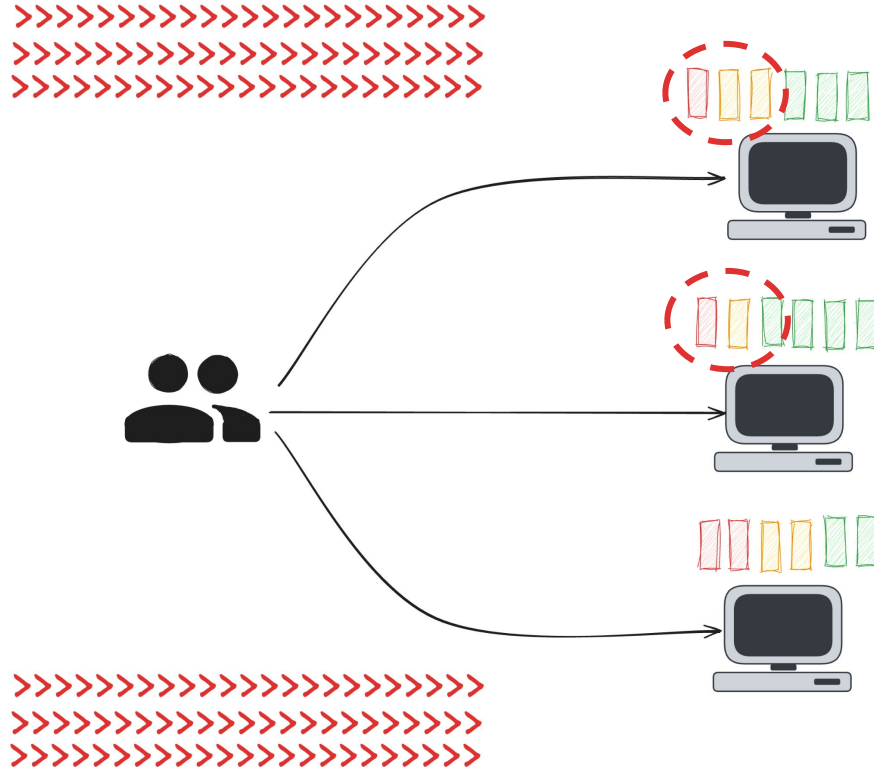


Local Measurements To Global Resilience



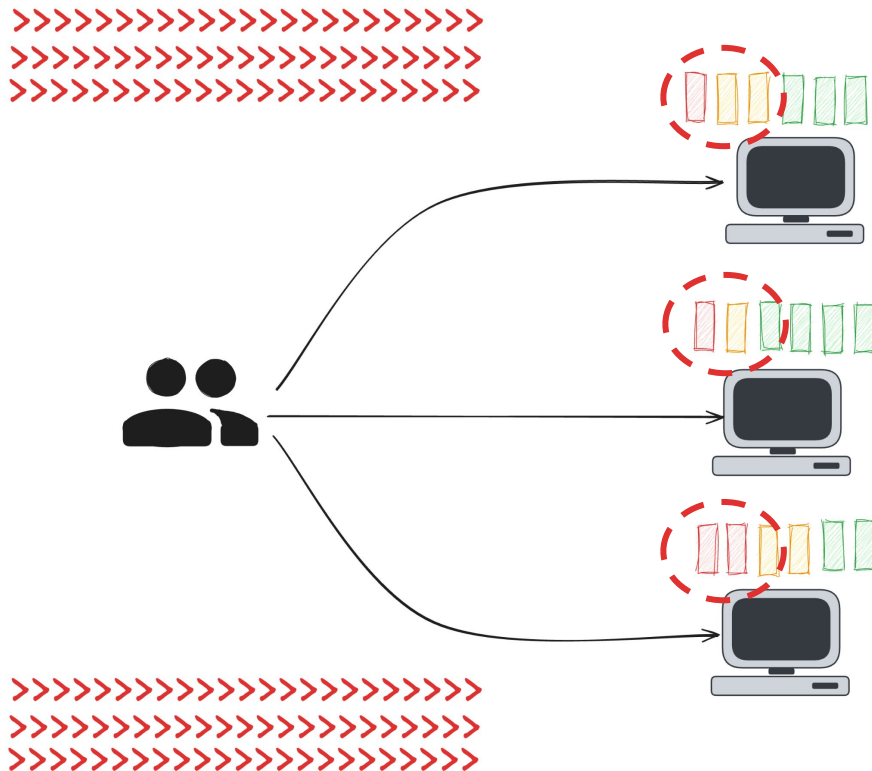


Local Measurements To Global Resilience



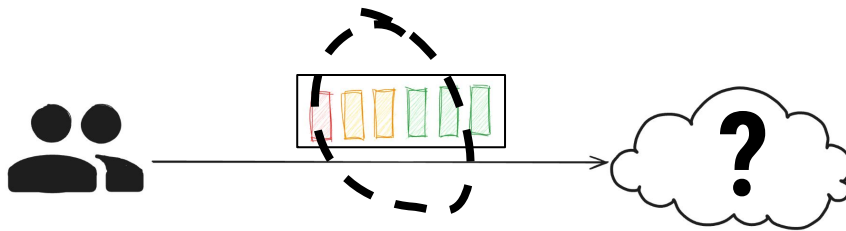


Local Measurements To Global Resilience





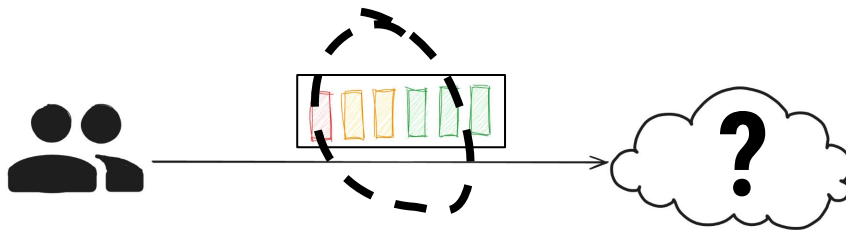
Local Measurements To Global Resilience





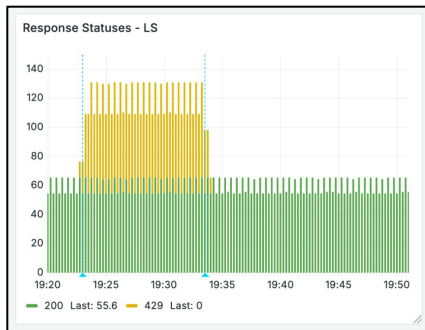
Local Measurements To Global Resilience

How well are requests
flowing through the system?

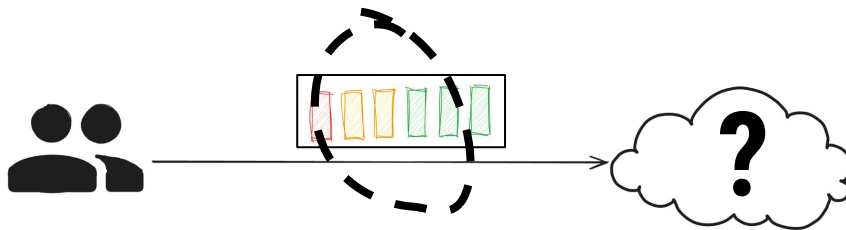




Local Measurements To Global Resilience

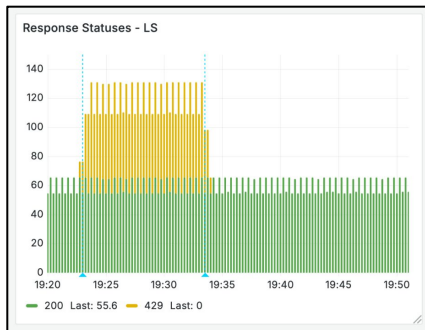


How well are requests
flowing through the system?

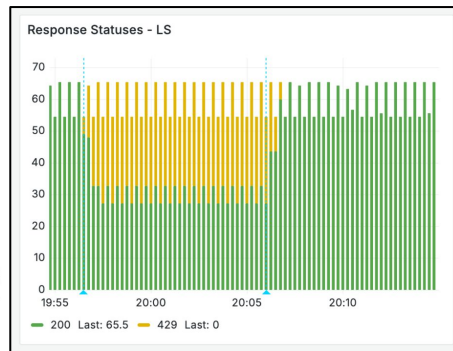
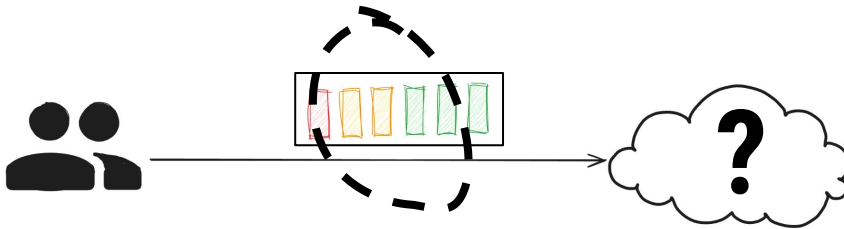




Local Measurements To Global Resilience

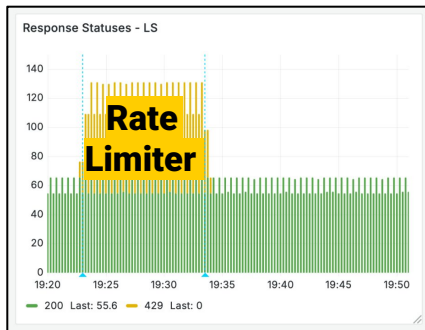


How well are requests
flowing through the system?

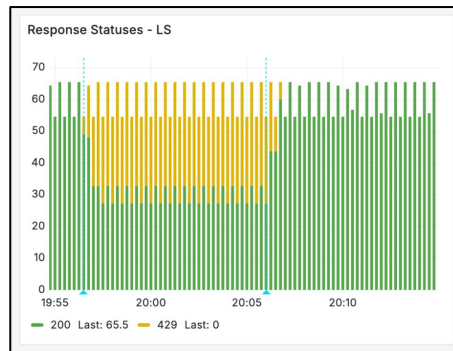
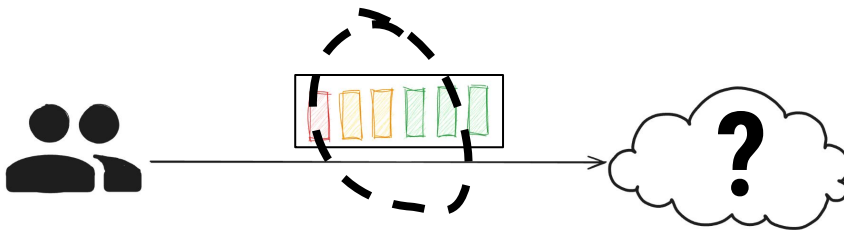




Local Measurements To Global Resilience

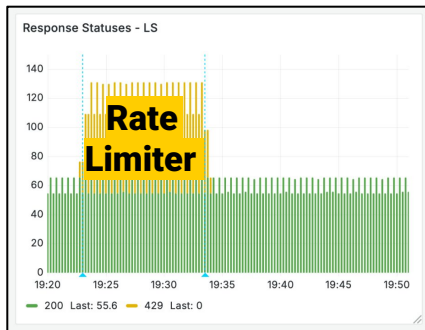


How well are requests
flowing through the system?

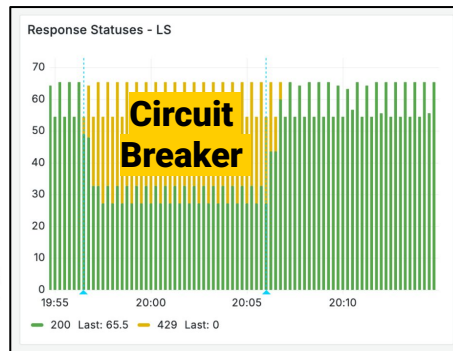
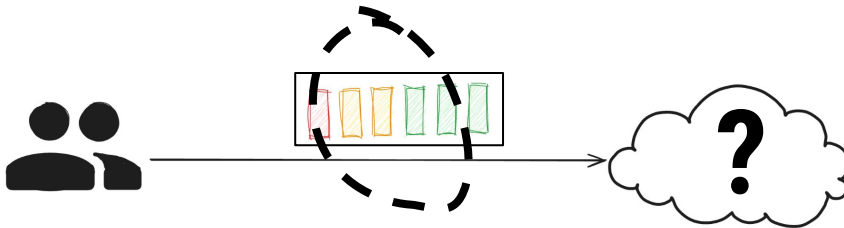




Local Measurements To Global Resilience



How well are requests
flowing through the system?





Dynamic Capacity Estimation



Dynamic Capacity Estimation

- How well we are able to keep up with the demand of work



Dynamic Capacity Estimation

- How well we are able to keep up with the demand of work
- Are requests **flowing** or **accumulating**?
 - Age: Time spent in the system
 - WIP: Number of requests in the system



Dynamic Capacity Estimation

- How well we are able to keep up with the demand of work
- Are requests **flowing** or **accumulating**?
 - Age: Time spent in the system
 - WIP: Number of requests in the system

} **Locally Measurable**



Dynamic Capacity Estimation

- How well we are able to keep up with the demand of work
- Are requests **flowing** or **accumulating**?
 - Age: Time spent in the system
 - WIP: Number of requests in the system
- **Independent** of system characteristics
 - How big is the system?
 - How many users are there?
 - CPU / Memory / GC / etc
 - Type of requests
 - ...

} **Locally Measurable**



Dynamic Capacity Estimation

- How well we are able to keep up with the demand of work
 - Are requests **flowing** or **accumulating**?
 - Age: Time spent in the system
 - WIP: Number of requests in the system
 - **Independent** of system characteristics
 - How big is the system?
 - How many users are there?
 - CPU / Memory / GC / etc
 - Type of requests
 - ...
- } **Locally Measurable**
- } **Universally Applicable**

Load is not what you should balance: Introducing Prequal

Bartek Wydrowski
Google Research

Robert Kleinberg
Google Research

Stephen M. Rumble
Google (YouTube)

Aaron Archer
Google Research

Abstract

We present Prequal (*Probing to Reduce Queuing and Latency*), a load balancer for distributed multi-tenant systems. Prequal aims to minimize real-time request latency in the presence of heterogeneous server capacities and non-uniform, time-varying antagonist load. It actively probes server load to leverage the *power of d choices* paradigm, extending it with asynchronous and reusable probes. Cutting against received wisdom, Prequal does not balance CPU load, but instead selects servers according to estimated latency and active requests-in-flight (RIF). We explore its major design features on a testbed system and evaluate it on YouTube, where it has been deployed for more than two years. Prequal has dramatically decreased tail latency, error rates, and resource use, enabling YouTube and other production systems at Google to run at much higher utilization.

1 Introduction

We report our experience deploying the power of d choices (PodC) load balancing paradigm [1, 2, 18] to run multiple large-scale web services at Google, focusing on YouTube, where it has run successfully for two years. To our knowledge, this is the first public report of PodC being used successfully for load balancing at this scale.

The PodC paradigm involves sampling $d \geq 2$ servers for their load, and sending the next request to the server with minimum load. The two central questions for any implementation are: (1) what signal is used to represent the load, and (2) how is the sampling done? We answer those questions with

primary new innovations. First, we combine RIF and latency in a new way that works especially well, called the *hot-cold lexicographic* (HCL) rule. Second, we introduce a novel asynchronous probing mechanism that reduces probing overheads (CPU + critical-path latency) while retaining the freshness of the load signal offered by synchronous probing.

For purposes of this paper, we refer to Prequal as a load balancing *system*, but technically, it is load a balancing *policy* implemented within Google's Stubby RPC framework (externally, gRPC [24]). As such, we expect that some of our innovations could be integrated into these other existing load balancers. Our main thrust is the two innovations above (HCL, async probing); other implementation details of our system are incidental to our message.

Prequal has been deployed in a diverse collection of 20+ large-scale services at Google over the past two years. In addition to driving most of the YouTube serving stack, it has found success in a wide variety of other applications, including search ads, logs processing, and serving ML models. In the typical application, query processing times are $O(10 - 100)$ milliseconds, but in one system the queries take $O(10)$ minutes, and in another they range from seconds to hours. Most of these services are part of a complex tangle of services calling each other, and we have seen benefits from deploying Prequal for the most critical services, regardless of whether others in the tangle are also using Prequal.

We care about CPU and latency overheads from probing, but in our deployment experience we have found them to be small and more than pay for themselves. Probe response times within a data center are well below 1 millisecond. For some applications, fractions of a millisecond matter, so we

Load is not what you should balance: Introducing Prequal

Bartek Wydrowski
Google Research

Robert Kleinberg
Google Research

Stephen M. Rumble
Google (YouTube)

Aaron Archer
Google Research

Abstract

primary new innovations. First, we combine RIF and latency in a new way that works especially well, called the *hot-cold*

with asynchronous and reusable probes. Cutting against received wisdom, Prequal does not balance CPU load, but instead selects servers according to estimated latency and active requests-in-flight (RIF). We explore its major design features

enabling YouTube and other production systems at Google to run at much higher utilization.

1 Introduction

We report our experience deploying the power of d choices (PodC) load balancing paradigm [1, 2, 18] to run multiple large-scale web services at Google, focusing on YouTube, where it has run successfully for two years. To our knowledge, this is the first public report of PodC being used successfully for load balancing at this scale.

The PodC paradigm involves sampling $d \geq 2$ servers for their load, and sending the next request to the server with minimum load. The two central questions for any implementation are: (1) what signal is used to represent the load, and (2) how is the sampling done? We answer these questions with

Prequal has been deployed in a diverse collection of 20+ large-scale services at Google over the past two years. In addition to driving most of the YouTube serving stack, it has found success in a wide variety of other applications, including search ads, logs processing, and serving ML models. In the typical application, query processing times are $O(10 - 100)$ milliseconds, but in one system the queries take $O(10)$ minutes, and in another they range from seconds to hours. Most of these services are part of a complex tangle of services calling each other, and we have seen benefits from deploying Prequal for the most critical services, regardless of whether others in the tangle are also using Prequal.

We care about CPU and latency overheads from probing, but in our deployment experience we have found them to be small and more than pay for themselves. Probe response times within a data center are well below 1 millisecond. For some applications, fractions of a millisecond matter, so we

Load is not what you should balance: Introducing Prequal

Bartek Wydrowski
Google Research

Robert Kleinberg
Google Research

Stephen M. Rumble
Google (YouTube)

Aaron Archer
Google Research

Abstract

primary new innovations. First, we combine RIF and latency in a new way that works especially well, called the *hot-cold*

with asynchronous and reusable probes. Cutting against received wisdom, Prequal does not balance CPU load, but instead selects servers according to estimated latency and active requests-in-flight (RIF). We explore its major design features

enabling YouTube and other production systems at Google to run at much higher utilization.

1 Introduction

We report our experience deploying the power of d choices (PodC) load balancing paradigm [1, 2, 18] to run multiple large-scale web services at Google, focusing on YouTube, where it has run successfully for two years. To our knowledge, this is the first public report of PodC being used successfully for load balancing at this scale.

The PodC paradigm involves sampling $d \geq 2$ servers for their load, and sending the next request to the server with minimum load. The two central questions for any implementation are: (1) what signal is used to represent the load, and (2) how is the sampling done? We answer those questions with

Prequal has been deployed in a diverse collection of 20+ large-scale services at Google over the past two years. In addition to driving most of the YouTube serving stack, it has found success in a wide variety of other applications, including search ads, logs processing, and serving ML models. In the typical application, query processing times are $O(10 - 100)$ milliseconds, but in one system the queries take $O(10)$ minutes, and in another they range from seconds to hours. Most of these services are part of a complex tangle of services calling each other, and we have seen benefits from deploying Prequal for the most critical services, regardless of whether others in the tangle are also using Prequal.

We care about CPU and latency overheads from probing, but in our deployment experience we have found them to be small and more than pay for themselves. Probe response times within a data center are well below 1 millisecond. For some applications, fractions of a millisecond matter, so we

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this period, the data throughput from LBL to UC Berkeley (sites separated by 400 yards and three IMP hops) dropped from 32 Kbps to 40 bps. Mike Karels¹ and I were fascinated by this sudden factor-of-thousand drop in bandwidth and embarked on an investigation of why things had gotten so bad. We wondered, in particular, if the 4.3BSD (Berkeley UNIX) TCP was mis-behaving or if it could be tuned to work better under abysmal network conditions. The answer to both of these questions was "yes".

Since that time, we have put seven new algorithms into the 4BSD TCP:

- (i) round-trip-time variance estimation
- (ii) exponential retransmit timer backoff
- (iii) slow-start
- (iv) more aggressive receiver ack policy
- (v) dynamic window sizing on congestion
- (vi) Karn's clamped retransmit backoff
- (vii) fast retransmit

Our measurements and the reports of beta testers suggest that the final product is fairly good at dealing with congested conditions on the Internet.

* This work was supported in part by the U.S. Department of Energy under Contract Number DE-AC03-76SF00098.

¹ The algorithms and ideas described in this paper were developed in collaboration with Mike Karels of the UC Berkeley Computer System Research Group. The reader should assume that anything clever is due to Mike. Opinions and mistakes are the property of the author.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

This paper is a brief description of (i) - (v) and the rationale behind them. (vi) is an algorithm recently developed by Phil Karn of Bell Communications Research, described in [KP87]. (vii) is described in a soon-to-be-published RFC.

Algorithms (i) - (v) spring from one observation: The flow on a TCP connection (or ISO TP-4 or Xerox NS SPP connection) should obey a 'conservation of packets' principle. And, if this principle were obeyed, congestion collapse would become the exception rather than the rule. Thus congestion control involves finding places that violate conservation and fixing them.

By 'conservation of packets' I mean that for a connection 'in equilibrium', i.e., running stably with a full window of data in transit, the packet flow is what a physicist would call 'conservative': A new packet isn't put into the network until an old packet leaves. The physics of flow predicts that systems with this property should be robust in the face of congestion. Observation of the Internet suggests that it was not particularly robust. Why the discrepancy?

There are only three ways for packet conservation to fail:

1. The connection doesn't get to equilibrium, or
2. A sender injects a new packet before an old packet has exited, or
3. The equilibrium can't be reached because of resource limits along the path.

In the following sections, we treat each of these in turn.

1 Getting to Equilibrium: Slow-start

Failure (1) has to be from a connection that is either starting or restarting after a packet loss. Another way to look at the conservation property is to say that the receiver uses acks as a 'clock' to strobe new packets into the network. Since the receiver can generate acks no faster than data packets can get through the network,



dl.acm.org

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this

This paper is a brief description of (i) - (v) and the rationale behind them. (vi) is an algorithm recently devel-

* This work was supported in part by the U.S. Department of Energy under Contract Number DE-AC03-76SF00098.

¹ The algorithms and ideas described in this paper were developed in collaboration with Mike Karels of the UC Berkeley Computer System Research Group. The reader should assume that anything clever is due to Mike. Opinions and mistakes are the property of the author.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee or specific permission.

1988 ACM 0-89791-279-9/88/008/0314

source limits along the path.

In the following sections, we treat each of these in turn.

1 Getting to Equilibrium: Slow-start

Failure (1) has to be from a connection that is either starting or restarting after a packet loss. Another way to think of the conservation property is to say that the receiver generates acks as a 'clock' to strobe new packets into the network. Since the receiver can generate acks no faster than data packets can get through the network,



dl.acm.org

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this

This paper is a brief description of (i) - (v) and the rationale behind them. (vi) is an algorithm recently devel-

WIP (Congestion Window)

Time

* This work was supported in part by the U.S. Department of Energy under Contract Number DE-AC03-76SF00098.

¹ The algorithms and ideas described in this paper were developed in collaboration with Mike Karels of the UC Berkeley Computer System Research Group. The reader should assume that anything clever is due to Mike. Opinions and mistakes are the property of the author.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires prior permission or specific permission.

1988 ACM 0-89791-279-9/88/008/0314

source limits along the path.

In the following sections, we treat each of these in turn.

1 Getting to Equilibrium: Slow-start

Failure (1) has to be from a connection that is either starting or restarting after a packet loss. Another way to think of the conservation property is to say that the receiver generates acks as a 'clock' to strobe new packets into the network. Since the receiver can generate acks no faster than data packets can get through the network,



dl.acm.org

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this

This paper is a brief description of $(i)-(v)$ and the rationale behind them. (vi) is an algorithm recently devel-

* This work was supported in part by the U.S. Department of Energy under Contract Number DE-AC03-76SF00098.

[†] The algorithms and ideas described in this paper were developed in collaboration with Mike Karels of the UC Berkeley Computer System Research Group. The reader should assume that anything clever is due to Mike. Opinions and mistakes are the property of the author.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee or specific permission.

1988 ACM 0-89791-279-9/88/008/0314

source limits along the path.

In the following sections, we treat each of these in turn.

1 Getting to Equilibrium: Slow-start

Failure (1) has to be from a connection that is either starting or restarting after a packet loss. Another way to think of the conservation property is to say that the receiver generates acks as a 'clock' to strobe new packets into the network. Since the receiver can generate acks no faster than data packets can get through the network,



dl.acm.org

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this

This paper is a brief description of $(i)-(v)$ and the rationale behind them. (vi) is an algorithm recently devel-

* This work was supported in part by the U.S. Department of Energy under Contract Number DE-AC03-76SF00098.

[†] The algorithms and ideas described in this paper were developed in collaboration with Mike Karels of the UC Berkeley Computer System Research Group. The reader should assume that anything clever is due to Mike. Opinions and mistakes are the property of the author.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires prior specific permission.

1988 ACM 0-89791-279-9/88/008/0314

source limits along the path.

In the following sections, we treat each of these in turn.

1 Getting to Equilibrium: Slow-start

Failure (1) has to be from a connection that is either starting or restarting after a packet loss. Another way to think of the conservation property is to say that the receiver generates acks as a 'clock' to strobe new packets into the network. Since the receiver can generate acks no faster than data packets can get through the network,



dl.acm.org

Congestion Avoidance and Control

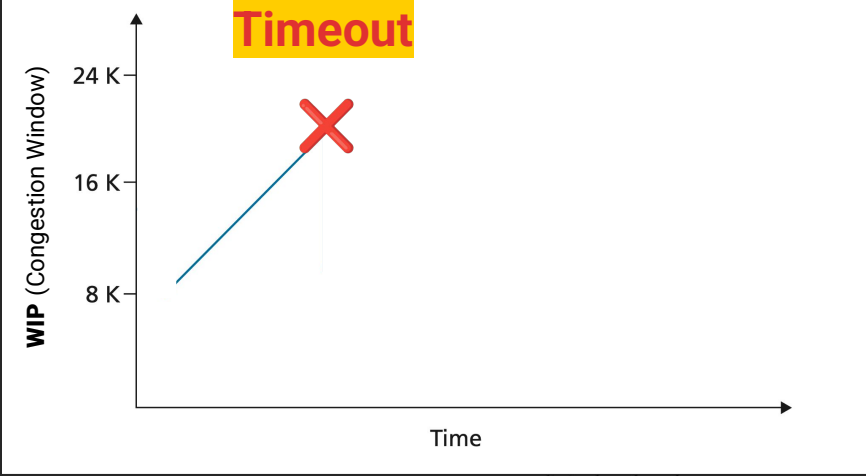
Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this

This paper is a brief description of (i) - (v) and the rationale behind them. (vi) is an algorithm recently devel-

Timeout



* This work was supported in part by the U.S. Department of Energy under Contract Number DE-AC03-76SF00098.

The algorithms and ideas described in this paper were developed in collaboration with Mike Karels of the UC Berkeley Computer System Research Group. The reader should assume that anything clever is due to Mike. Opinions and mistakes are the property of the author.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee or specific permission.

1988 ACM 0-89791-279-9/88/008/0314

source limits along the path.

In the following sections, we treat each of these in turn.

1 Getting to Equilibrium: Slow-start

Failure (1) has to be from a connection that is either starting or restarting after a packet loss. Another way to think of the conservation property is to say that the receiver generates acks as a 'clock' to strobe new packets into the network. Since the receiver can generate acks no faster than data packets can get through the network,



dl.acm.org

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this

This paper is a brief description of (i) - (v) and the rationale behind them. (vi) is an algorithm recently devel-

* This work was supported in part by the U.S. Department of Energy under Contract Number DE-AC03-76SF00098.

The algorithms and ideas described in this paper were developed in collaboration with Mike Karels of the UC Berkeley Computer System Research Group. The reader should assume that anything clever is due to Mike. Opinions and mistakes are the property of the author.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires prior specific permission.

1988 ACM 0-89791-279-9/88/008/0314

source limits along the path.

In the following sections, we treat each of these in turn.

1 Getting to Equilibrium: Slow-start

Failure (1) has to be from a connection that is either starting or restarting after a packet loss. Another way to look at the conservation property is to say that the receiver generates acks as a 'clock' to strobe new packets into the network. Since the receiver can generate acks no faster than data packets can get through the network,



dl.acm.org

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this

This paper is a brief description of (i) - (v) and the rationale behind them. (vi) is an algorithm recently devel-

Timeout

Age too high

Reduce WIP

WIP (Congestion Window)

24 K

16 K

8 K

Time

* This work was supported in part by the U.S. Department of Energy under Contract Number DE-AC03-76SF00098.

The algorithms and ideas described in this paper were developed in collaboration with Mike Karels of the UC Berkeley Computer System Research Group. The reader should assume that anything clever is due to Mike. Opinions and mistakes are the property of the author.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires prior or specific permission.

1988 ACM 0-89791-279-9/88/008/0314

source limits along the path.

In the following sections, we treat each of these in turn.

1 Getting to Equilibrium: Slow-start

Failure (1) has to be from a connection that is either starting or restarting after a packet loss. Another way to think of the conservation property is to say that the receiver generates acks as a 'clock' to strobe new packets into the network. Since the receiver can generate acks no faster than data packets can get through the network,



dl.acm.org

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this

This paper is a brief description of (i) - (v) and the rationale behind them. (vi) is an algorithm recently devel-

WIP (Congestion Window)

24 K

16 K

8 K

Timeout

Age too high

Reduce WIP

Time

* This work was supported in part by the U.S. Department of Energy under Contract Number DE-AC03-76SF00098.

The algorithms and ideas described in this paper were developed in collaboration with Mike Karels of the UC Berkeley Computer System Research Group. The reader should assume that anything clever is due to Mike. Opinions and mistakes are the property of the author.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires prior specific permission.

1988 ACM 0-89791-279-9/88/008/0314

source limits along the path.

In the following sections, we treat each of these in turn.

1 Getting to Equilibrium: Slow-start

Failure (1) has to be from a connection that is either starting or restarting after a packet loss. Another way to look at the conservation property is to say that the receiver generates acks as a 'clock' to strobe new packets into the network. Since the receiver can generate acks no faster than data packets can get through the network,



dl.acm.org

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this

This paper is a brief description of $(i)-(v)$ and the rationale behind them. (vi) is an algorithm recently devel-

Timeout

Age too high

Reduce WIP

WIP (Congestion Window)

24 K

16 K

8 K

Time

* This work was supported in part by the U.S. Department of Energy under Contract Number DE-AC03-76SF00098.

The algorithms and ideas described in this paper were developed in collaboration with Mike Karels of the UC Berkeley Computer System Research Group. The reader should assume that anything clever is due to Mike. Opinions and mistakes are the property of the author.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires prior specific permission.

1988 ACM 0-89791-279-9/88/008/0314

source limits along the path.

In the following sections, we treat each of these in turn.

1 Getting to Equilibrium: Slow-start

Failure (1) has to be from a connection that is either starting or restarting after a packet loss. Another way to look at the conservation property is to say that the receiver generates acks as a 'clock' to strobe new packets into the network. Since the receiver can generate acks no faster than data packets can get through the network,



Age and WIP control congestion on the internet

dl.acm.org

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this

This paper is a brief description of (i) - (v) and the rationale behind them. (vi) is an algorithm recently devel-

Timeout
Age too high

Reduce WIP

* This work was supported in part by the U.S. Department of Energy under Contract Number DE-AC03-76SF00098.

The algorithms and ideas described in this paper were developed in collaboration with Mike Karels of the UC Berkeley Computer System Research Group. The reader should assume that anything clever is due to Mike. Opinions and mistakes are the property of the author.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

1988 ACM 0-89791-279-9/88/008/0314

source limits along the path.

In the following sections, we treat each of these in turn.

1 Getting to Equilibrium: Slow-start

Failure (1) has to be from a connection that is either starting or restarting after a packet loss. Another way to look at the conservation property is to say that the receiver generates acks as a 'clock' to strobe new packets into the network. Since the receiver can generate acks no faster than data packets can get through the network,

Resilience in a

System of Microservices



Resilience in a **System** of Microservices

*"Failure of a computer
you didn't even know existed can
render your own computer
unusable"*

-Leslie Lamport





“Everything Fails. All the Time.”



“Everything Fails. All the Time.”

- Bursts of traffic
- Slow dependencies



“Everything Fails. All the Time.”

- Bursts of traffic
- Slow dependencies
- Latency bugs



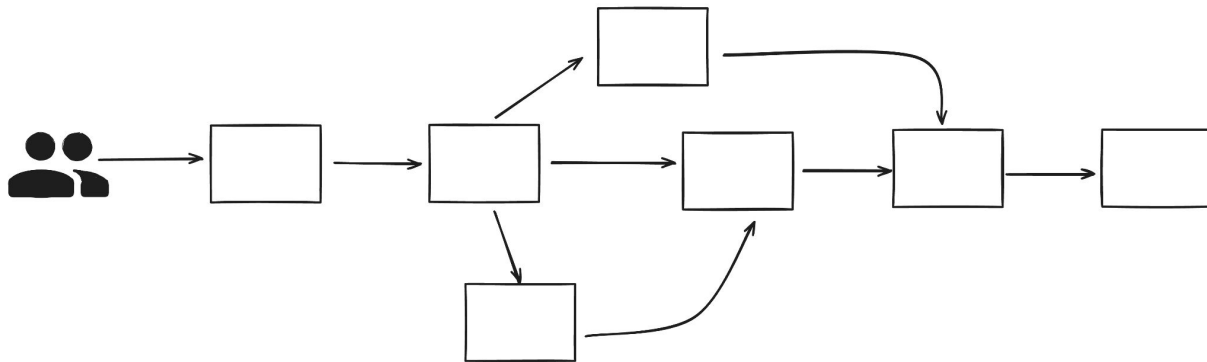
“Everything Fails. All the Time.”

- Bursts of traffic
- Slow dependencies
- Latency bugs
- Background processes
- ...



“Everything Fails. All the Time.”

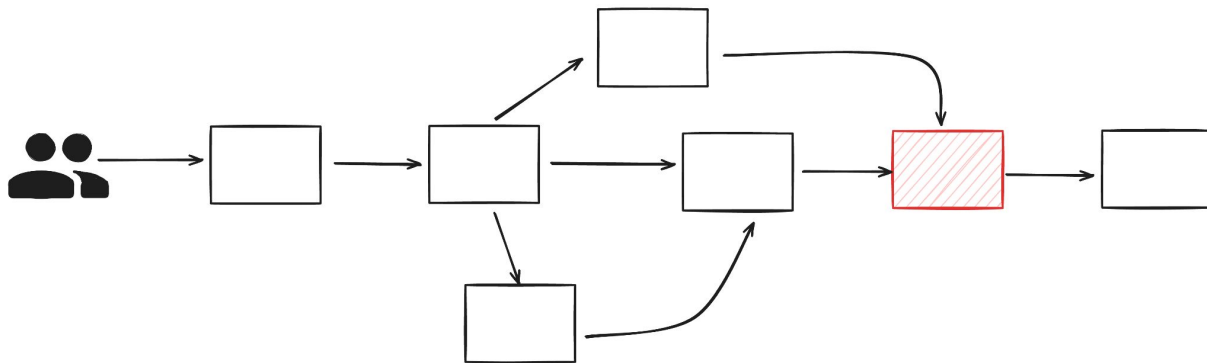
- Bursts of traffic
- Slow dependencies
- Latency bugs
- Background processes
- ...





“Everything Fails. All the Time.”

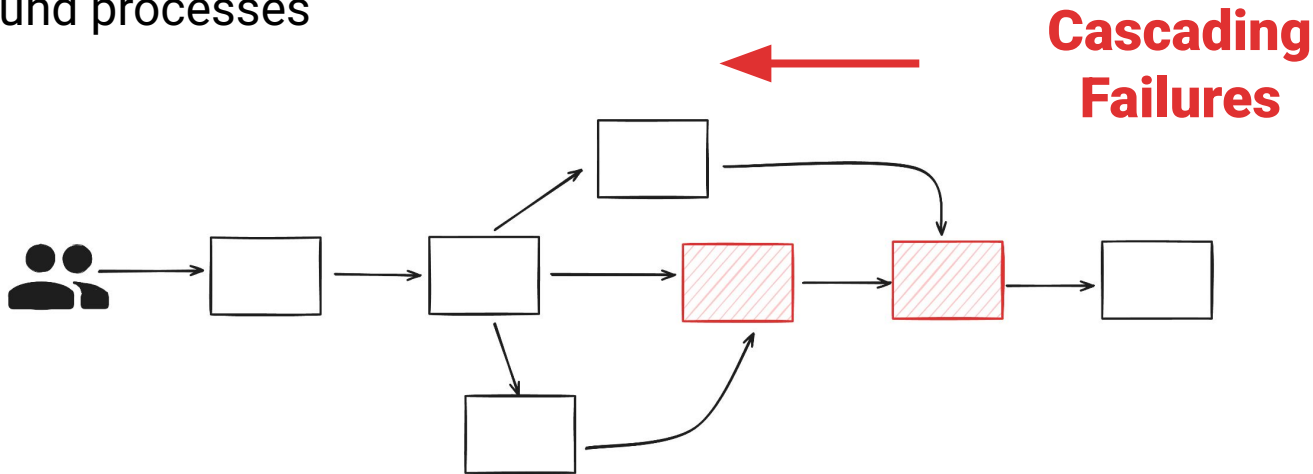
- Bursts of traffic
- Slow dependencies
- Latency bugs
- Background processes
- ...





“Everything Fails. All the Time.”

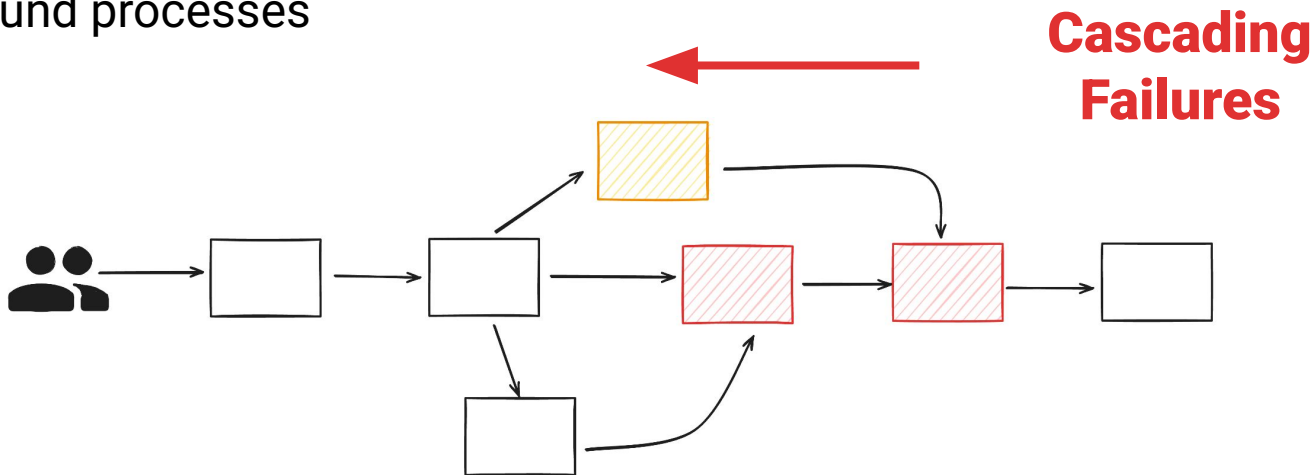
- Bursts of traffic
- Slow dependencies
- Latency bugs
- Background processes
- ...





“Everything Fails. All the Time.”

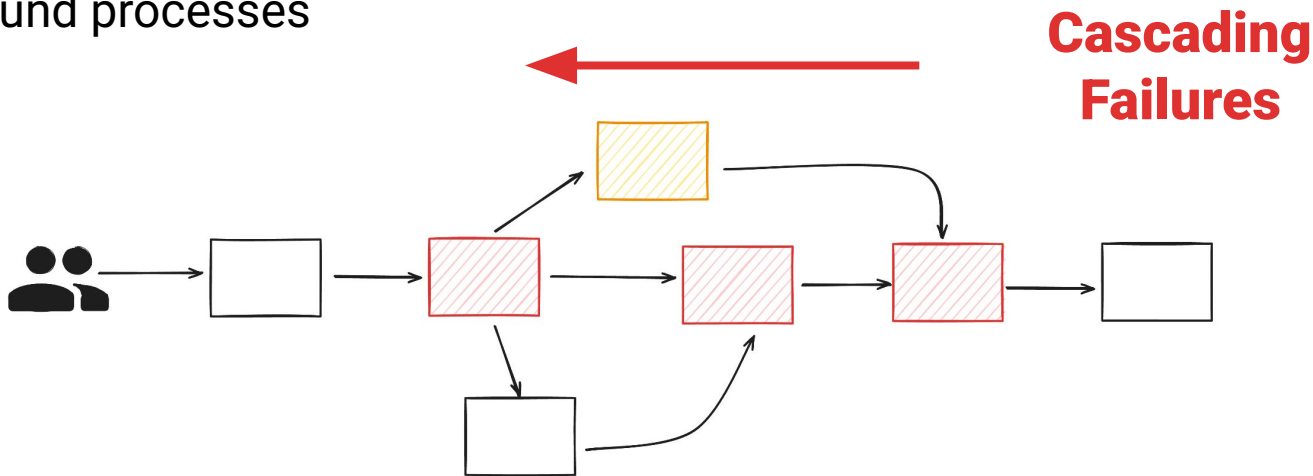
- Bursts of traffic
- Slow dependencies
- Latency bugs
- Background processes
- ...





“Everything Fails. All the Time.”

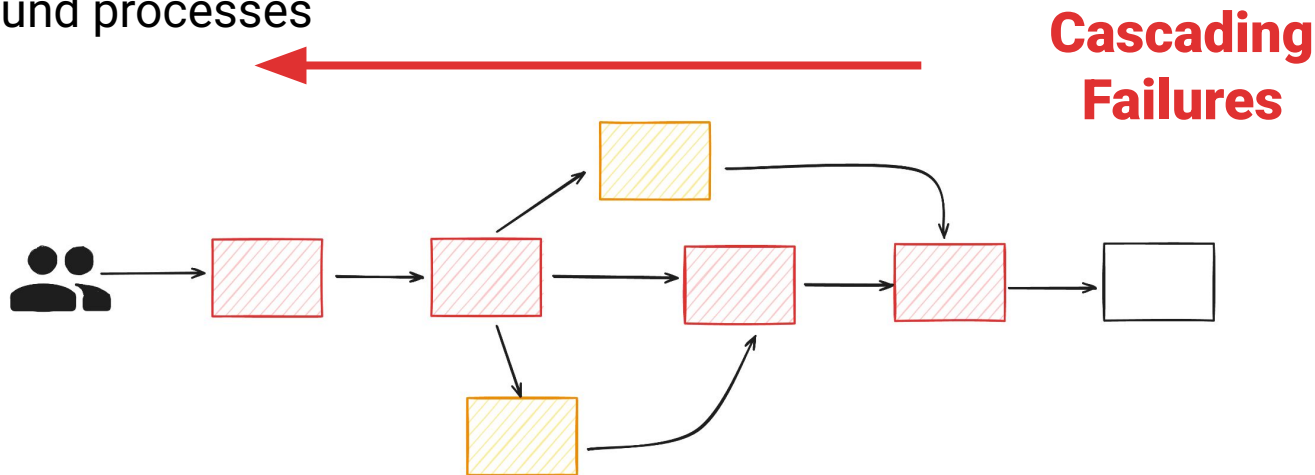
- Bursts of traffic
- Slow dependencies
- Latency bugs
- Background processes
- ...





“Everything Fails. All the Time.”

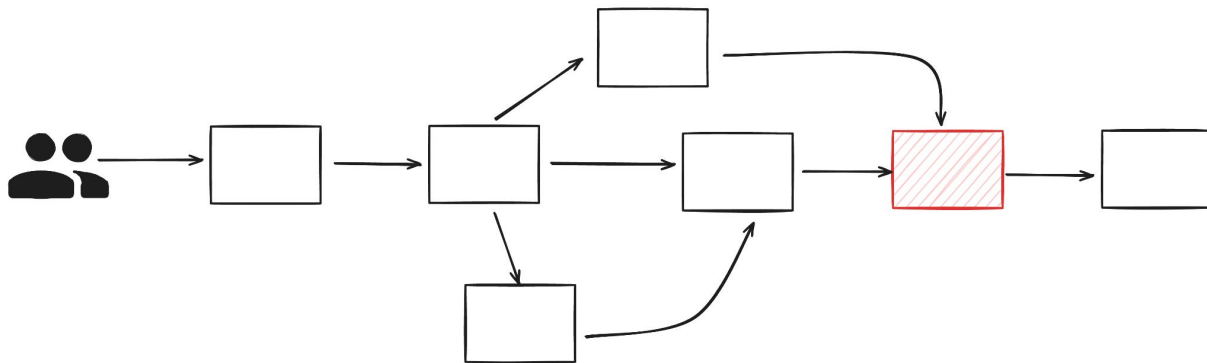
- Bursts of traffic
- Slow dependencies
- Latency bugs
- Background processes
- ...





“Everything Fails. All the Time.”

- Bursts of traffic
- Slow dependencies
- Latency bugs
- Background processes
- ...

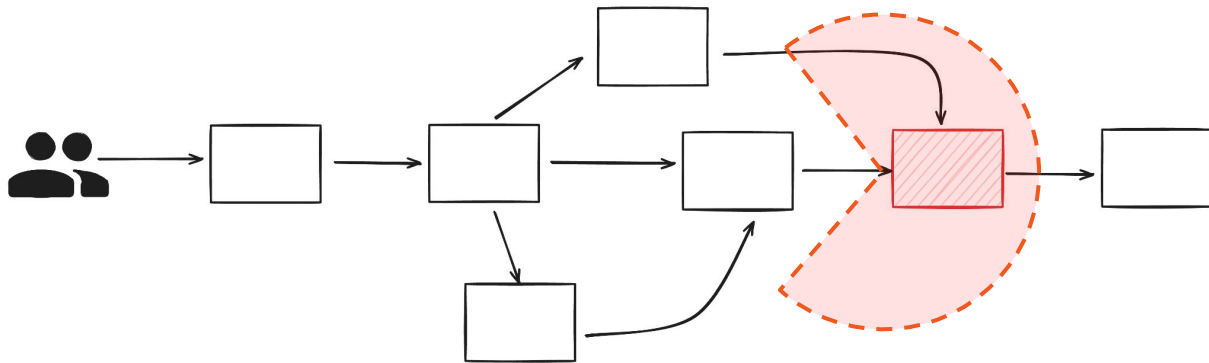




“Everything Fails. All the Time.”

- Bursts of traffic
- Slow dependencies
- Latency bugs
- Background processes
- ...

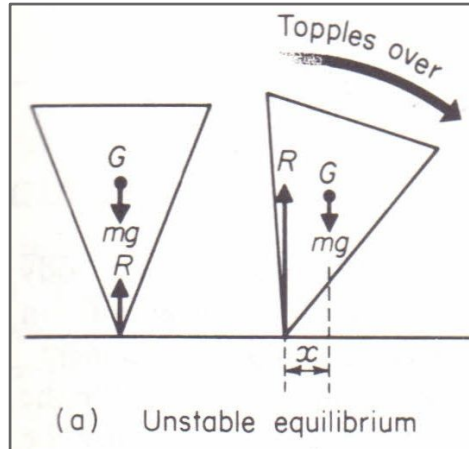
**Limit the radius
of a failure**



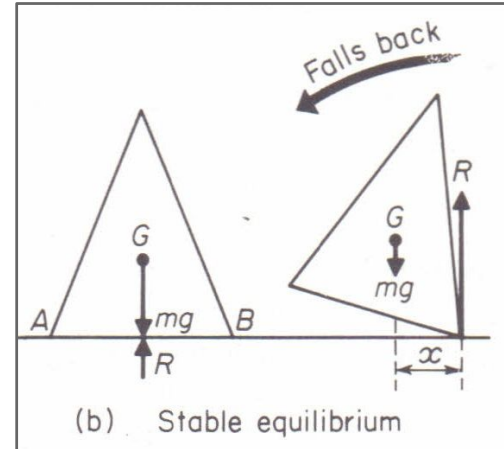
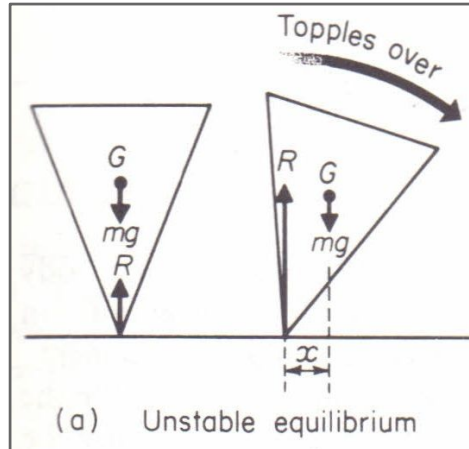


Resilience **By** **Design**

Resilience **By** **Design**

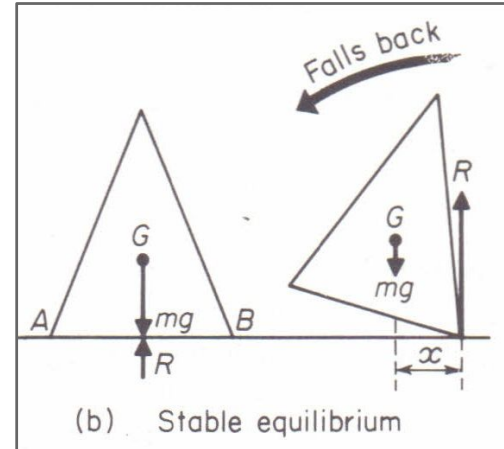
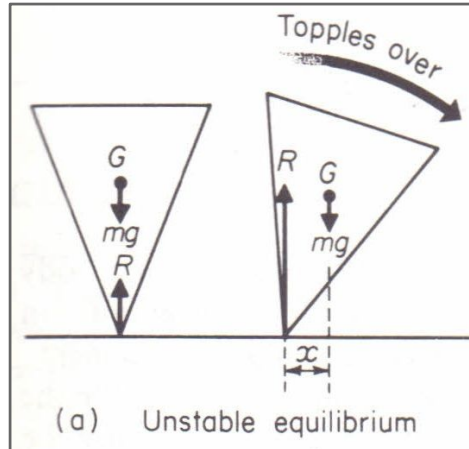


Resilience By Design



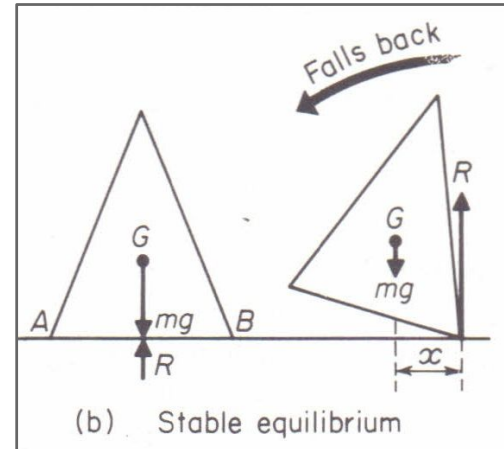
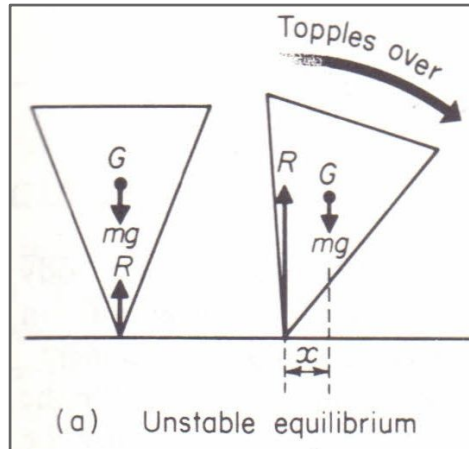
Resilience By Design

- **Measure what matters** to the customer
 - Flow of requests



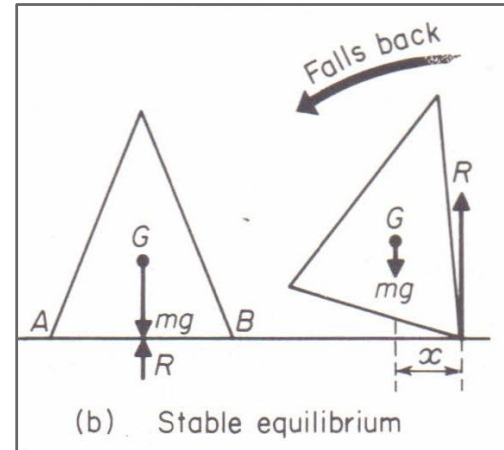
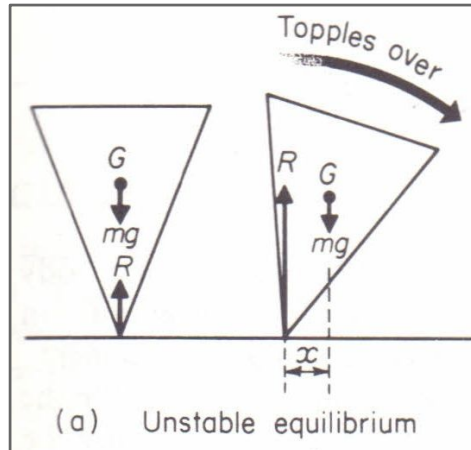
Resilience By Design

- **Measure what matters** to the customer
 - Flow of requests
- With **system agnostic** metrics
 - Locally measurable, universally applicable



Resilience By Design

- **Measure what matters** to the customer
 - Flow of requests
- With **system agnostic** metrics
 - Locally measurable, universally applicable
- Recognize the **inability to keep up with demand**



Thank You!

Resilience in Microservices With

Congestion Control



Mourjo Sen

September 16, 2025

<https://mourjo.me>